マルチクラウド自動デプロイシステム - AIエージェント完全ガイド

AWS / Azure / GCP 統合運用ドキュメント

multicloud-auto-deploy プロジェクト

2026年2月

- 00 — 重要ルール：最初に必ず読むこと
  - このドキュメントを読んだ後
  - ルール0 — すべてのドキュメントは日本語版と英語版を併記
  - ルール1 — ファイル操作：単一バッチで大量のファイルを作成・編集しない
  - ルール2 — Dev Container は ARM (aarch64) で実行；本番では x86_64
  - ルール3 — `main` ブランチ = 即座に本番環境にデプロイされる
  - ルール4 — `AUTH_DISABLED` はステージングと本番環境では常に `false` でなければならない
  - ルール5 — Lambda / Cloud Run の環境変数は Pulumi 出力から取得し、GitHub シークレットから取得しない
  - ルール6 — AWS ステージングフロントエンドは `VITE_BASE_PATH=/sns/` でビルドしなければならない
  - ルール7 — S3 画像バケットはプライベート：常に署名付き GET URL を返す
  - ルール8 — AWS 本番環境 CloudFront：任意の `pulumi up` の前に Pulumi 設定を指定する

- ルール9 — Cognito `id_token` 検証：`verify_at_hash: False` を設定する
- ルール10 — GCP：デプロイ ZIP 内で常に `function.py` を `main.py` としてコピーする
- ルール11 — GCP：`generate_signed_url()` は `service_account_email` + `access_token` が必要
- ルール12 — GCP：Firebase 認可ドメインは Identity Toolkit API を介して登録する必要がある
- ルール13 — GCP：CDN は `Cross-Origin-Opener-Policy: same-origin-allow-popups` を送信する必要がある
- ルール14 — Azure Functions：CORS は Python コードではなく、プラットフォームレベルで設定する必要がある
- ルール15 — Azure Functions：`host.json` は `routePrefix: ""` を設定する必要がある
- クイックリファレンス：何がどこにあるか

- 01 — コンテキスト：概要とリポジトリレイアウト
  - このプロジェクトは何か
  - ライブエンドポイント（ステージング）
    - AWS (ap-northeast-1)
    - Azure (japaneast)
    - GCP (asia-northeast1)
  - 技術スタックサマリー
  - リポジトリディレクトリツリー
  - クイックファイルリファレンス
  - 開発環境
    - ホストマシン
    - Dev Container

        - ARM ビルド警告
        - ローカル開発
        - クラウド認証情報（ホストから自動マウント）
    - ブランチ戦略
    - 次のセクション
- 02 — アーキテクチャ
    - システム概要
    - ストレージパス構造
    - AWS アーキテクチャ詳細
    - Azure アーキテクチャ詳細
    - GCP アーキテクチャ詳細
    - API ルート
    - バックエンドクラウド自動検出ロジック
    - 認証フロー
    - セキュリティ設定ステータス
    - 次のセクション
- 03 — API とデータモデル
    - API エンドポイント
    - リクエスト / レスポンス スキーマ
        - POST /posts — リクエストボディ
        - GET /limits — レスポンス
        - GET /posts — レスポンス
        - Post オブジェクト
        - GET /posts クエリパラメータ
        - POST /uploads/presigned-urls — リクエストボディ

        - POST /uploads/presigned-urls — レスポンス
    - データモデル（DynamoDB Single-Table Design）
        - テーブル： `{project}-{stack}-posts` （AWS）/ `simple-sns-local` （ローカル）
        - GSI： `PostIdIndex`
    - バックエンド実装クラス
        - バックエンド選択
    - ローカル開発環境
        - 必要な環境変数（docker-compose で自動設定）
    - テスト実行
    - 次のセクション
- 04 — インフラストラクチャ（Pulumi）
    - Pulumi 概要
    - 一般的な操作
    - AWS Pulumi スタック
        - リソース一覧
        - 主要な設定キー
        - 主要な出力
    - Azure Pulumi スタック
        - リソース一覧
        - 主要な設定キー
        - 主要な出力
    - GCP Pulumi スタック
        - リソース一覧

        - 主要な設定キー
        - 主要な出力
        - GCS リソース競合（Error 409 / Error 412）
        - Azure CLI 認証エラー
        - Azure Pulumi 保留中の操作
    - Lambda Layer 設定
    - Lambda Layer ビルドステップ
    - 次のセクション
- 05 — CI/CDパイプライン
    - ワークフロー一覧
    - ブランチ → 環境マッピング
    - トリガー条件 (パスフィルタ)
    - クラウド設定ファイル (.github/config/)
        - 保存先
        - 保存内容
        - ワークフローでの使用方法
        - 更新時期
        - GitHub Secrets に保存しない理由
    - 必要な GitHub Secrets
        - AWS
        - Azure
        - GCP
    - デプロイフロー (AWS バックエンド例)
    - デプロイフロー (フロントエンド例)
    - 手動デプロイ (緊急時)

- 手動デプロイ (緊急時)
- CI/CD 現在ステータス (2026-02-24)
- 修正済み CI バグ (再発を監視)
- バージョン管理
  - 現在のバージョン (2026-02-24)
  - バージョンインクリメント規則
  - 実装ファイル
  - セットアップ (初回のみ)
  - よく使うコマンド
  - バージョンバンプをスキップ
- 次のセクション

- 06 — 環境ステータス
  - セッション 2026-02-27 (継続 4): アーキテクチャ図アイコン強化
    - 完了作業
    - アイコン配置戦略
    - 生成ファイル
    - ドキュメント更新
  - セッション 2026-02-27 (継続 3): セキュリティデプロイ・ドキュメント更新
    - 完了作業
    - 本番エンドポイント (2026-02-27 現在)
    - セキュリティ実装ステータス (本番環境にデプロイ済み)
  - セッション 2026-02-27: GCP 監査ログ・課金予算修復
    - 完了作業
    - コード変更
    - 既知の課題 / 次のステップ

- ステージング環境概要
- AWS (ap-northeast-1)
- Azure (japaneast)
- GCP (asia-northeast1)
- 接続確認コマンド
- 本番環境
  - 本番ステータス概要 (2026-02-23)
  - 本番エンドポイント
  - カスタムドメインステータス (ashnova.jp) — 2026-02-21
  - ☒Production Issues — 全件解決済み (2026-02-24 v5)
- E2E テストスクリプト (2026-02-24)
  - `test-sns-all.sh` (新規)
  - 各スクリプトの改良内容
  - テスト一覧 (write モード時の追加項目)
- コスト監視ツール
  - CLI レポート (`scripts/cost_report.py`)
  - macOS メニューバーウィジェット (`scripts/mac-widget/`)
  - 通貨処理
  - .env 設定ファイル
  - CLI レポート (`scripts/cost_report.py`)
  - macOS メニューバーウィジェット (`scripts/mac-widget/`)
  - 通貨処理
  - .env 設定ファイル
- AWS 管理コンソールリンク
- Azure ポータルリンク

- GCP コンソールリンク
- FinOps — GCP 未使用静的 IP アドレス監査 (2026-02-21)
  - すべての IP アドレス
  - 未使用 IP の背景
  - 解放コマンド
- FinOps — GCP 未使用 Cloud Storage バケット監査 (2026-02-21)
  - すべてのバケット (プロジェクト: ashnova)
  - 削除可能バケットの背景
  - おまけ: 壊れた Cloud Function（関連リソース）
  - 削除コマンド
- 次のセクション

- 07 — ランブック
  - [RB-01] Lambda 関数を手動更新する
  - [RB-02] Lambda ログを確認する
  - [RB-03] React フロントエンドを手動デプロイする（AWS）
  - [RB-04] ランディングページを手動デプロイする（AWS）
  - [RB-05] Azure Functions を手動デプロイする
  - [RB-06] GCP Cloud Run を手動デプロイする
  - [RB-07] Pulumi スタックを再デプロイする
  - [RB-08] API エンドポイントテストを実行する
  - [RB-09] DynamoDB PostIdIndex GSI を確認 / 作成する
  - [RB-11] API Lambda を再ビルドしてホットフィックスデプロイする（CI なし）
  - [RB-12] AWS simple-sns E2E テストを実行する
  - [RB-10] ローカル開発環境を起動する
    - 環境変数の上書き（デバッグ用）

    - ARM に関する注意
  - [RB-13] GCP Cloud Function を再ビルドしてホットフィックスデプロイする（linux/amd64）
  - [RB-14] Azure Front Door 502 を修正する（Dynamic Consumption → FC1 FlexConsumption）
  - [RB-15] AWS CloudFront HTTPS 証明書エラーを修正する（ERR_CERT_COMMON_NAME_INVALID）
  - [RB-16] Azure FC1 InaccessibleStorageException を修正する（デプロイ用ストレージアカウント削除時）
  - [RB-17] マルチクラウドのコスト監視
    - ターミナルでフルレポートを出す
    - 認証情報の設定
    - 為替レート
    - macOS メニューバーウィジェット（xbar）
    - ファイル構成
  - 次のセクション
- 08 — セキュリティ
  - セキュリティ設定現況
  - 認証設定
    - AWS — Amazon Cognito
    - Azure — Azure AD
    - GCP — Firebase Auth
  - IAM 最小権限ポリシー
    - AWS Lambda 実行ロール
    - Azure サービスプリンシパル (pulumi-deploy)
    - GCP — サービスアカウント (github-actions-deploy)

- シークレット管理
- 未構築のセキュリティ課題（優先度順）
  - Defender for Cloud — 対応困難な項目（現行アーキテクチャの制約）
- セキュリティヘッダ（AWS CloudFront — 確認済み 2026-02-23）
- S3 セキュリティ (AWS — フロントエンドバケット)
- Azure CORS 設定（重要 ー Azure に触れる前に必読）
  - Layer 1 — Function App platform CORS (controls API requests)
  - Layer 2 — Blob Storage CORS (controls image uploads via SAS URL)
  - Summary
- 監査ログ設定（2026-02-24 実装）
  - AWS — CloudTrail
  - GCP — Cloud Audit Logs
  - Azure — Log Analytics Workspace
- ID・アクセス管理（IAM/RBAC）
  - 基本原則 ー 最小権限の法則（Principle of Least Privilege）
  - AWS IAM 権限構成 (2026-02-27 更新)
  - Azure RBAC 権限構成 (2026-02-27 更新)
  - GCP IAM 権限構成 (2026-02-27 更新)
  - ポリシー定義詳細
  - チェックリスト ー 権限設定検証
- 次セクション

- 09 — 残タスク
  - ステータス概要（Updated 2026-02-27 Session 5）
  - ☒高優先度タスク（2026-02-27 Session 3 更新）
    - ☒2026-02-27 Session 3 で解決済み
    - ☒2026-02-27 Session 2 で解決済み
  - 2026-02-24 セッションで解決済みの問題
  - 2026-02-23 セッションで解決済みの問題
  - 2026-02-27 セッションで解決済みの内容
  - ☒中優先度タスク
  - ☒低優先度タスク
  - 推奨作業順序
  - 解決済みタスク（履歴）
- 10 — カスタムドメイン・テスト
  - カスタムドメインステータス (全て確認済み 2026-02-21)
    - ステージング CDN エンドポイント (カスタムドメインなし)
  - ⚠ 重要: 再実行前の Pulumi 設定
  - DNS レコード (確認済み)
    - AWS
    - Azure
    - GCP
  - 再セットアップ手順 (ドメイン再構成が必要な場合)
    - AWS
    - Azure
    - GCP
  - ドメイン変更後の CORS

- 検証
- 全環境のテスト
  - クイック接続確認 (~30秒、認証不要)
  - 全認証テスト (全3クラウド)
  - 認証トークンの取得
  - クラウドごとのテストスクリプト
  - pytest 統合テスト (unit/integration、モック)
- 次のセクション

- 11 — バグ・修正レポート
  - 概要
  - 目次
  - 1. AWS SNS 修正レポート（2026-02-20）
    - Key Fixes
    - Key Lesson
  - 2. AWS Production HTTPS Fix (2026-02-21)
    - Symptom
    - Root Cause
    - Fix (Immediate)
    - Fix (Permanent — run before every `pulumi up --stack production`)
  - 3. AWS Production SNS Fix (2026-02-21)
    - Root Cause Detail — Bug 1
    - Root Cause Detail — Bug 2
    - Key Lessons

- 4. React SPA Migration & CDN Routing Fix (2026-02-21)
  - Background
  - CDN Routing Bugs Fixed
  - AWS Additional Bug — S3 directory index
  - CI/CD Workflow Changes
  - Cache-Control Pattern (AWS/GCP)
- 5. Azure SNS Fix (2026-02-21)
  - Part A — 503/404 Initial Setup Bugs
  - Part B — AFD 502 Intermittent Errors (Production)
  - Key Rules for Azure Deployment
- 6. AWS SNS Fix Report (2026-02-22)
  - Key Technical Points
- 7. AWS + Azure Combined SNS Fix (2026-02-22)
  - AWS Bug — Cognito `unauthorized_client`
  - Azure Bugs Fixed
  - CI/CD Changes (Azure)
- 8. Refactoring & Infrastructure Fix (2026-02-22)
  - 8-1. Azure Front Door — SPA URL Rewrite Rule
  - 8-2. CI/CD Workflow Cleanup
  - 8-3. AWS Pulumi Dead Code Removal
  - 8-4. Staging Bug: `ModuleNotFoundError: pulumi_azuread`
  - 8-5. Staging Bug: `ModuleNotFoundError: monitoring`
  - 8-6. GCP: `Error 412: Invalid fingerprint` on URLMap
  - 8-7. GCP: `Error 409: bucket already exists`
  - 8-8. GCP: `Error 400: ssl_certificate already in use`

- 9. GCP SNS Fix Report (2026-02-23)
  - Key Technical Points
- 10. OCR Formula Merge Bugs (2026-02-27)
  - Background
  - Heuristic Formula Region Detection (fallback for Bug 3)
  - Merge Strategy (final)
  - Known Minor Issues

- 12 — OCR・数式解答サービス
  - 概要
  - 機能トグル
  - API エンドポイント
    - POST /v1/solve
    - GET /v1/ocr-debug?limit=20
    - GET /v1/ocr-debug/diag
  - OCR ソースとスコアリング
  - 2パス数式マージ（azure_di_merged）
  - 環境変数
    - Azure Solver（AzureMathSolver）
    - GCP Solver（GcpMathSolver）
    - コスト制御用変数
  - 参照 PDF 検索
  - ソースファイル
  - クイックテスト
  - AWS 非対応の理由
    - 1. AWS Textract に LaTeX 数式抽出機能がない（最大の障壁）

        - 2. API Gateway の 29 秒応答タイムアウト
        - 3. gpt-4o と Gemini は AWS Bedrock で提供されていない
        - 4. 設計経緯 — 以前は Bedrock フォールバックがあった
        - 5. まとめ — AWS で solve を動かすために必要な変更
    - 既知の制約と今後の改善
- 13 — テスト
    - テストマップ
    - クイックコマンド
        - 1 — 最速: 全クラウドヘルスチェック
        - 2 — E2E スモークテスト (全3クラウド、パブリックエンドポイント)
        - 3 — クラウド別完全 SNS テスト (パブリックエンドポイントのみ)
        - 4 — 認証付き完全 SNS テスト
        - 5 — ローカル docker-compose スタックテスト
        - 6 — pytest (unit + バックエンドモックテスト — ネットワーク不要)
        - 7 — デプロイ済みライブ API に対する pytest
    - トークン取得
        - AWS — Cognito アクセストークン
        - Azure — Azure AD id_token
        - GCP — Firebase id_token
    - ローカル Docker Compose スタック
    - pytest マーカー
    - ステージングエンドポイント (リファレンス)
    - CI/CD テスト統合

- トラブルシューティング

00 — 重要ルール：最初に必ず読むこと

> このドキュメントはリポジトリ内の他のドキュメントを読む前に必ず読むこと。 データロス・本番環境障害・数時間のデバッグを招く最小限のルールセットが含まれている。 ここに書かれたすべての項目は過去のインシデントから学んだ教訓である。

このドキュメントを読んだ後

以下の14個のルールをすべて読んだら、この順序で続行する：

```
1. AI_AGENT_01_CONTEXT.md    ← プロジェクトの概要、ライブエンドポイント、使用技術
2. AI_AGENT_02_ARCHITECTURE.md ← ディレクトリツリー、各ファイルの配置（5分）
3. AI_AGENT_03_API.md ← システムトポロジー、ストレージパスレイアウト（5分）
4. AI_AGENT_06_STATUS.md      ← 3つのクラウド環境すべての現在の健全性
5. AI_AGENT_09_TASKS.md       ← 優先度付きバックログ — 次に何をするか
```

タスクで必要な場合だけこれらを読む：

```
AI_AGENT_04_INFRA.md     ← エンドポイント仕様、リクエスト/レスポンススキーマ、デー
AI_AGENT_05_CICD.md      ← Pulumiスタック、リソース名、設定キー、スタック出力
AI_AGENT_07_RUNBOOKS.md ← GitHub Actionsワークフロー、必要なシークレット、トリ
AI_AGENT_08_SECURITY.md ← ステップバイステップ：Lambda デプロイ、React デプロ
```

ルール0 — すべてのドキュメントは日本語版と英語版を併記

すべてのソースコードコメント、コミットメッセージ、プルリクエスト説明、GitHub Actionsワークフローコメント、Pulumiスタックコメント、ドキュメントファイルは 英語で記述しなければならない（本版）。

このマルチクラウドインフラプロジェクトは異なるチームのエンジニアがレビューする可能性がある。英語での一貫性により、すべての自動ツール（検索、grep、AIエージェント）がコンテンツを正しく処理できる。

ただし、日本語ユーザー向けの解説書や日本語訳ドキュメント（このファイルなど）は日本語で記述してもよい。

ルール1 — ファイル操作：単一バッチで大量のファイルを作成・編集しない

単一のツール呼び出しバッチで大量のファイルを作成・編集すると、ネットワークエラーやタイムアウトが発生する可能性がある。部分的な結果が無声のうちに破損し、デバッグが非常に困難になる。

常に段階的に作業する：

1. 一度に数個のファイルだけを作成または編集する。
2. 各ステップを検証してから続行する（コンパイルチェック、構文チェック、テスト）。
3. 途中でネットワークエラーが発生した場合、既に作成されたものを確認してから再試行する。

ルール2 — **Dev Container** は **ARM (aarch64)** で実行；本番では **x86_64**

VS Code dev containerは `linux/aarch64` 用に構築されている。すべてのクラウドランタイム（Lambda、Azure Functions、Cloud Run、Cloud Functions）は `linux/amd64` で実行される。**aarch64**上でネイティブにコンパイルされた**Python**パッケージはバイナリ互換性がなく、実行時に `.so` インポートエラーでクラッシュする。

デプロイ用に**Python**パッケージをインストールする場合、常に**Docker** を使用し、ターゲットプラットフォームを明示的に設定する：

```
docker run --rm --platform linux/amd64 ¥
  -v /tmp/deploy:/out python:3.12-slim ¥
  bash -c "pip install --no-cache-dir --target /out -r requirements.txt -q"
```

これは以下に適用される：

- Lambda ZIP パッケージング（ `requirements-aws.txt` , `requirements-layer.txt` ）
- GCP Cloud Functions ZIP パッケージング（ `requirements-gcp.txt` ）
- Azure Functions デプロイ（ `requirements-azure.txt` ）

dev container 内で `pip install` をホストマシンで実行してデプロイアーティファクトを生成しない。

ルール3 — `main` ブランチ = 即座に本番環境にデプロイされる

```
develop  →  staging（ステージング）
main     →  production（本番）← push後すぐに本番環境で実行される
```

本番環境にすぐにデプロイするつもりがない限り、`main` に直接プッシュしない。 常に `develop`（またはフィーチャーブランチ）で作業し、ステージングが成功したことを確認してから `main` にマージする。

ルール4 — `AUTH_DISABLED` はステージングと本番環境では常に `false` でなければならない

環境変数 `AUTH_DISABLED` は常に `false` でなければならない。過去のインシデントでは、誤って `true` に設定されていたため、すべてのAPIエンドポイントに対して認証なしでアクセスできていた。

CI ワークフロー、Pulumi 設定、`.env` ファイルのどこでも `AUTH_DISABLED=true` を見つけた場合、重大なバグとして扱い、直ちに修正する。

ルール5 — Lambda / Cloud Run の環境変数は Pulumi 出力から取得し、GitHub シークレットから取得しない

過去のバグにより、対応する GitHub シークレットが本番環境に存在しなかったため、`API_GATEWAY_ENDPOINT` 値が無声のうちに空になった。

従うべきパターン（`deploy-*.yml` 内）：

```
        - name: Get Pulumi outputs
          run: |
            cd infrastructure/pulumi/aws
            echo "API_URL=$(pulumi stack output api_url --stack
$STACK)" >> $GITHUB_ENV
```

`pulumi stack output` から導出できる値には `${{ secrets.API_GATEWAY_ENDPOINT }}` のような GitHub シークレットを決して使用しない。シークレットが存在しない場合、変数は無声のうちに空の文字列になり、Lambda/Function はハードコードされたフォールバック（しばしば `localhost:8000` ）を使用して起動する。

ルール6 — AWS ステージングフロントエンドは `VITE_BASE_PATH=/sns/` でビルドしなければならない

SNS React アプリは `s3://bucket/sns/` にデプロイされる。`VITE_BASE_PATH=/sns/` なしでビルドすると、すべての JS/CSS アセットが `/assets/...` を参照し、404 が発生する。CloudFront はすべてのア

セットリクエストに対して `Content-Type: text/html` で `index.html` を配信し、MIME タイプエラーでアプリ全体が破損する。

```
cd services/frontend_react
set -a && source .env.aws.staging && set +a
VITE_BASE_PATH=/sns/ npm run build
```

CloudFront カスタムエラーページも `/sns/index.html` を指す必要があり、`/index.html` ではない。

ルール7 — S3 画像バケットはプライベート：常に署名付き GET URL を返す

`multicloud-auto-deploy-staging-images` はすべてのパブリックアクセスがブロックされている。バックエンドはフロントエンドに投稿を返す前に `_resolve_image_urls()` を呼び出す必要がある。これは保存された S3 キー（`imageKeys`）を有効期限1時間の署名付き GET URL に変換する。

生の S3 キーを返すか、署名付き URL をデータベースにキャッシュすると、1時間後に破損した画像リンクが得られる。

ルール8 — AWS 本番環境 CloudFront：任意の `pulumi up` の前に Pulumi 設定を指定する

以下の設定がないと、`pulumi up --stack production` は CloudFront をデフォルト証明書に戻し、すべての HTTPS ビジターに対して `NET::ERR_CERT_COMMON_NAME_INVALID` が発生する。

```
        cd infrastructure/pulumi/aws
        pulumi config set customDomain www.aws.ashnova.jp --stack
production
        pulumi config set acmCertificateArn ¥
          arn:aws:acm:us-east-1:278280499340:certificate/
914b86b1-4c10-4354-91cf-19c4460dcde5 ¥
          --stack production
```

本番 AWS スタックで `pulumi up` を実行する前に、これら2つの設定値が存在することを常に確認する。

ルール9 — Cognito `id_token` 検証： `verify_at_hash: False` を設定する

フロントエンドが `id_token` のみを送信する場合（エスコート `access_token` なし）、JWT ライブラリは `at_hash` を計算または検証できない。`jwt_verifier.py` への呼び出しは `verify_at_hash=False` を渡さなければならない。この設定を戻すと、すべての認証付き API 呼び出しが破損する。

参照： `services/api/app/jwt_verifier.py`

ルール10 — GCP：デプロイ ZIP 内で常に `function.py` を `main.py` としてコピーする

`--entry-point` が別の関数に名前を付けている場合でも、Cloud Build は `missing main.py` で失敗する。修正方法はデプロイソースに `main.py` を含めることである：

```
        cp services/api/function.py /tmp/deploy_gcp/.deployment/main.py
```

常に `function.py` と `main.py` の両方を ZIP に含める。これを怠ると、Cloud Build はソースを拒否し、関数は更新されない。

ルール11 — GCP：`generate_signed_url()` は `service_account_email` + `access_token` が必要

Cloud Run と Cloud Functions は Compute Engine 認証情報を使用する（秘密鍵なしのアクセストークンのみ）。追加の引数なしで `blob.generate_signed_url()` を呼び出すと、以下が発生する：

```
AttributeError: you need a private key to sign credentials
```

正しい呼び出し：

```
        blob.generate_signed_url(
            expiration=3600,
```

```
            method="GET",
            service_account_email=settings.gcp_service_account,
            access_token=credentials.token,
        )
```

サービスアカウントメールは Cloud Run サービスで環境変数 `GCP_SERVICE_ACCOUNT` を介して提供される。SA は `roles/iam.serviceAccountTokenCreator` も持つ必要がある。

ルール12 — GCP：Firebase 認可ドメインは Identity Toolkit API を介して登録する必要がある

新しい Cloud Run URL またはカスタムドメインが追加される場合、Firebase Auth に明示的に登録する必要がある。Identity Toolkit Admin v2 `PATCH` エンドポイントをヘッダー `x-goog-user-project: PROJECT_ID` で使用する — このヘッダーを省略すると、有効な管理者トークンがあっても `403 PERMISSION_DENIED` が返される。

このステップは `deploy-gcp.yml`（`Update Firebase Authorized Domains`）で自動化されている。これを削除またはスキップしない。

ルール13 — GCP：CDN は `Cross-Origin-Opener-Policy: same-origin-allow-popups` を送信する必要がある

このレスポンスヘッダーがないと、Firebase `signInWithPopup` は `popup.closed` をチェックできず、COOP 警告を繰り返し、厳格なブラウザー環境でのログインフローを潜在的に破損する。

これを CDN バックエンドバケットカスタムレスポンスヘッダーとして設定する（Pulumi または `gcloud` 経由）。 変更後、CDN キャッシュを無効化する：

```
gcloud compute url-maps invalidate-cdn-cache <URL_MAP_NAME> ¥
  --path "/*" --project ashnova
```

ルール14 — Azure Functions：CORS は Python コードではなく、プラットフォームレベルで設定する必要がある

Azure Functions（Flex Consumption）は **Python** ランタイムの前に **Kestrel**（**.NET HTTP** サーバー）を実行する。Kestrel がすべての `OPTIONS` プリフライトリクエストを Python コードに到達する前に傍受する。FastAPI CORS ミドルウェアで `Access-Control-Allow-Origin` を設定しても、効果がない — Kestrel がミドルウェアに到達する前にレスポンスを返す。

常に **Azure CLI** または **Portal** 経由で **CORS** を設定する：

```
# Function App CORS（ブラウザからの API コールを制御）
az functionapp cors add ¥
  --resource-group "$RESOURCE_GROUP" ¥
  --name "$FUNCTION_APP_NAME" ¥
  --allowed-origins "https://your.domain.com"

# 最初にワイルドカードを削除する（ワイルドカードは個別オリジンを抑制する）
az functionapp cors remove ¥
  --resource-group "$RESOURCE_GROUP" ¥
  --name "$FUNCTION_APP_NAME" ¥
  --allowed-origins '*'
```

Azure Blob Storage CORS は **Function App CORS** から完全に独立している。 画像アップロードは SAS URL 経由で Blob Storage に直接移動する（Function App をバイパス）。 Blob Storage CORS は別途設定する必要がある：

```
az storage cors add ¥
  --account-name "$STORAGE_ACCOUNT" ¥
  --services b ¥
  --methods GET POST PUT DELETE OPTIONS ¥
  --origins "https://your.domain.com" ¥
  --allowed-headers "*" ¥
  --exposed-headers "*" ¥
  --max-age 3600
```

また： `SCM_DO_BUILD_DURING_DEPLOYMENT` は Flex Consumption では サポートされていない ー `InvalidAppSettingsException` をトリガーする。 常に Docker を使用してパッケージをローカルでビルドしてからデプロイする。

ルール15 — Azure Functions：`host.json` は `routePrefix: ""` を設定する必要がある

Azure Functions は デフォルトで `routePrefix: "api"` に設定されており、すべての HTTP トリガーに `/api/` のプリフィックスを付ける。フロントエンドは `/api/` プリフィックスなしで `/posts`、`/health` のようなパスを呼び出す。 この修正がないと、すべての API エンドポイントが 404 を返す。

必須の **`host.json`** コンテンツ：

```
{
  "version": "2.0",
  "extensions": {
    "http": {
      "routePrefix": ""
    }
  },
  "extensionBundle": {
    "id": "Microsoft.Azure.Functions.ExtensionBundle",
    "version": "[4.*, 5.0.0)"
  }
}
```

また：Azure AD `post_logout_redirect_uri` はアプリの `redirect_uris` に明示的に登録する必要がある。 `/sns/auth/callback` コールバック URL のみを登録することは不十分 — ログアウトリターン URL（例：`/sns/`）も追加する必要がある：

```
az ad app update ¥
  --id "$AD_APP_ID" ¥
  --web-redirect-uris ¥
```

```
            "https://your.domain.com/sns/auth/callback" ¥
            "https://your.domain.com/sns/"
```

クイックリファレンス：何がどこにあるか

トピック	ファイル
ライブエンドポイント URL	AI_AGENT_01_CONTEXT.md
リポジトリディレクトリツリー	AI_AGENT_01_CONTEXT.md
システムアーキテクチャ	AI_AGENT_02_ARCHITECTURE.md
API ルート & データモデル	AI_AGENT_03_API.md
Pulumi / IaC	AI_AGENT_04_INFRA.md
CI/CD パイプライン	AI_AGENT_05_CICD.md
現在の環境健全性	AI_AGENT_06_STATUS.md
ステップバイステップランブック	AI_AGENT_07_RUNBOOKS.md
セキュリティ設定	AI_AGENT_08_SECURITY.md
残りのタスク / バックログ	AI_AGENT_09_TASKS.md
	AI_AGENT_GUIDE.md

トピック	ファイル
すべて ー エントリーポイント	

01 ー コンテキスト：概要とリポジトリレイアウト

> 第I部 ー オリエンテーション | 親: AI_AGENT_GUIDE.md

このプロジェクトは何か

multicloud-auto-deploy は、同一のフルスタックアプリケーション（SNSスタイルのメッセージングアプリ）を **AWS**、**Azure**、**GCP** に完全に自動化された CI/CD パイプライン経由で同時にデプロイするプラットフォームである。

- フロントエンド: React 18 + Vite + TypeScript + Tailwind CSS
- バックエンド: FastAPI（Python 3.12）— Lambda / Azure Functions / Cloud Run
- データベース: DynamoDB / Cosmos DB / Firestore（共有論理スキーマ）
- IaC: Pulumi Python SDK
- CI/CD: GitHub Actions

ライブエンドポイント（ステージング）

AWS (ap-northeast-1)

目的	URL
CDN (CloudFront)	`https://d1tf3uumcm4bo1.cloudfront.net`
API (API Gateway)	`https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com`
カスタムドメイン	`https://www.aws.ashnova.jp`

Azure (japaneast)

目的	URL
CDN (Front Door)	`https://mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net`
API (Functions)	`https://multicloud-auto-deploy-staging-func-d8a2guhfere0etcq.japaneast-01.azurewebsites.net`
カスタムドメイン	`https://www.azure.ashnova.jp`

> 注意：Azure API URL は Function App ベース URL であり、パスサフィックスは含まない。 関数はワイルドカードルーム `{*route}` を使用するため、すべての API パス（例： `/health` 、 `/posts` ） がベー

> ス URL で直接配信される。 `/api/HttpTrigger` パスは 使用されない。

GCP (asia-northeast1)

目的	URL
CDN (Cloud CDN)	`http://34.117.111.182`
API (Cloud Run)	`https://multicloud-auto-deploy-staging-api-899621454670.asia-northeast1.run.app`
Web フロントエンド (Cloud Run)	`https://multicloud-auto-deploy-staging-frontend-web-899621454670.asia-northeast1.run.app` (legacy SSR)
カスタムドメイン	`https://www.gcp.ashnova.jp`

> 注意：Cloud Run `frontend-web` サービスは存在するが、CDN は `/sns/*` を GCS バケット（React SPA）にルーティングする ー Cloud Run ではない。Cloud Run URL はレガシーである。

技術スタックサマリー

```
フロントエンド（SNSページ）
  AWS:   React 18.2 / Vite 7.3 / TypeScript / Tailwind CSS  ← S3 内の静的
  Azure: React 18.2 / Vite / TypeScript  ← Blob Storage $web/sns/ 内の静
         (services/frontend_web Python SSR はスーパーセッドされた; CI は現在
  GCP:   React 18.2 / Vite / TypeScript  ← GCS sns/ プリフィックス内の静的
         (Cloud Run frontend-web は存在するが、CDN は Cloud Run ではなく GC
```

```
バックエンド API
  FastAPI 1.0 / Python 3.12 / Pydantic v2
  AWS:   Lambda（x86_64）+ API Gateway v2（HTTP）+ Lambda Layer + Mangum
  Azure: Azure Functions（Python 3.12、FC1 FlexConsumption）
  GCP:   Cloud Run（Python 3.12、gen2）
  ローカル: uvicorn + DynamoDB Local + MinIO

データベース（共有論理スキーマ）
  AWS:   DynamoDB — テーブル: {project}-{stack}-posts
  Azure: Cosmos DB Serverless — db: messages / container: messages
  GCP:   Firestore（Native）— コレクション: messages / posts

インフラストラクチャ
  Pulumi Python SDK 3.x
  状態: Pulumi Cloud（リモート）

認証
  AWS:   Amazon Cognito（Pulumi により自動プロビジョニング）
  Azure: Azure AD（Pulumi により自動プロビジョニング）
  GCP:   Firebase Auth（Google Sign-In、httponly Cookie セッション）
  ステージング: AUTH_DISABLED=false  ← 決して true になってはいけない
```

リポジトリディレクトリツリー

```
multicloud-auto-deploy/              ← ワークスペースルート = git リポジトリ
│
├── .github/
│   └── workflows/                   ← ★ 実際のワークフロー — CI はこれらの
│       ├── deploy-aws.yml
│       ├── deploy-azure.yml
│       ├── deploy-gcp.yml
│       ├── deploy-landing-aws.yml
│       ├── deploy-landing-azure.yml
```

```
│       ├── deploy-landing-gcp.yml
│       ├── deploy-frontend-web-aws.yml
│       ├── deploy-frontend-web-azure.yml
│       └── deploy-frontend-web-gcp.yml
│
├── infrastructure/
│   └── pulumi/
│       ├── aws/
│       │   ├── __main__.py          ← すべての AWS リソース（S3/CF/Lambd
│       │   ├── Pulumi.yaml
│       │   ├── Pulumi.staging.yaml
│       │   └── requirements.txt
│       ├── azure/
│       │   ├── __main__.py          ← すべての Azure リソース（Storage/F
│       │   ├── Pulumi.yaml
│       │   └── requirements.txt
│       └── gcp/
│           ├── __main__.py          ← すべての GCP リソース（GCS/CloudRu
│           ├── Pulumi.yaml
│           └── requirements.txt
│
├── services/
│   ├── api/                         ← FastAPI バックエンド
│   │   ├── app/
│   │   │   ├── main.py              ← FastAPI アプリ、CORS、Mangum ハン
│   │   │   ├── config.py            ← Pydantic Settings（環境変数をロー
│   │   │   ├── models.py            ← Pydantic モデル（Post、Profile な
│   │   │   ├── auth.py              ← JWT 認証ミドルウェア
│   │   │   ├── jwt_verifier.py      ← Cognito / Azure AD / Firebase J
│   │   │   ├── backends/
│   │   │   │   ├── base.py          ← BackendBase 抽象クラス
│   │   │   │   ├── aws_backend.py   ← DynamoDB + S3 実装
│   │   │   │   ├── azure_backend.py ← Cosmos DB + Blob Storage 実装
│   │   │   │   ├── gcp_backend.py   ← Firestore + Cloud Storage 実装
│   │   │   │   └── local_backend.py ← DynamoDB Local + MinIO 実装
│   │   │   └── routes/
```

```
│   │   │       ├── posts.py          ← POST/GET/PUT/DELETE /posts
│   │   │       ├── profile.py        ← GET/PUT /profile/{userId}
│   │   │       └── uploads.py        ← POST /uploads/presigned-url
│   │   ├── index.py                  ← Lambda ハンドラー（Mangum ラッパー
│   │   ├── function_app.py           ← Azure Functions ハンドラー
│   │   ├── function.py               ← GCP Cloud Functions ハンドラー
│   │   ├── requirements.txt          ← 共有依存関係（fastapi、mangum、pyd
│   │   ├── requirements-aws.txt      ← AWS 固有（boto3 など）
│   │   ├── requirements-azure.txt    ← Azure 固有（azure-cosmos など）
│   │   ├── requirements-gcp.txt      ← GCP 固有（google-cloud-firestore
│   │   └── requirements-layer.txt    ← Lambda Layer 依存関係（boto3 を除
│   │
│   ├── frontend_react/               ← React フロントエンド（SNS アプリ ー
│   │   ├── src/
│   │   │   ├── main.tsx
│   │   │   ├── App.tsx
│   │   │   ├── api/                  ← Axios クライアント
│   │   │   ├── components/           ← UI コンポーネント
│   │   │   └── hooks/                ← カスタムフック
│   │   ├── vite.config.ts            ← base: "/sns/" を設定
│   │   └── package.json
│   │
│   └── frontend_web/                 ← Python FastAPI フロントエンド（SNS
│
├── static-site/                      ← ランディングページ（平文 HTML、SPA
│   ├── index.html                    ← 共有ランディング（クラウド/ローカル環
│   ├── aws/  azure/  gcp/            ← クラウドテーマ付きランディングバリアン
│   └── nginx.conf
│
├── scripts/                          ← デプロイ / テストシェルスクリプト
├── docs/
│   ├── AI_AGENT_GUIDE.md             ← ★ AI エージェント エントリーポイント
│   ├── AI_AGENT_00_CRITICAL_RULES.md ← 最初に読む
│   ├── AI_AGENT_01_CONTEXT.md        ← このファイル
│   └── archive/                      ← アーカイブ / スーパーセッドされたドキ
│
```

```
├── docker-compose.yml                ← api + dynamodb-local + minio
├── Makefile
└── README.md
```

クイックファイルリファレンス

やりたいこと	編集するファイル（群）
API エンドポイントを追加	`services/api/app/routes/*.py`
DB ロジックを変更（AWS）	`services/api/app/backends/aws_backend.py`
DB ロジックを変更（Azure）	`services/api/app/backends/azure_backend.py`
DB ロジックを変更（GCP）	`services/api/app/backends/gcp_backend.py`
環境変数を追加	`services/api/app/config.py` + `Pulumi.*.yaml`
AWS インフラを変更	`infrastructure/pulumi/aws/__main__.py`
Azure インフラを変更	`infrastructure/pulumi/azure/__main__.py`
GCP インフラを変更	`infrastructure/pulumi/gcp/__main__.py`
CI/CD ワークフロー編集	`.github/workflows/*.yml`（ワークスペースルート）
	`services/frontend_react/src/`

やりたいこと	編集するファイル（群）
React フロントエンド UI 編集	
Python フロントエンド編集	`services/frontend_web/`
ランディングページ編集	`static-site/`

開発環境

ホストマシン

- アーキテクチャ: **ARM**（**Apple Silicon M** シリーズ **Mac**）
- 開発環境: VS Code Dev Container（`.devcontainer/`）

Dev Container

コンポーネント	詳細
ベースイメージ	`mcr.microsoft.com/devcontainers/base:ubuntu`
Python	3.12
Node.js	22
Docker	Docker-in-Docker v2
クラウド CLI	AWS CLI、Azure CLI、Google Cloud SDK、GitHub CLI

コンポーネント	詳細
IaC	Pulumi CLI
公開ポート	3000（フロントエンド開発）、8000（API）

ARM ビルド警告

> ⚠ dev container は `linux/aarch64` 。すべてのクラウドランタイムは `linux/amd64` で実行される。 デプロイ用に Python パッケージは常にクロスコンパイルする：
>
> ```
> docker run --rm --platform linux/amd64 ¥
> -v /tmp/deploy:/out python:3.12-slim ¥
> bash -c "pip install --no-cache-dir --target /out -r requirements.txt -q"
> ```
>
> 詳細は AI_AGENT_00_CRITICAL_RULES.md のルール 2 を参照。

ローカル開発

```
cd /workspaces/multicloud-auto-deploy
docker compose up -d        # API (uvicorn) + DynamoDB Local + MinIO + frontend_web
curl http://localhost:8000/health
```

クラウド認証情報（ホストから自動マウント）

- `~/.aws` → AWS CLI 認証情報

- `~/.azure` → Azure CLI 認証情報
- `~/.config/gcloud` → Google Cloud SDK 認証情報

ブランチ戦略

```
feature/xxx  →  develop  →  push  →  staging 自動デプロイ
                  ↓
                  main   →  push  →  本番環境 自動デプロイ  ⚠ 即座に
```

次のセクション

→ 02 — アーキテクチャ

02 — アーキテクチャ

> 第II部 — アーキテクチャ・設計 | 親: AI_AGENT_GUIDE.md

システム概要

```
ユーザー
  │
  ├─ [AWS]   CloudFront ──► S3（React SPA：ランディング + SNSページ）   ← 静的
  │         API Gateway v2 ──► Lambda（Python 3.12）──► DynamoDB
  │
  ├─ [Azure] Front Door ─┬─ /sns/* ──► Blob Storage $web/sns/（React SP
  │                      └─ /*     ──► Blob Storage $web（ランディング ←
  │         Azure Functions func ──► Cosmos DB（Serverless）
  │
```

```
└─ [GCP]   Cloud CDN ─┬─ /sns/* ──► GCS バケット /sns/（React SPA ← 静的
                     └─ /*     ──► GCS（ランディング ← 静的）
           Cloud Run api ──► Firestore
```

> フロントエンドアーキテクチャ：3つのクラウドすべてが SNS アプリを静的 **React SPA** として提供 オブジェクトストレージ（S3 / Blob Storage / GCS）にデプロイされ、CDN 経由で配信される。 Python `services/frontend_web` SSR サービスは スーパーセッドされた もので、CDN パスに含まれていない。 React SPA ワークフロー： `deploy-frontend-web-{aws,azure,gcp}.yml` SPA ルーミング（ `/sns/*` → `/sns/index.html` にリライト）：
>
> - AWS：CloudFront Function `spa-sns-rewrite-{stack}`
> - Azure：AFD RuleSet `SpaRuleSet` ＋ URL Rewrite アクション
> - GCP：CDN バックエンドバケットが GCS をサーブ； `/sns/*` パスルール URL マップから削除（GCS にフォールスルー）

ストレージパス構造

```
bucket-root/
├── index.html          ← ランディングページ
├── error.html
├── aws/
├── azure/
├── gcp/
└── sns/                ← React SNS アプリ（AWS のみ ー Vite ビルド、base="/
    ├── index.html
    └── assets/
```

```
        ├── index-*.js
        └── index-*.css
```

CI デプロイ先：

コンテンツ	**AWS**	**Azure**	**GCP**
ランディングページ	`s3://bucket/`	`$web/`	`gs://bucket/`
SNSページ	`s3://bucket/sns/`（React SPA）	`$web/sns/`（React SPA）	`gs://bucket/sns/`（React SPA）

3つのクラウドすべてが SNS アプリを静的 React SPA として提供。CDN が SPA ルーティングを処理：

- **AWS**：CloudFront Function が `/sns` および `/sns/` → `/sns/index.html` にリライト
- **Azure**：AFD `SpaRuleSet` URL Rewrite が `/sns/*`（非アセット）→ `/sns/index.html` にリライト
- **GCP**：GCS が `sns/index.html` をデフォルトとして配信；ディープリンクには CDN レベルの処理が必要（SPA ルーティングはブラウザー History API で部分的に処理）

AWS アーキテクチャ詳細

```
CloudFront（E1TBH4R432SZBZ / ステージング、E214XONKTXJEJD / 本番）
  ├── /sns/* → S3: multicloud-auto-deploy-{env}-frontend/sns/（React SP
  │            CloudFront Function `spa-sns-rewrite-{stack}` が /sns を
  └── /*     → S3: multicloud-auto-deploy-{env}-frontend/（ランディングペ
```

```
S3: multicloud-auto-deploy-{env}-frontend
  ├── index.html        ← React SPA エントリーポイント（Vite ビルド）
  ├── assets/           ← JS / CSS バンドル
  └──── （ランディング、aws/、azure/、gcp/ ページ）

API Gateway v2 HTTP（z42qmqdqac / ステージング）
  └── $default → Lambda: multicloud-auto-deploy-{env}-api（バックエンド）
                  └── DynamoDB: multicloud-auto-deploy-{env}-posts（PAY
                       ← Single Table Design（PK/SK）
                       ← POSTS パーティション：投稿データ（GSI：postId / use
                       ← PROFILES パーティション：ユーザープロフィール
                  └── S3: multicloud-auto-deploy-{env}-images（画像アップ
                       ← IMAGES_BUCKET_NAME 環境変数で参照

Lambda: multicloud-auto-deploy-{env}-frontend-web [削除済み ー Python SSR
  デッドコード削除 2026-02-22。CloudFront `/sns/*` は現在 S3 に直接ルーティング。
  参照: REFACTORING_REPORT_20260222.md § 3
```

注意： `frontend-web` Lambda は当初 Python で SNS 画面を SSR するために作られたが、 React + S3 へ移行済み。Lambda 自体は削除されていない場合があるが、CloudFront の `/sns/*` 動作は現在 API Gateway（バックエンド API）を向いている。

Lambda Layer： `multicloud-auto-deploy-staging-dependencies` — FastAPI / Mangum / JWT 依存関係のみを含む；boto3 は Lambda ランタイムに含まれる。 ー アプリコード（~78 KB）と Layer（~8-10 MB）は別々にデプロイ。

主要環境変数： `POSTS_TABLE_NAME` 、 `IMAGES_BUCKET_NAME` 、 `COGNITO_USER_POOL_ID` 可観測性：AWS Lambda Powertools（Logger / Tracer / Metrics）— `SimpleSNS` ネームスペース

Azure アーキテクチャ詳細

> ⌧**React SPA** に移行：SNS ページは現在 Blob Storage（静的ファイル）から配信される。 Python `frontend_web` Azure Function は `deploy-frontend-web-azure.yml` に統合された。

```
Front Door（multicloud-auto-deploy-staging-fd）
  エンドポイント: mcad-staging-d45ihd
  ├── /sns/*  → オリジン：Blob Storage $web/sns/（React SPA — 静的 HTML/JS
  │               SpaRuleSet が /sns/* → /sns/index.html にリライト（SPA ル
  └── /*      → オリジン：Blob Storage $web（ランディングページのみ）
                  mcadwebd45ihd.z11.web.core.windows.net

Azure Functions frontend-web（FC1 FlexConsumption）[レガシー — CDN パスに含
  └── デプロイ済みだが、React SPA in Blob Storage に統合された
    本番環境：multicloud-auto-deploy-production-frontend-web-v2（alwaysRe

Azure Functions：multicloud-auto-deploy-staging-func（Flex Consumption）
  └── HTTP トリガー：/{*route}（関数名：HttpTrigger）
        │  ← FastAPI（Mangum なし、カスタム ASGI ブリッジ）にフォワード
        └── Cosmos DB（Serverless）
             ← DB：simple-sns / コンテナー：items
           ← 環境変数：COSMOS_DB_ENDPOINT / COSMOS_DB_KEY
             ← COSMOS_DB_DATABASE（デフォルト：simple-sns）
             ← COSMOS_DB_CONTAINER（デフォルト：items）
        └── Azure Blob Storage：images コンテナー（画像アップロード）
             ← AZURE_STORAGE_ACCOUNT_NAME / AZURE_STORAGE_ACCOUNT_KEY /
```

リソースグループ： `multicloud-auto-deploy-staging-rg` （japaneast） **WAF**：未設定（Standard SKU；Premium SKU で追加可能）

GCP アーキテクチャ詳細

> ⌧**React SPA** に移行：SNS ページは現在 GCS（静的ファイル）から Cloud CDN 経由で配信される。 Cloud Run `frontend-web` へのルーティングの `/sns/*` パスルールは URL マップから削除された。

```
グローバル IP：34.117.111.182
  └── HTTP フォワーディングルール
        └── URL マップ
              └─── /*（デフォルト）→ バックエンドバケット → GCS：ashnova-multi
                                （/sns/ の React SPA + / のランディング）
             注記：/sns/* パスルール削除（2026-02-22）— GCS デフォルトにフォールス

Cloud Run：multicloud-auto-deploy-staging-frontend-web [レガシー — CDN バ
  URL：https://multicloud-auto-deploy-staging-frontend-web-899621454670
  └─── デプロイ済みだが CDN はここにリクエストをルーティングしない
  CDN カスタムレスポンスヘッダー：Cross-Origin-Opener-Policy: same-origin-al

Cloud Run：multicloud-auto-deploy-staging-api（バックエンド API）
  └── Firestore（デフォルト）
       ← posts コレクション：投稿データ（GCP_POSTS_COLLECTION、デフォルト：pos
       ← profiles コレクション：ユーザープロフィール（GCP_PROFILES_COLLECTION
  └── GCS：ashnova-multicloud-auto-deploy-staging-uploads（署名付き URL ア
       ← GCP_STORAGE_BUCKET 環境変数で参照
```

注意：GCP はクラシック外部 LB（`EXTERNAL` スキーム）を使用。 URL マップパスベースルーティング（`/sns/*` → Cloud Run）には `EXTERNAL_MANAGED` が必要；現在は すべてのパスで GCS にフォールバックする可能性 — 検証が必要。

API ルート

ルータープレフィックス	主なエンドポイント	認証
`/posts`	GET/POST/PUT/DELETE（投稿 CRUD）	必要
`/uploads`	POST（署名付き URL 発行）	必要
`/profile`	GET/PUT（プロフィール取得・更新）	必要
`/api/messages/`	旧フロントエンド互換エイリアス	オプション

旧フロントエンドとの後方互換のため `/api/messages/` エイリアスは維持されているが、新規実装は `/posts` を使用する。

バックエンドクラウド自動検出ロジック

```
# services/api/app/config.py
# 環境変数 CLOUD_PROVIDER で明示的に設定するか、自動検出：
AWS_EXECUTION_ENV    存在 → "aws"
WEBSITE_INSTANCE_ID 存在 → "azure"
K_SERVICE            存在 → "gcp"
それ以外                  → "local"
```

認証フロー

```
API バックエンド（services/api）：
  クライアント
    → Authorization: Bearer <JWT>
    → FastAPI auth.py（AUTH_DISABLED=false の場合）
         → jwt_verifier.py
              AWS:   Cognito JWKS エンドポイント検証
              Azure: Azure AD JWKS 検証
              GCP:   Firebase Auth JWKS 検証
        → 検証 OK → routes/*

GCP frontend-web（services/frontend_web）— 別の httponly Cookie フロー：
  ブラウザー → /sns/login
    → Firebase Google Sign-In ポップアップ（Firebase SDK v10.8.0）
    → POST /sns/session { token: <Firebase ID トークン> }
    → FastAPI がトークンを検証 → httponly セッションクッキーを設定
    → onIdTokenChanged → トークン有効期限時にセッションクッキーを自動更新
```

ステージング：`AUTH_DISABLED=false`（過去には誤って `true` に設定されていた — 現在は修正）

セキュリティ設定ステータス

機能	**AWS**	**Azure**	**GCP**
HTTPS 強化	☒CloudFront	☒Front Door	☒HTTP のみ（要修正）
WAF	☒未設定	☒ 未設定（TODO）	☒Cloud Armor
データ暗号化	☒SSE-S3	☒Azure SSE	☒Google 管理

機能	AWS	Azure	GCP
バージョンストレージ	☒	☒	☒
アクセスログ	☒	☒	☒
セキュリティヘッダー	☒CloudFront RP	☒	☒

次のセクション

→ 03 — API とデータモデル

03 — API とデータモデル

> 第II部 — アーキテクチャ・設計 | 親: AI_AGENT_GUIDE.md

API エンドポイント

メソッド	パス	認証	説明
GET	`/`	☒	ルート — クラウド情報
GET	`/health`	☒	ヘルスチェック
GET	`/limits`	☒	アップロード/投稿制限（`maxImagesPerPost`）
GET	`/docs`	☒	Swagger UI

メソッド	パス	認証	説明
GET	`/redoc`	☒	ReDoc
GET	`/posts`	☒	投稿一覧（ページネーション+タグフィルター対応）
GET	`/posts/{post_id}`	☒	単一投稿を取得
POST	`/posts`	☒	投稿を作成
PUT	`/posts/{post_id}`	☒	投稿を更新（所有者または管理者）
DELETE	`/posts/{post_id}`	☒	投稿を削除（所有者または管理者）
GET	`/profile/{user_id}`	☒	プロフィールを取得
PUT	`/profile/{user_id}`	☒	プロフィールを更新
POST	`/uploads/presigned-url`	☒	画像アップロード用署名付き URL を発行（単数）
POST	`/uploads/presigned-urls`	☒	一括アップロード用署名付き URL を発行

リクエスト / レスポンス スキーマ

POST /posts — リクエストボディ

```
{
  "content": "string (1-10000) ",
  "isMarkdown": false,
  "imageKeys": ["key1", "key2"], // 最大 16（サーバー側で MAX_IMAGES_PER_POST で強制）
  "tags": ["tag1", "tag2"]
}
```

GET /limits — レスポンス

```
{ "maxImagesPerPost": 10 }
```

フロントエンドはマウント時にこれを取得。 `MAX_IMAGES_PER_POST` はバックエンドの環境変数（デフォルト：10）。 これが正規ソース — フロントエンドにアップロード制限をハードコードしない。

GET /posts — レスポンス

```
{
  "items": [ <Post> ],
  "limit": 20,
  "nextToken": "string | null"
}
```

Post オブジェクト

```
{
  "postId": "uuid",
  "userId": "string",
  "nickname": "string | null",
  "content": "string",
  "isMarkdown": false,
  "imageUrls": ["https://..."],
  "tags": ["tag"],
  "createdAt": "2026-02-20T00:00:00Z",
  "updatedAt": "2026-02-20T00:00:00Z",

  // 後方互換フィールド（snake_case）
  "id": "uuid",
  "author": "string（= userId）",
  "created_at": "...",
  "updated_at": "...",
  "image_url": "string | null"
}
```

GET /posts クエリパラメータ

パラメータ	型	デフォルト	説明
`limit`	int（1-50）	20	返すアイテム数
`nextToken`	string	null	ページネーショントークン
`tag`	string	null	タグフィルター

> **GCP 注記：** `generate_signed_url()` は `service_account_email` + `access_token` が必須（IAM `signBlob` API パス）Compute

> Engine 認証情報は秘密鍵を含まないため。
> `AI_AGENT_00_CRITICAL_RULES.md` のルール 11 を参照。

POST /uploads/presigned-urls — リクエストボディ

```
{
  "count": 3, // ファイル数（ge=1、le=100；実際の制限は /limits 経由）
  "contentTypes": ["image/jpeg", "image/png", "image/webp"] // ファイルあたり 1 つ
}
```

POST /uploads/presigned-urls — レスポンス

```
{
  "urls": [
    { "uploadUrl": "https://...署名付き-url...", "key": "uploads/uuid.jpg" },
    ...
  ]
}
```

> **GCP** 注記： `generate_signed_url()` は `service_account_email` + `access_token` が必須（IAM `signBlob` API パス）Compute Engine 認証情報は秘密鍵を含まないため。
> `AI_AGENT_00_CRITICAL_RULES.md` のルール 11 を参照。

データモデル（DynamoDB Single-Table Design）

ローカルと AWS 向け共有テーブル設計。Azure/GCP は同じ論理モデルを使用。

テーブル： `{project}-{stack}-posts` （AWS）/ `simple-sns-local` （ローカル）

アイテムタイプ	PK	SK	キー属性
投稿	`POSTS`	`<ISO-timestamp>#<uuid>`	`postId` 、 `userId` 、 `content` 、 `tags` 、 `createdAt`
プロフィール	`USER#<userId>`	`PROFILE`	`postId=PROFILE#<userId>` 、 `userId` 、 `nickname` 、 `bio`

GSI： `PostIdIndex`

キー	値
ハッシュキー	`postId`
プロジェクション	ALL

直接投稿ルックアップ（ `/posts/{id}` ）と プロフィールルックアップ（ `postId = PROFILE#<userId>` ）に使用。

バックエンド実装クラス

```
BackendBase（base.py） ← 抽象クラス
  ├── list_posts(limit、next_token、tag) → （list[Post]、next_token）
  ├── get_post(post_id) → Post
  ├── create_post(body、user) → Post
  ├── update_post(post_id、body、user) → Post
  ├── delete_post(post_id、user) → dict
  ├── get_profile(user_id) → Profile
  ├── update_profile(user_id、body、user) → Profile
  └── generate_upload_urls(keys、user) → list[UploadUrl]

AwsBackend    → DynamoDB（boto3）+ S3
AzureBackend  → Cosmos DB（azure-cosmos）+ Blob Storage
GcpBackend    → Firestore（google-cloud-firestore）+ Cloud Storage
LocalBackend  → DynamoDB Local + MinIO（S3 互換）
```

バックエンド選択

```
        # services/api/app/backends/__init__.py
        def get_backend() -> BackendBase:
            provider = config.CLOUD_PROVIDER  # "aws" | "azure" |
"gcp" | "local"
           ...
```

ローカル開発環境

```
        cd multicloud-auto-deploy
        docker compose up   # api(:8000) + dynamodb-local(:8001) +
minio(:9000/:9001)
```

サービス	URL	認証情報
FastAPI	http://localhost:8000	—
Swagger UI	http://localhost:8000/docs	—
DynamoDB Local シェル	http://localhost:8001/shell/	—
MinIO コンソール	http://localhost:9001	minioadmin / minioadmin

必要な環境変数（docker-compose で自動設定）

```
CLOUD_PROVIDER=local
DYNAMODB_ENDPOINT=http://dynamodb-local:8001
DYNAMODB_TABLE_NAME=simple-sns-local
MINIO_ENDPOINT=http://minio:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin
MINIO_BUCKET_NAME=simple-sns
AUTH_DISABLED=true   ← ローカルのみ true は許容
```

テスト実行

```
# ユニット + 統合テスト（モック）
cd services/api
pytest tests/test_backends_integration.py -v

# AWS のみ
```

```
        pytest -m aws

        # ライブデプロイ API テスト
        export API_BASE_URL=https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com
        pytest tests/test_api_endpoints.py -v

        # E2E（すべてのクラウド、curl ベース）
        ./scripts/test-e2e.sh
```

次のセクション

→ 04 — インフラストラクチャ（Pulumi）

04 — インフラストラクチャ（Pulumi）

> 第II部 — アーキテクチャ・設計 | 親: AI_AGENT_GUIDE.md

Pulumi 概要

フィールド	値
IaC ツール	Pulumi Python SDK 3.x
状態	Pulumi Cloud（リモート状態）
スタック	`staging` / `production`
言語	Python

フィールド	値
コードパス	`infrastructure/pulumi/{aws,azure,gcp}/__main__.py`

一般的な操作

```
# スタック一覧表示
pulumi stack ls

# デプロイ
pulumi up --stack staging

# 出力を表示
pulumi stack output

# 変更内容をプレビュー（ドライラン）
pulumi preview --stack staging

# リソースを削除（注意が必要）
pulumi destroy --stack staging
```

AWS Pulumi スタック

ディレクトリ： `infrastructure/pulumi/aws/`

リソース一覧

Pulumi 論理名	**AWS** リソース	名前パターン
`lambda-role`	IAM Role	`{project}-{stack}-lambda-role`
`app-secret`	Secrets Manager Secret	—
`dynamodb-table`	DynamoDB テーブル	`{project}-{stack}-posts`
`lambda-function`	Lambda 関数	`{project}-{stack}-api`
`api-gateway`	API Gateway v2	—
`frontend-bucket`	S3 バケット	`{project}-{stack}-frontend`
`landing-bucket`	S3 バケット	`{project}-{stack}-landing`
`cloudfront-distribution`	CloudFront（PriceClass_200）	—
`security-headers-policy`	CloudFront ResponseHeadersPolicy	`{project}-{stack}-security-headers`
`cognito-user-pool`	Cognito ユーザープール	—
`sns-topic`	SNS トピック（アラート）	—

Pulumi 論理名	AWS リソース	名前パターン
CloudWatch Alarms（複数）	CloudWatch	—

主要な設定キー

```
        pulumi config set aws:region ap-northeast-1
        pulumi config set allowedOrigins "https://example.com"
        pulumi config set alarmEmail your@email.com
        pulumi config set staticSiteDomain "aws.example.com"
# カスタムドメイン（オプション）
        pulumi config set staticSiteAcmCertificateArn "arn:..."
# ACM 証明書（オプション）
```

> ⚠ 本番環境スタック向け重要事項： `pulumi up --stack production` を実行する前に、必ず `customDomain` と `acmCertificateArn` を設定する。設定がないと、CloudFront はデフォルト証明書に戻り、 すべての HTTPS ビジターに対して破断する。
>
> ```
> pulumi config set customDomain www.aws.ashnova.jp --stack production
> pulumi config set acmCertificateArn ¥
> arn:aws:acm:us-east-1:278280499340:certificate/914b86b1-4c10-4354-91cf-19c4460dcde5 ¥
> --stack production
> ```
>
> ACM 証明書は `us-east-1` に存在する必要があります（CloudFront に必須）。現在の証明書は 2027-03-12 に失効します。

主要な出力

```
        pulumi stack output api_url                     # API Gateway URL
        pulumi stack output cloudfront_url              # CloudFront URL
        pulumi stack output cloudfront_distribution_id  # CloudFront Distribution ID
        pulumi stack output frontend_bucket_name        # S3 バケット名
        pulumi stack output lambda_function_name        # Lambda 関数名
        pulumi stack output cognito_user_pool_id
        pulumi stack output cognito_client_id
```

Azure Pulumi スタック

ディレクトリ：`infrastructure/pulumi/azure/`

リソース一覧

Pulumi 論理名	Azure リソース	名前パターン
`resource-group`	リソースグループ	`{project}-{stack}-rg`
`functions-storage`	ストレージアカウント	`mcadfunc{suffix}`
`frontend-storage`	ストレージアカウント	`mcadweb{suffix}`
`landing-storage`	ストレージアカウント	`mcadlanding{suffix}`
`function-app`	Azure Functions	

Pulumi 論理名	Azure リソース	名前パターン
		`{project}-{stack}-func`
`cosmos-account`	Cosmos DB アカウント	—
`frontdoor-profile`	Front Door プロファイル	`{project}-{stack}-fd`
`azure-ad-app`	Azure AD アプリケーション	—
`spa-rule-set`	AFD RuleSet	`SpaRuleSet`
`spa-rewrite-rule`	AFD ルール	`SpaIndexHtmlRewrite`
Action Groups + Alerts	Azure Monitor	—

> **SPA** ルーティング： `SpaRuleSet` はすべての非静的な `/sns/*` リクエストを `/sns/index.html` にリライトして、 React クライアント側ルーティングが直接 URL アクセスとページリロード時に機能するようにします。 RuleSet 名は 英数字のみ（ハイフンなし）である必要があります。条件あたり最大 10 個の match_values。

主要な設定キー

```
pulumi config set azure-native:location japaneast
pulumi config set environment staging
pulumi config set alarmEmail your@email.com
```

```
pulumi config set staticSiteDomain "azure.example.com"  # オプション
```

主要な出力

```
pulumi stack output api_url
pulumi stack output frontdoor_url
pulumi stack output frontend_storage_name
pulumi stack output azure_ad_tenant_id
pulumi stack output azure_ad_client_id
```

GCP Pulumi スタック

ディレクトリ：`infrastructure/pulumi/gcp/`

リソース一覧

Pulumi 論理名	GCP リソース	名前
`frontend-bucket`	GCS バケット	`ashnova-{project}-{stack}-frontend`
`uploads-bucket`	GCS バケット	`ashnova-{project}-{stack}-uploads`
`backend-bucket`	Compute バックエンドバケット	`{project}-{stack}-cdn-backend`
`cdn-ip-address`	グローバル外部 IP	`{project}-{stack}-cdn-ip`
`url-map`	URL マップ	—

Pulumi 論理名	GCP リソース	名前
`cloud-run-service`	Cloud Run	`{project}-{stack}-api`
`firestore-db`	Firestore	（デフォルト）
`managed-ssl-cert`	SSL 証明書	オプション
Alert Policies（複数）	Cloud Monitoring	—

主要な設定キー

```
pulumi config set gcp:project ashnova
pulumi config set environment staging
pulumi config set alarmEmail your@email.com
pulumi config set staticSiteDomain "gcp.example.com"  # オプション
pulumi config set monthlyBudgetUsd 50                 # 本番環境のみ
```

主要な出力

```
pulumi stack output api_url
pulumi stack output cdn_url
pulumi stack output cdn_ip_address
pulumi stack output frontend_bucket_name
```

> **GCP 固有の注記：**
>
> - `uploads-bucket` （ `ashnova-{project}-{stack}-uploads` ）は `allUsers:objectViewer` を持つ ー パブリック。 `frontend-bucket` にパブリック読み込みを付与しないこと。
> - `ManagedSslCertificate` は `ignore_changes=["name", "managed"]` を使用して、名前ハッシュが変わったときに Pulumi が証明書の置換を試みるのを防ぎます（証明書が HTTPS プロキシに接続されている場合、GCP は 400 を返します）。
> - `pulumi up` が URLMap の `Error 412: Invalid fingerprint` で失敗する場合、 `pulumi up` 前に `pulumi refresh --yes --skip-preview` ステップを追加します。
> - Firebase 認可ドメインは Identity Toolkit Admin v2 API 経由で更新する必要があります（ `x-goog-user-project` ヘッダーが必須）。 これは `deploy-gcp.yml` で自動化されています。

GCS リソース競合（Error 409 / Error 412）

```
        # Error 409：バケットすでに存在（状態が同期していない）
        # 修正：pulumi up 前に既存バケットを Pulumi 状態にインポート
        pulumi import gcp:storage/bucket:Bucket uploads-bucket ¥
          ashnova-multicloud-auto-deploy-staging-uploads --stack staging

        # Error 412：URLMap 上で無効なフィンガープリント（Pulumi 状態が古い）
        # 修正：pulumi up 前に pulumi refresh を追加
        pulumi refresh --yes --skip-preview --stack staging
        pulumi up --yes --stack staging
```

Azure CLI 認証エラー

```
        # サービスプリンシパル認証情報を明示的に設定
        export AZURE_CLIENT_ID=$(echo $AZURE_CREDENTIALS | jq -r
'.clientId')
        export AZURE_CLIENT_SECRET=$(echo $AZURE_CREDENTIALS | jq -r
'.clientSecret')
        export AZURE_SUBSCRIPTION_ID=$(echo $AZURE_CREDENTIALS | jq -
r '.subscriptionId')
        export AZURE_TENANT_ID=$(echo $AZURE_CREDENTIALS | jq -r
'.tenantId')
```

Azure Pulumi 保留中の操作

```
        # エラー：「スタックに保留中の操作がある」
        pulumi stack export | ¥
          python3 -c "import sys,json; d=json.load(sys.stdin);
d['deployment']['pending_operations']=[];
print(json.dumps(d))" | ¥
          pulumi stack import --force
```

Lambda Layer 設定

2つのオプション（詳細は `LAMBDA_LAYER_OPTIMIZATION.md` を参照）：

オプション **A — Klayers**（デフォルト、推奨）：ビルド不要。パブリックなコミュニティ管理 Lambda Layers を使用。Pulumi 設定で有効化：

```
        pulumi config set use_klayers true
```

オプション **B** — カスタムレイヤー（完全な制御、特定バージョン）： `./scripts/build-lambda-layer.sh` でビルド（boto3 / Azure / GCP SDK を ZIP から除外）。

Lambda Layer ビルドステップ

```
# 1. レイヤーをビルド（deps のみ；boto3 は除外）
./scripts/build-lambda-layer.sh
# → generates services/api/lambda-layer.zip

# 2. レイヤーを発行
aws lambda publish-layer-version ¥
  --layer-name multicloud-auto-deploy-staging-dependencies ¥
  --zip-file fileb://services/api/lambda-layer.zip ¥
  --compatible-runtimes python3.12 ¥
  --region ap-northeast-1

# 3. アプリコードのみをパッケージ（~78 KB）
cd services/api
cp -r app .build/package/
cp index.py .build/package/
cd .build/package && zip -r ../../lambda.zip .

# 4. Lambda を更新（直接 ZIP アップロード、S3 不要）
aws lambda update-function-code ¥
  --function-name multicloud-auto-deploy-staging-api ¥
  --zip-file fileb://lambda.zip
```

次のセクション

→ 05 — CI/CD パイプライン

05 — CI/CDパイプライン

> Part II — Architecture & Design | Parent: AI_AGENT_GUIDE.md

---

ワークフロー一覧

> ⚠ リポジトリルートの `.github/workflows/` を常に編集してください。 `multicloud-auto-deploy/.github/workflows/` は CI から無視されます。

ファイル	トリガー条件	ターゲット	環境
`deploy-aws.yml`	push: develop/main, 手動	Lambda + API	staging / production
`deploy-azure.yml`	push: develop/main, 手動	Functions + API	staging / production
`deploy-gcp.yml`	push: develop/main, 手動	Cloud Run (API)	staging / production
`deploy-frontend-aws.yml`	push: develop/main, 手動	S3 `sns/`	staging / production
`deploy-frontend-azure.yml`	push: develop/main, 手動	Blob `$web/sns/`	staging / production

ファイル	トリガー条件	ターゲット	環境
`deploy-frontend-gcp.yml`	push: develop/main, 手動	GCS `sns/`	staging / production
`deploy-frontend-web-aws.yml`	push: develop/main, 手動	Lambda (frontend-web)	staging / production
`deploy-frontend-web-azure.yml`	push: develop/main, 手動	Functions (frontend-web)	staging / production
`deploy-frontend-web-gcp.yml`	push: develop/main, 手動	Cloud Run (frontend-web, Docker)	staging / production
`deploy-landing-aws.yml`	push: develop/main, 手動	S3 `/`	staging / production
`deploy-landing-azure.yml`	push: develop/main, 手動	Blob `$web/`	staging / production
`deploy-landing-gcp.yml`	push: develop/main, 手動	GCS `/`	staging / production

ブランチ → 環境マッピング

```
develop  →  staging
main     →  production  ⚠ すぐに本番環境に反映されます
```

トリガー条件 (パスフィルタ)

変更されたパス	トリガーされるワークフロー
`services/api/**`	deploy-{aws,azure,gcp}.yml
`infrastructure/pulumi/**`	deploy-{aws,azure,gcp}.yml
`services/frontend_react/**`	deploy-frontend-{aws,azure,gcp}.yml
`services/frontend_web/**`	deploy-frontend-web-{aws,azure,gcp}.yml
`static-site/**`	deploy-landing-{aws,azure,gcp}.yml

クラウド設定ファイル (.**github/config/**)

> 単一の信頼できるソース として機能する、環境ごとの安定した値。以前は GitHub Secrets、インラインの `case/esac` ブロック、gitignore された `Pulumi.*.yaml` ファイルに分散していました。

保存先

```
.github/config/
  aws.production.env
  aws.staging.env
  azure.production.env
  azure.staging.env
  gcp.production.env
  gcp.staging.env
```

保存内容

クラウド	キー	例
すべて	`CLOUD_CUSTOM_DOMAIN`	`www.gcp.ashnova.jp`
Azure	`CLOUD_AZURE_CLIENT_ID`	`0b926ff6-...` (AD ap registration)
Azure	`CLOUD_AZURE_TENANT_ID`	`a3182bec-...`
AWS	`CLOUD_ACM_CERT_ARN`	`arn:aws:acm:us-east-1:...`
AWS	`CLOUD_CLOUDFRONT_DOMAIN`	`d1qob7569mn5nw.cloudfront.ne`
AWS	`CLOUD_CLOUDFRONT_DISTRIBUTION_ID`	`E214XONKTXJEJD`

ワークフローでの使用方法

すべてのメインデプロイワークフロー（`deploy-aws.yml`, `deploy-azure.yml`, `deploy-gcp.yml`）は、**Pulumi** スタック名の決定の直後にクラウド設定を読み込むステップを含みます。

```
        CONFIG_FILE=".github/config/aws.${STACK_NAME}.env"
        source "$CONFIG_FILE"
        echo "custom_domain=$CLOUD_CUSTOM_DOMAIN" >> $GITHUB_OUTPUT
        # ... その他の出力
```

その後のすべてのステップは、フォールバックロジックの代わりに `${{ steps.cloud_config.outputs.custom_domain }}` などを参照します。

更新時期

イベント	アクション
カスタムドメイン変更	関連する `.env` ファイルの `CLOUD_CUSTOM_DOMAIN` を更新
ACM 証明書更新 (AWS)	`CLOUD_ACM_CERT_ARN` を更新
Azure AD アプリ再登録	`CLOUD_AZURE_CLIENT_ID` を更新
新しいスタック追加	新しい `<cloud>.<stack>.env` ファイルを追加

GitHub Secrets に保存しない理由

リポジトリレベルの GitHub Secrets は環境に依存しません。リポジトリレベルで `AZURE_CUSTOM_DOMAIN=www.azure.ashnova.jp` を設定すると、ステージングデプロイも本番環境のドメインを受け取ります。環境レベルの Secrets はこの問題を解決しますが、GitHub UI で設定する必要があり、コード レビューには表示されません。`.github/config/` のコンフィグ

ファイルは git でトラッキングされ、diff で表示され、GitHub UI のメンテナンスは不要です。

必要な GitHub Secrets

AWS

Secret	用途
`AWS_ACCESS_KEY_ID`	IAM アクセスキー
`AWS_SECRET_ACCESS_KEY`	IAM シークレットキー
`PULUMI_ACCESS_TOKEN`	Pulumi Cloud 認証トークン

> ⚠ `AWS_CUSTOM_DOMAIN` , `AWS_ACM_CERTIFICATE_ARN` , `AWS_CLOUDFRONT_DOMAIN` は使用されなくなりました。 これらの値は `.github/config/aws.<stack>.env` から読み込まれます。

Azure

Secret	用途
`AZURE_CREDENTIALS`	Service Principal JSON（`az ad sp create-for-rbac --sdk-auth`）
`AZURE_SUBSCRIPTION_ID`	サブスクリプション ID
`AZURE_RESOURCE_GROUP`	リソースグループ名
`PULUMI_ACCESS_TOKEN`	Pulumi Cloud 認証トークン

> ⚠ `AZURE_CUSTOM_DOMAIN` は **`deploy-azure.yml`** では使用されなくなりました。 `CLOUD_CUSTOM_DOMAIN` , `CLOUD_AZURE_CLIENT_ID` , `CLOUD_AZURE_TENANT_ID` は `.github/config/azure.<stack>.env` から読み込まれます。

GCP

Secret	用途
`GCP_CREDENTIALS`	Service Account JSON
`GCP_PROJECT_ID`	プロジェクト ID (`ashnova`)
`GCP_API_ENDPOINT`	Cloud Run API URL (frontend-web `API_BASE_URL` 用)
`FIREBASE_API_KEY`	Firebase Web API キー (frontend-web 認証用)
`FIREBASE_AUTH_DOMAIN`	Firebase Auth ドメイン (frontend-web 認証用)
`FIREBASE_APP_ID`	Firebase App ID (frontend-web 認証用)
`PULUMI_ACCESS_TOKEN`	Pulumi Cloud 認証トークン

デプロイフロー (**AWS** バックエンド例)

```
# deploy-aws.yml ステップ（要約）
1. Checkout
2. AWS 認証（aws-actions/configure-aws-credentials）
```

```
        3. Python 3.12 セットアップ
        4. Pulumi ログイン（PULUMI_ACCESS_TOKEN）
        5. Pulumi スタック名決定 → STACK_NAME = staging | production
        6. クラウド設定を読み込む → source .github/config/aws.$
{STACK_NAME}.env
           # 出力: custom_domain, acm_cert_arn, cloudfront_domain
        7. pulumi up --stack staging # インフラストラクチャ作成/更新
        8. Pulumi 出力取得 → バケット名、CloudFront ID、Cognito ID な
ど
        9. Lambda Layer ビルド（./scripts/build-lambda-layer.sh）
        10. Lambda Layer を公開 （name = multicloud-auto-deploy-$
{STACK_NAME}-dependencies）
        11. アプリコードを ZIP でパッケージ（app/ + index.py）
        12. Lambda 関数コードを更新
        13. Lambda 環境変数を更新
           - CLOUD_PROVIDER=aws
           - AUTH_DISABLED=false
           - AUTH_PROVIDER=cognito
           - COGNITO_USER_POOL_ID (Pulumi 出力から)
           - COGNITO_CLIENT_ID (Pulumi 出力から)
```

デプロイフロー (フロントエンド例)

```
        # deploy-frontend-aws.yml ステップ（要約）
        1. Checkout
        2. AWS 認証
        3. Node.js セットアップ
        4. npm ci
        5. Pulumi 出力から S3 バケット名と CloudFront ID を取得
        6. VITE_API_URL を設定して npm run build を実行
           # vite.config.ts: base="/sns/" は既に設定されています
        7. aws s3 sync dist/ s3://{bucket}/sns/ --delete
        8. CloudFront キャッシュ無効化（/*）
```

手動デプロイ (緊急時)

```
# GitHub CLI 経由でワークフローを手動トリガー
gh workflow run deploy-aws.yml ¥
  --ref develop ¥
  -f environment=staging

# または production へ
gh workflow run deploy-aws.yml ¥
  --ref main ¥
  -f environment=production

# 実行ステータス確認
gh run list --workflow=deploy-aws.yml --limit 5
gh run watch <run-id>
```

手動デプロイ (緊急時)

```
# GitHub CLI経由でワークフローを手動トリガー
gh workflow run deploy-aws.yml ¥
  --ref develop ¥
  -f environment=staging

gh workflow run deploy-frontend-gcp.yml ¥
  --ref main ¥
  -f environment=production

# 実行ステータス確認
gh run list --workflow=deploy-aws.yml --limit 5
gh run watch <run-id>
```

CI/CD 現在ステータス (2026-02-24)

ワークフロー	ブランチ	ステータス	コミット
Deploy Frontend Web to GCP	develop	☒	`0ed0805`
Deploy to GCP (Pulumi + API)	develop	☒	`0ed0805`
Deploy Frontend to GCP	develop	☒	`591ce0b`
Deploy Frontend to AWS	develop	☒	`591ce0b`
Deploy Frontend to Azure	develop	☒	`591ce0b`
Deploy Landing to GCP	develop	☒	`591ce0b`
Deploy Landing to AWS	develop	☒	`591ce0b`
Deploy Landing to Azure	develop	☒	`591ce0b`

修正済み CI バグ (再発を監視)

バグ	症状	修
ワークフロー重複	サブディレクトリ編集のみで CI に反映されない	パ
`deploy-landing-gcp.yml` 認証方式	`workload_identity_provider` (secret 未設定)	``
`deploy-landing-aws.yml` 認証方式	`role-to-assume` (secret 未設定)	`` (
フロントエンドがランディングページを上書き	`dist/` がバケットルートに同期された	``

バグ	症状	修
AWS/Azure staging で `AUTH_DISABLED=true`	デプロイ後も認証が無効のまま	新 認
GCP URLMap `Error 412: Invalid fingerprint`	Pulumi 状態と GCP リソースが非同期	[illegible]
GCP Cloud Build `missing main.py`	Cloud Build が `--entry-point` を指定してもソースを拒否	[illegible]
Azure `ModuleNotFoundError: pulumi_azuread`	`pulumi-azuread` が誤って `requirements.txt` から削除された	[illegible]
Azure `ModuleNotFoundError: monitoring`	`monitoring.py` が `main` には存在するが `develop` にない	[illegible]
Azure FC1: 誤った `az functionapp create`	`--consumption-plan-location` で Y1 Dynamic 作成 → AFD で stale TCP 502	[illegible]
GCP `Error 409: uploads-bucket exists`	Pulumi が GCP 内に既存のバケットを作成しようとした	[illegible]
GCP `ManagedSslCertificate Error 400: in use`	ブランチ間で SSL 証明書名ハッシュが変更；GCP が削除を拒否	証 [illegible]
Firebase Auth: 新ドメインが未認可	`signInWithPopup` 失敗 — ドメインが Firebase 認可ドメインにない	[illegible]
Azure dead CI ステップ (SSR Lambda)	React SPA 移行後も古い `frontend_web` Lambda/S3 ステップが実行	[illegible]
		[illegible]

バグ	症状	修
Lambda Layer 名が常に `staging`	`github.event.inputs.environment || 'staging'` → push イベントには `inputs` がない → production デプロイが `...-staging-dependencies` という名前のレイヤーを作成	
環境変数が消去される / デプロイごとに誤った環境	`Pulumi.*.yaml` が gitignore 対象 → `grep` が常に失敗 → リポジトリレベル Secrets へフォールバック → Secrets は環境非依存 → staging と production が同じドメイン/client_id	1 < e
Azure AD Client ID が CI デプロイ後に空	`pulumi stack output azure_ad_client_id` が空を返す → `VITE_AZURE_CLIENT_ID=''` → 認証プロバイダーが `'none'` → ログインページで「認証設定が不完全」エラー	後
AWS/Azure/GCP の誤ったカスタムドメイン	`secrets.AZURE_CUSTOM_DOMAIN` (リポジトリレベル) に staging ドメインが含まれる → production フロントエンドが `staging.azure.ashnova.jp` redirect URI でビルドされる	
AWS `ResourceConflictException` 競合	Pulumi の設定更新中に `update-function-code` が発行される	
Azure FC1: zip deploy で `InaccessibleStorageException`	`config-zip` が `BlobUploadFailedException: Name or service not known` で即座に失敗 — Kudu `StorageAccessibleCheck` 検証失敗 → すべての試行でステータス=3	A

バグ	症状	修
Azure FC1: `"on-going"` メッセージが CI エラー扱い	`config-zip` が `ERROR: Deployment is still on-going...` を返す — 古い `grep ERROR:` パターンがこの FC1 非同期受付応答にマッチ → パイプラインが即座にコード 1 で終了	
Azure FC1: デプロイ後の古い `WEBSITE_RUN_FROM_PACKAGE` で 404	デプロイがエラーなく完了するが全関数ルートが 404 を返す；設定内の古い期限切れ SAS URL が新デプロイパッケージをオーバーライド	
GitHub Actions YAML ブロックスカラー内の Python heredoc	`run: |` ブロック内の `python3 - <<'EOF' ... EOF` が `yaml.scanner.ScannerError: while scanning a simple key` を引き起こす — YAML パーサーが 0 列目の `EOF` でエラー	
Azure FC1: `POST /uploads/presigned-urls` → 500	`AZURE_STORAGE_ACCOUNT_NAME` / `AZURE_STORAGE_ACCOUNT_KEY` / `AZURE_STORAGE_CONTAINER` が production Function App で欠落 → `generate_blob_sas(account_name=None)` が例外発生	

バージョン管理

> 各アーティファクトは **A.B.C.D** (4桁) 形式でバージョン追跡されます。 バージョン定義ファイル: `versions.json` `X.Y.Z` から `A.B.C.D` に変更 (2026-02-24, commit `c2f6870`)

現在のバージョン (2026-02-24)

コンポーネント	バージョン	ステータス
`aws-static-site`	`1.0.84.204`	stable
`azure-static-site`	`1.0.84.204`	stable
`gcp-static-site`	`1.0.84.204`	stable
`simple-sns`	`1.0.84.204`	stable

バージョンインクリメント規則

桁	記号	意味	インクリメント条件	担当
1	A	メジャー	手動指示のみ	`./scripts/bump-version.sh major all`
2	B	マイナー	手動指示のみ	`./scripts/bump-version.sh minor all`
3	C	プッシュ数	`git push` のたびに +1（リセットなし）	GitHub Actions `version-bump.yml` が自動実行
4	D			

桁	記号	意味	インクリメント条件	担当
		コミット数	`git commit` のたびに +1（リセットなし）	`.githooks/pre-commit` が自動実行

> 重要: どの桁をインクリメントしても他の桁はリセットしません。 パスフィルタ: `static-site/aws/**` → A のみ、 `services/frontend_react/**` → SNS のみ、それ以外 → 全コンポーネント

実装ファイル

ファイル	役割
`versions.json`	各コンポーネントの現在バージョンを保持
`scripts/bump-version.sh`	バージョン操作スクリプト
`.githooks/pre-commit`	コミット前に D（コミット数）を自動インクリメント
`.github/workflows/version-bump.yml`	push 時に C（プッシュ数）を自動インクリメント

セットアップ (初回のみ)

```
make hooks-install
# → git config core.hooksPath .githooks を実行
# → 各コミット時に D を自動インクリメント
```

よく使うコマンド

```
# 現在のバージョンを表示
./scripts/bump-version.sh show

# D (+1) — pre-commit hook が自動実行（手動実行も可）
./scripts/bump-version.sh commit all

# C (+1) — GitHub Actions が自動実行（手動実行も可）
./scripts/bump-version.sh push all

# B (+1) — 手動指示で実行
./scripts/bump-version.sh minor all

# A (+1) — 手動指示で実行
./scripts/bump-version.sh major all

# バージョン直接設定（移行・修正時）
./scripts/bump-version.sh set all 1.0.84.204
```

バージョンバンプをスキップ

対象	スキップ方法
pre-commit hook (D +=1)	環境変数: `SKIP_VERSION_BUMP=1 git commit -m "..."`
GitHub Actions (C +=1)	コミットメッセージに `[skip-version-bump]` を含める

```
# pre-commit フックをスキップしてコミット
SKIP_VERSION_BUMP=1 git commit -m "docs: update readme"
```

```
        # GitHub Actions バンプをスキップ
        git commit -m "chore: some change [skip-version-bump]"
```

次のセクション

→ 06 — 環境ステータス

06 — 環境ステータス

> Part III — Operations | Parent: AI_AGENT_GUIDE.md 最終確認: 2026-02-27 Session 4 (アーキテクチャ図アイコン強化完了 ☒ デュアルアイコン配置実装 ☒ ドキュメント更新完了 ☒ 前回: 2026-02-27 Session 3 (S1・S2・Task 13 完了 ☒ エンドポイント本番運用 ☒ README セキュリティ情報反映 ☒

セッション 2026-02-27 (継続 4): アーキテクチャ図アイコン強化

完了作業

タスク	結果	状況
デュアルアイコン配置実装	ノード左上（24px）+ テキスト横（20px）の2箇所にアイコン表示	☒
	foreignObject / text 要素の両方に対応する	☒

タスク	結果	状況
generate_icon_diagram.py JavaScript 更新	スマート検出ロジック実装	
3環境**HTML**再生成	staging/production/combined の3ファイル全て再生成	☒
CLOUD_ARCHITECTURE_MAPPER.md 更新	Features / Technical Details / Known Limitations セクション更新	☒
CHANGELOG.md 更新	2026-02-27 エントリに詳細な実装内容とファイルサイズ更新	☒
README.md アーキテクチャリンク追加	インタラクティブHTML図への直接リンク追加済み	☒

アイコン配置戦略

アイコン配置:

1. ノード左上コーナー (24x24px):
   - 位置: (rect.x + 6, rect.y + 6)
   - 目的: リソースタイプの素早い視覚識別
   - ノードサイズに関わらず常に表示
2. テキストインライン (20x20px):
   - 位置: ノードラベルテキスト左の4px

- 目的: テキスト関連付けで読みやすさ向上
- スマート検出: `foreignObject` と SVG `text` / `tspan` 要素の両方に対応

JavaScript DOM 操作:

```
        // 1. 左上コーナー
        const topIcon = createSVGImage(iconUrl, rectX + 6, rectY + 6,
24, 24);
        node.appendChild(topIcon);

        // 2. テキストインライン（foreignObject vs text 要素検出）
        if (textElement.tagName === "foreignObject") {
          textX = parseFloat(textElement.getAttribute("x") || 0);
          textY =
            parseFloat(textElement.getAttribute("y") || 0) + height /
2 - iconSize / 2;
        } else {
          const tspan = textElement.querySelector("tspan");
          textX = parseFloat((tspan ||
textElement).getAttribute("x") || 0);
          textY =
            parseFloat((tspan || textElement).getAttribute("y") || 0)
- iconSize / 2;
        }
        const textIcon = createSVGImage(iconUrl, textX - 24, textY,
20, 20);
        labelGroup.insertBefore(textIcon, labelGroup.firstChild);
```

生成ファイル

ファイル	サイズ	アイコン	説明
`architecture.staging.html`	78KB	AWS (5) + Azure (4) + GCP (5)	ステージング環境（デュアルアイコン配置）
`architecture.production.html`	78KB	AWS (5) + Azure (4) + GCP (5)	本番環境（デュアルアイコン配置）
`architecture-combined.html`	84KB	AWS (5) + Azure (4) + GCP (5)	統合ビュー（color-coded nodes）

アイコンソース:

- AWS: 14KB (cloudfront, lambda, s3, dynamodb, api-gateway)
- Azure: 16KB (cdn, function, storage, cosmos-db)
- GCP: 20KB (cdn, run, storage, firestore, load-balancer)
- 埋め込みアセット合計: ~50KB Base64 エンコード SVG データ URI

ドキュメント更新

☒**CLOUD_ARCHITECTURE_MAPPER.md**:

- Features セクション: 「Dual icon placement」箇条書きを追加

- Technical Details: デュアル配置ロジックを含む JavaScript コードサンプルを拡張
- Known Limitations: テキストインライン配置の分散に関する注記を追加

☒**CHANGELOG.md**:

- 2026-02-27 エントリに詳細な実装ノートを更新
- ファイルサイズ変更を追加（staging/production で 85KB → 78KB）
- デュアルアイコン配置戦略を文書化

☒**README.md**:

- アーキテクチャセクションに 3 つのインタラクティブ HTML 図へのリンクを追加
- 図リンクに視覚的インジケーター（☒ を追加

セッション 2026-02-27 (継続 3): セキュリティデプロイ・ドキュメント更新

完了作業

タスク	結果	状況
S1: GCP ステージング pulumi up	HTTPS リダイレクト / 監査ログ反映済み（unchanged 33）	☒
S1: AWS 本番 pulumi up	CloudTrail / CORS 反映済み（unchanged 40）	☒

タスク	結果	状況
S1: GCP 本番 refresh+up	State drift 解決後、監査ログ反映済み（unchanged 34）	☒
S1: Azure ステージング pulumi up	Key Vault パージ保護反映（updated 1, unchanged 32）	☒
S1: Azure 本番 pulumi up	Key Vault パージ保護本番反映済み（unchanged 33）	☒
S2: Function App マネージド ID	ステージング/本番 両方に SystemAssigned MSI 割り当て成功	☒
Task 13: README 更新	エンドポイント・セキュリティ実装・テスト結果・デプロイ状況を最新情報に更新	☒
Task 20/21 補完: Key Vault 診断設定（CLI）	`az monitor diagnostic-settings create` で Log Analytics との統合が完了（AuditEvent ストリーミング）	☒

本番エンドポイント (2026-02-27 現在)

クラウド	**CDN** / フロントエンド	**API**	ステータス
AWS	CloudFront ☒	API Gateway	☒本番運用中
Azure	Front Door ☒	Functions	

クラウド	**CDN** / フロントエンド	**API**	ステータス
			☒本番運用中
GCP	CDN（www.gcp.ashnova.jp）☒	Cloud Functions	☒本番運用中

セキュリティ実装ステータス (本番環境にデプロイ済み)

対策	**AWS**	**Azure**	**GCP**	ステータス
CORS 絞り込み	☒	☒	☒	☒本番反映
CloudTrail / 監査ログ	☒	☒	☒	☒本番反映
Key Vault パージ保護	—	☒	—	☒本番反映
Key Vault 診断ログ	—	☒（Log Analytics）	—	☒本番反映
マネージド ID	—	☒	—	☒ ステージング/本番
HTTPS リダイレクト	—	—	☒	☒本番反映

対策	AWS	Azure	GCP	ステータス
Cloud Armor	—	—	☒	☒本番反映

セッション 2026-02-27: GCP 監査ログ・課金予算修復

完了作業

タスク	結果	状況
GCP 監査ログ再有効化 (IAMAuditConfig)	staging/production で Cloud Audit Logs (`allServices`) を有効化。Pulumi リソース作成完了	☒
ADC (Application Default Credentials) 更新	`gcloud auth application-default login` で sat0sh1kawada00@gmail.com 再認証。quota project=ashnova に設定	☒
GCP 課金アカウント設定	Pulumi設定に `billingAccountId: 01F139-282A95-9BBA25` を追加	☒
課金予算エラー軽減	ADC quota project エラー回避。monitoring.py で `billing_account_id` を optional パラメータ化。コードで `enable_billing_budget=False` にデフォルト無効化	☒
GCP side budget cleanup	`gcloud billing budgets delete` で古い budgetリソース削除	☒
		☒

タスク	結果	状況
monitoring.py リファクタリング	`create_billing_budget()` に `billing_account_id` 追加。`setup_monitoring()` で `billing_budget=None` when not enabled	

コード変更

- **infrastructure/pulumi/gcp/main.py**: `enable_billing_budget = False` (ハードコード無効化) + `billing_account_id=None` を常時 monitoring へ参照
- **infrastructure/pulumi/gcp/monitoring.py**: `billing_account_id` パラメータ追加、optional化。budget作成条件を `if stack=="production" and billing_account_id:` に変更

既知の課題 / 次のステップ

- GCP production `pulumi up` が preview conflict 状態。コード修正後は再実行予定（次セッション）
- staging/production 共に監査ログ有効化完了、billing warning 回避完了
- billingbudgets API 認証エラーは ADC quotaProjectで解消するが、service account接続時の権限不足で deprecated。本番運用では GCP service account 設定または ignore_changes で対応推奨

ステージング環境概要

クラウド	ランディング (`/`)	**SNS** アプリ (`/sns/`)	**API**
GCP	☒	☒	☒Cloud Run (2026-02-24)
AWS	☒	☒	☒ Lambda (完全運用中, 2026-02-24)
Azure	☒	☒	☒ Azure Functions FC1 (2026-02-24)

AWS (ap-northeast-1)

```
CDN URL  : https://d1tf3uumcm4bo1.cloudfront.net
API URL  : https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com
```

リソース	名前 / **ID**	ステータス
CloudFront	`E1TBH4R432SZBZ` (PriceClass_200: NA/EU/JP/KR/IN)	☒
CloudFront RHP	`multicloud-auto-deploy-staging-security-headers` (HSTS + CSP + 4 headers)	☒
S3 (フロントエンド)	`multicloud-auto-deploy-staging-frontend`	☒
S3 (画像)		☒

リソース	名前 / **ID**	ステータス
	`multicloud-auto-deploy-staging-images` (CORS: *)	
Lambda (API)	`multicloud-auto-deploy-staging-api` (Python 3.12, **1769MB** = 1 vCPU)	☒
Lambda (frontend-web)	`multicloud-auto-deploy-staging-frontend-web` (512MB, 30s)	☒
API Gateway	`z42qmqdqac` (HTTP API v2)	☒
DynamoDB	`multicloud-auto-deploy-staging-posts` (PAY_PER_REQUEST)	☒
Cognito	Pool `ap-northeast-1_AoDxOvCib` / Client `1k41lqkds4oah55ns8iod30dv2`	☒
WAF	WebACL が CloudFront に紐付け済み	☒

確認済み機能 (2026-02-22):

- Cognito ログイン → `/sns/auth/callback` → セッションクッキー設定 ☒
- ポストフィード、最大10枚のマルチイメージ投稿作成 ☒
- 画像が正しく表示（S3 プリサインド GET URL、1時間有効） ☒
- `GET /posts/{post_id}` 個別ポスト表示 ☒
- プロフィールページ（ニックネーム、アバター、自己紹介） ☒
- ニックネームがポストリストに保存・表示 ☒

- 画像アップロード: S3 プリサインド URL、`MAX_IMAGES_PER_POST` でサーバーサイド制限 ☒
- `GET /limits` エンドポイント（認証不要）が `{"maxImagesPerPost": 10}` を返す ☒
- ログアウト → Cognito ホステッドログアウト → `/sns/` へリダイレクト ☒
- CI/CD パイプライン: push ごとに環境変数が正しく設定される ☒
- フロントエンドバンドルが `VITE_BASE_PATH=/sns/` でビルドされ、アセットパスが正常 ☒
- CloudFront カスタムエラーページ: `/sns/index.html` (403+404) ☒
- CloudFront Response Headers Policy: HSTS/CSP(`upgrade-insecure-requests`)/X-Content-Type-Options/X-Frame-Options/Referrer-Policy/XSS-Protection ☒(2026-02-23)
- CloudFront PriceClass_200: 日本・韓国・インドのエッジを使用 ☒(旧: PriceClass_100 = 米国/欧州のみ)
- OAuth フロー PKCE (S256) 実装: `response_type=code` + code_verifier/challenge ☒(2026-02-23)
- Cognito `implicit` フロー削除: `allowed_oauth_flows=["code"]` のみ ☒(2026-02-23)
- S3 パブリックアクセス完全遮断: `BlockPublicAcls/IgnorePublicAcls/BlockPublicPolicy/RestrictPublicBuckets=True` ☒(2026-02-23)
- S3 バケットポリシー OAI 専用: `Principal:*` 削除 ☒(2026-02-23)
- Lambda `_resolve_image_urls` : `http://` URL をスキップして Mixed Content を防止 ☒(2026-02-23)

現在のフロントエンドバンドル: `index-B0gzRu__.js` (2026-02-23 アップロード, PKCE対応)

AWS staging ビルドコマンド:

```
cd services/frontend_react
set -a && source .env.aws.staging && set +a
VITE_BASE_PATH=/sns/ npm run build
```

既知の制限:

- Production スタックは staging リソースと共有（独立した prod スタック未デプロイ）
- WAF ルールセット未調整
- `DELETE /posts` で SNS Unsubscribe 呼び出しが失敗する可能性（未テスト）

Azure (japaneast)

```
CDN URL  : https://mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net
API URL  : https://multicloud-auto-deploy-staging-func-d8a2guhfere0etcq
```

リソース	名前	ステータス
Front Door	`multicloud-auto-deploy-staging-fd` / endpoint: `mcad-staging-d45ihd`	☒
Storage Account	`mcadwebd45ihd`	☒
Function App	`multicloud-auto-deploy-staging-func` (Python 3.12, always-ready=1)	☒

リソース	名前	ステータス
Cosmos DB	`messages` (Serverless, db: messages / container: messages)	☒
Resource Group	`multicloud-auto-deploy-staging-rg`	☒

構成済み (2026-02-23):

- FlexConsumption always-ready インスタンス: `http=1` → コールドスタート解消 ☒

未解決の課題:

- `PUT /posts/{id}` のエンドツーエンド検証が不完全
- WAF 未構成 (Front Door Standard SKU)

GCP (asia-northeast1)

```
CDN URL : http://34.117.111.182
API URL : https://multicloud-auto-deploy-staging-api-899621454670.asia-
```

リソース	名前 / **ID**	ステータス
グローバル IP	`34.117.111.182`	☒

リソース	名前 / **ID**	ステータス
GCS バケット (前面)	`ashnova-multicloud-auto-deploy-staging-frontend`	☒
GCS バケット (画像)	`ashnova-multicloud-auto-deploy-staging-uploads` (public read)	☒
Cloud Run (API)	`multicloud-auto-deploy-staging-api` (Python 3.12, **min=1**)	☒
Firestore	`(default)` — collections: messages, posts	☒
Backend Bucket	`multicloud-auto-deploy-staging-cdn-backend`	☒

確認済み機能 (2026-02-21):

- Firebase Google Sign-In → `/sns/auth/callback` → httponly Cookie セッション ☒
- ポストフィード、作成/編集/削除 ☒
- 画像アップロード: GCS プリサインド URL (IAM `signBlob` API 経由署名)、最大16ファイル/投稿 ☒
- アップロードされた画像がポストフィードに表示される ☒
- Firebase ID トークン自動更新 (`onIdTokenChanged`) ☒
- ダークテーマ背景 SVG (星空、リング) が正しくレンダリングされる ☒

修正済みの課題 (2026-02-21):

- `GcpBackend` に未実装の `like_post` / `unlike_post` 抽象メソッド → `TypeError` → `/posts` が 500 を返す → `like_post` / `unlike_post` のスタブ実装を追加（コミット `a9bc85e`）
- `frontend-web` Cloud Run で `API_BASE_URL` 未設定 → localhost:8000 へフォールバック → `gcloud run services update` で環境変数を設定
- Firebase Auth 未実装 → Google Sign-In フロー全体を実装（コミット `3813577`）
- `x-ms-blob-type` ヘッダーが GCS CORS に未登録 → CORS 更新 + uploads.js 修正（コミット `1cf53b7`, `b5b4de5`）
- GCS プリサインド URL 生成で `content_type` が `"image/jpeg"` にハードコード → `content_types[index]` を正しく使用（コミット `148b7b5`）
- Firebase ID トークン期限切れ（401）→ `onIdTokenChanged` で自動更新（コミット `8110d20`）
- CI/CD で `GCP_SERVICE_ACCOUNT` 環境変数欠落 → `deploy-gcp.yml` に追加（コミット `27b10cc`）
- CSS 背景 SVG が絶対パス `/static/` を使用 → 相対パス `./` に変更（コミット `0ed0805`）
- GCS アップロードバケットが非公開 → `allUsers:objectViewer` を付与 + Pulumi 定義に IAMBinding 追加（コミット `0ed0805`）

構成済み (2026-02-23):

- Cloud Run `--min-instances=1` → コールドスタート（最大5秒）解消 ☒
- `gcp_backend.py`：`google.auth.default()` を `__init__()` でキャッシュ → リクエストごとのメタデータサーバー呼び出し排除 ☒

残る課題:

- CDN に HTTPS 未構成（HTTP のみ）。 `TargetHttpsProxy` + マネージド SSL 証明書が必要
- CDN 経由の SPA ディープリンクが HTTP 404 を返す（Cloud Run URL はブラウザで正常動作）

接続確認コマンド

```
        # AWS
        curl -I https://d1tf3uumcm4bo1.cloudfront.net/
        curl -s https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com/health

        # Azure
        curl -I https://mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net/
        curl -s "https://multicloud-auto-deploy-staging-func-d8a2guhfere0etcq.japaneast-01.azurewebsites.net/health"

        # GCP
        curl -s http://34.117.111.182/ | head -3
        curl -s https://multicloud-auto-deploy-staging-api-899621454670.asia-northeast1.run.app/health
```

本番環境

> 本番環境は独自の Pulumi スタック（デプロイ完了）を持ち、ステージングと完全に分離されています。 フロントエンドは **React SPA**（Vite ビルド）として、CDN を経由するオブジェクトストレージから配信されます。 `frontend_web` （Python SSR）は本番環境では使用されな

> くなりました。 詳細マイグレーション情報: REACT_SPA_MIGRATION_REPORT.md

本番ステータス概要 (2026-02-23)

本番環境のすべての API エンドポイントおよび CDN が正常に稼働しており、3つのクラウド環境すべてで同期されています。

クラウド	**CDN** ランディング (`/`)	**SNS** アプリ (`/sns/`)	**API**
AWS	☒HTTP 200	☒React SPA	☒ /health /posts ok
Azure	☒HTTP 200	☒React SPA	☒/api/health ok
GCP	☒HTTP 200	☒React SPA	☒ /health /limits ok

本番エンドポイント

3つのクラウドプロバイダーの本番環境における CDN / フロントエンド配信エンドポイント、API エンドポイント、および Distribution ID / リソース識別子を以下に示します。

クラウド	**CDN /** エンドポイント	**API** エンドポイント
AWS	`d1qob7569mn5nw.cloudfront.net` / `www.aws.ashnova.jp`	`https://qkzypr32af.execute-a` `northeast-1.amazonaws.com`
Azure	`mcad-production-diev0w-f9ekdmehb0bga5aw.z01.azurefd.net`	`https://multicloud-auto-depl` `cfdne7ecbngnh0d0.japaneast-01`

クラウド	**CDN /** エンドポイント	**API** エンドポイント
GCP	`www.gcp.ashnova.jp`	`https://multicloud-auto-depl` `son5b3ml7a-an.a.run.app`

AWS Production SNS App (`https://www.aws.ashnova.jp/sns/`):

AWS 本番環境の SNS アプリケーション構成の詳細。すべてのコアサービスが Pulumi により管理・デプロイされています。

項目	値
フロントエンド	React SPA — S3 `multicloud-auto-deploy-production-frontend/sns/`
CF Function	`spa-sns-rewrite-production` (LIVE) — `/sns/` → `/sns/index.html` 変換
Lambda (API)	`multicloud-auto-deploy-production-api`
API_BASE_URL	`https://qkzypr32af.execute-api.ap-northeast-1.amazonaws.com`
Cognito Pool	`ap-northeast-1_50La963P2`
Cognito Client	`4h3b285v1a9746sqhukk5k3a7i`
Cognito Redirect	`https://www.aws.ashnova.jp/sns/auth/callback`

項目	値
DynamoDB	`multicloud-auto-deploy-production-posts`

カスタムドメインステータス (ashnova.jp) — 2026-02-21

`ashnova.jp` ドメインの3つのクラウド環境での SSL/TLS デプロイ状態。すべてのカスタムドメインが完全に運用中であり、HTTPS による安全な通信が確立されています。

クラウド	**URL**	ステータス
AWS	https://www.aws.ashnova.jp	☒完全運用中 (HTTP/2 200, ACM 証明書設定済み)
Azure	https://www.azure.ashnova.jp	☒ 完全運用中 (HTTPS 200, DigiCert/GeoTrust 管理証明書)
GCP	https://www.gcp.ashnova.jp	☒完全運用中 (HTTPS 200, TLS 証明書 Pulumi 管理)

Landing page test (2026-02-23): `test-landing-pages.sh --env production` → **37/37 PASS (100%)** ☒

完了作業 (2026-02-21)

クラウド	作業内容	結果
AWS	ACM 証明書検証	☒証明書 `914b86b1` の `www.aws.ashnova.jp` (2027-03-12 有効) ISSUED 確認

クラウド	作業内容	結果
AWS	`aws cloudfront update-distribution` でエイリアス+ACM設定	☒Distribution `E214XONKTXJEJD` にエイリアス `www.aws.ashnova.jp` + 証明書 `914b86b1` を設定 → `NET::ERR_CERT_COMMON_NAME_INVALID` 解決 → HTTP/2 200 運用中
AWS	Production `frontend-web` Lambda 環境変数修正 (2026-02-21)	☒ `API_BASE_URL` が空→ `localhost:8000` フォールバック（原因: `deploy-frontend-web-aws.yml` がシークレット依存、本番シークレット未設定）→ CI/CD を Pulumi outputs 使用に更新（コミット `fd1f422` ）
Azure	`az afd custom-domain create` + ルート紐付け	☒ DNS 承認 → 管理証明書成功 (GeoTrust, 2026-02-21 – 2026-08-21)
Azure	AFD ルート無効化→有効化トグル	☒ エッジノードへのデプロイをトリガー → HTTPS 200 運用中
Azure	`az afd custom-domain update` (証明書エッジデプロイ)	☒ `CN=www.azure.ashnova.jp` 証明書を AFD POP に配布

クラウド	作業内容	結果
Azure	`frontend-web` Function App 環境変数設定	☒ API_BASE_URL, AUTH_PROVIDER, AZURE_TENANT_ID, AZURE_CLIENT_ID 等を設定
Azure	Azure AD app redirect URI 追加	☒ `https://www.azure.ashnova.jp/sns/auth/callback` を追加
GCP	`pulumi up --stack production` (SSL証明書作成)	☒ 証明書 `multicloud-auto-deploy-production-ssl-cert-3ee2c3ce` PROVISIONING
GCP	ACTIVE 証明書 `ashnova-production-cert-c41311` 追加	☒ HTTPS プロキシに追加 → `https://www.gcp.ashnova.jp` HTTPS 即座に運用開始
GCP	Firebase 認可ドメイン更新	☒ Firebase Auth 認可ドメインに `www.gcp.ashnova.jp` を追加

残課題

- **GCP**: ☒ `ashnova-production-cert-c41311` を HTTPS プロキシから切り離し・削除済み (2026-02-24)。`multicloud-auto-deploy-production-ssl-cert-3ee2c3ce` のみ使用中。推奨される設定状態は以降の pulumi up では自動的に維持されます。

☒Production Issues — 全件解決済み (2026-02-24 v5)

本番環境で検出された 7 つのイシューはすべて解決され、確認テストも全て PASS しています。

☒1. Azure Function App — 0 registered functions (API 404) — RESOLVED 2026-02-24

症状: `/api/health` → HTTP 404。Function App は Running 状態だが、Admin インターフェースで関数が表示されない。

根本原因 (多層的で複雑):

1. `AzureWebJobsStorage` が非存在ストレージ `multicloudautodeploa148` を指していた
2. `functionAppConfig.deployment.storage.value` が非存在 blob URL を指していた
3. 決定的原因: `functionAppConfig.runtime.version = "3.12"` だが、デプロイされた zip は Python 3.11 (`cpython-311`) でビルドされていた → クラウド実行時にモジュールロード失敗 → 関数が読み込まれない

修正: Python 3.12 (`docker run python:3.12-slim`) でパッケージを正しいアーキテクチャ向けに再ビルド・デプロイ

確認: `admin/functions` → `[{"name":"HttpTrigger"}]` ☒/ `/api/health` → HTTP 200 ☒

☒3. GCP Production API — `/limits` エンドポイント 404 ー **RESOLVED 2026-02-24**

症状: `https://multicloud-auto-deploy-production-api-son5b3ml7a-an.a.run.app/limits` へのリクエストが HTTP 404 を返す。直接 Cloud Run サービスへのアクセスも同様に失敗。

修正: GCP production API を最新リビジョン (`00013-big`) に再デプロイ。 `CORS_ORIGINS` 環境変数に本番ドメイン `https://www.gcp.ashnova.jp` を追加して、クロスオリジンリクエストを許可。

確認済み: `GET /limits` → `{"maxImagesPerPost":10}` HTTP 200 ☒ CDN 経由のリクエストも成功

☒4. AWS Production CloudFront — セキュリティヘッダーポリシー未設定 ー **RESOLVED** 2026-02-24

症状: CloudFront Distribution が HSTS、CSP、X-Frame-Options などのセキュリティヘッダーを返していない。

修正: 既存ポリシー `multicloud-auto-deploy-production-security-headers` (ID: `aaad020f-c94c-4143-ba2c-4b7921a1a6de`) を DefaultCacheBehavior と `/sns*` CacheBehavior の両方に適用。ETag が `E3P5R0KL5A1OLE` から `E3JWKAKR8XB7XF` に更新されました。

確認され: CloudFront Distribution `E214X0NKTXJEJD` がすべてのセキュリティヘッダー（HSTS/CSP/X-Content-Type-Options/X-Frame-Options/Referrer-Policy/XSS-Protection）をレスポンスに含める ☒

⌧5. AWS Production SNS — Network Error (CI/CD customDomain 上書き) — RESOLVED 2026-02-24

症状: `https://www.aws.ashnova.jp/sns/` の SNS アプリで API 呼び出し時に “Network Error” が発生。Axios が HTTP status 0 を返す（CORS エラーの典型的な兆候）。

根本原因チェーン:

1. `deploy-aws.yml` の “Sync Pulumi Config” ステップが GitHub リポジトリシークレット `${{ secrets.AWS_CUSTOM_DOMAIN }}` = `staging.aws.ashnova.jp` を使用
2. `pulumi config set multicloud-auto-deploy-aws:customDomain "staging.aws.ashnova.jp"` が本番 `Pulumi.production.yaml` を誤って上書き
3. `pulumi stack output custom_domain` が `staging.aws.ashnova.jp` を返す（期待値: `www.aws.ashnova.jp`）
4. Lambda 環境変数 `CORS_ORIGINS` = `...,https://staging.aws.ashnova.jp,...` （誤）→ FastAPI が `Origin: https://www.aws.ashnova.jp` を拒否
5. ブラウザが “Network Error” を報告

修正 (コミット **`3ea6a08`** **v1.17.10)**:

- `deploy-aws.yml` を GitHub Secrets ではなく `Pulumi.${STACK_NAME}.yaml` から値を読むように修正
- React SPA 再ビルド・デプロイ（新バンドル `index-Ch-ro-3Y.js`）
- Lambda `CORS_ORIGINS` から `staging.aws.ashnova.jp` を即時削除

確認: Lambda `CORS_ORIGINS` = `https://d1qob7569mn5nw.cloudfront.net,https://www.aws.ashnova.jp,http://localhost:5173` ☒

☒6. Azure プロフィール画面 CORS エラー — RESOLVED 2026-02-24

症状: `https://www.azure.ashnova.jp/sns/profile` でプロフィール取得時に CORS エラー

根本原因:

1. Azure Function App は Kestrel がプラットフォームレベル CORS 判定を FastAPI `CORSMiddleware` の手前に実行
2. `deploy-azure.yml` が `secrets.AZURE_CUSTOM_DOMAIN` (= `staging.azure.ashnova.jp`) を使用
3. `CORS_ORIGINS` とプラットフォーム CORS の両方に `https://www.azure.ashnova.jp` が欠落

即時修正:

- `az functionapp config appsettings set ... CORS_ORIGINS=...,https://www.azure.ashnova.jp,...` ☒
- `az functionapp cors add --allowed-origins "https://www.azure.ashnova.jp"` ☒

根本修正 (**v1.17.15**): `deploy-azure.yml` を stack 名から `customDomain` を読むように変更

☒7. Azure ログイン後に staging SNS に遷移 — RESOLVED 2026-02-24

症状: `www.azure.ashnova.jp/sns/` でログイン後に `staging.azure.ashnova.jp/sns/` にリダイレクト

根本原因:

1. フロントエンドが `VITE_AZURE_REDIRECT_URI=https://staging.azure.ashnova.jp/sns/auth/callback` でビルドされていた
2. Azure AD アプリの redirect URIs に `www.azure.ashnova.jp` がなかった

即時修正:

- `az ad app update` で `www.azure.ashnova.jp` を redirect URIs に追加 ☒
- フロントエンドを `VITE_AZURE_REDIRECT_URI=https://www.azure.ashnova.jp/sns/auth/callback` で再ビルド → `index-CPcQQsCR.js` ☒

根本修正 (**v1.17.16**): `deploy-azure.yml` の全4箇所の `secrets.AZURE_CUSTOM_DOMAIN` を stack マッピングに変更

テスト結果 (2026-02-24):

すべての本番環境エンドポイントでテストを実施し、完全な機能動作を確認しました。

```
test-cloud-env.sh production → PASS: 14, FAIL: 0, WARN: 3
test-azure-sns.sh            → PASS: 10, FAIL: 0
test-gcp-sns.sh              → PASS: 10, FAIL: 0
```

E2E テストスクリプト (2026-02-24)

> commit `73af560` — `scripts/` 配下の4ファイルを改良

`test-sns-all.sh` (新規)

3クラウド統合ラッパー。すべてのクラウドを一括でテストし、最後にサマリーテーブルを表示する。

```
        # 基本使用（read-only, production）
        bash scripts/test-sns-all.sh --env production

        # 特定クラウドのみ
        bash scripts/test-sns-all.sh --env production --only azure

        # 書き込みテスト有効（AWS: Cognito 自動認証）
        bash scripts/test-sns-all.sh --env production --write ¥
          --aws-username user@example.com --aws-password *** --aws-
client-id 4h3b285v1a9746sqhukk5k3a7i

        # 書き込みテスト有効（GCP: gcloud 自動認証）
        bash scripts/test-sns-all.sh --env production --write --gcp-
auto-token
```

サマリー出力例 (production read-only, 2026-02-24):

```
Cloud        PASS    FAIL    SKIP  Status
──────────────  ──────────  ──────────  ──────────  ──────────────────
aws            9       0       4  ☒PASS
azure         17       0       2  ☒PASS
gcp           13       0       4  ☒PASS
──────────────  ──────────  ──────────  ──────────  ──────────────────
TOTAL         39       0      10
```

各スクリプトの改良内容

スクリプト	追加機能
`scripts/test-sns-aws.sh`	`--username` / `--password` / `--client-id` で Cognito 自動認証、X-Amz-Signature 検証、binary PUT to S3、imageUrl HTTP 200 確認
`scripts/test-sns-gcp.sh`	`--auto-token` で `gcloud auth print-identity-token` 自動認証、X-Goog-Signature 検証、binary PUT to GCS、imageUrl HTTP 200 確認
`scripts/test-sns-azure.sh`	`x-ms-blob-type: BlockBlob` で binary PUT to Azure Blob (HTTP 201)、SAS read URL HTTP 200 確認

テスト一覧 (**write** モード時の追加項目)

#	テスト	概要
5-2	署名URL検証	presigned URL に `X-Amz-Signature=` / `X-Goog-Signature=` / SAS token が含まれることを確認
5-3		

#	テスト	概要
	binary PUT	1×1 PNG を実際に presigned URL へ PUT し HTTP 200/201 を確認
5-4	imageUrl アクセス確認	PUT したキーで POST /posts → GET /posts/:id → imageUrls[0] に GET → HTTP 200 を確認

コスト監視ツール

マルチクラウド + GitHub の費用を一元管理するツールが `scripts/` 配下に実装済みです。

CLI レポート（`scripts/cost_report.py`）

```
python3 scripts/cost_report.py                # デフォルト: 過去3ヶ月
python3 scripts/cost_report.py --months 6     # 過去6ヶ月
python3 scripts/cost_report.py --json         # JSON 出力
python3 scripts/cost_report.py --aws-only     # AWS のみ
python3 scripts/cost_report.py --azure-only   # Azure のみ
```

macOS メニューバーウィジェット（`scripts/mac-widget/`）

xbar を使った 1 時間ごと自動更新ウィジェット。

```
brew install --cask xbar
bash scripts/mac-widget/install.sh
open -e scripts/mac-widget/.env    # 認証情報を設定
```

通貨処理

Provider	方式
AWS	Cost Explorer は USD 固定 → open.er-api.com で リアルタイム USD/JPY 変換 (失敗時 ¥150 固定)
Azure	Cost Management API の `rows[n][2]` から通貨コードを直接取得 (JPY)
GCP	Billing API — サービスアカウント or `gcloud auth`
GitHub	Billing API 廃止 (HTTP 410) → `actions/cache/usage` + runs 件数で代替

.env 設定ファイル

`scripts/mac-widget/.env` (git 管理外) に認証情報を記載します。 テンプレート: `scripts/mac-widget/cost-monitor.env.sample`

変数	用途
`AZURE_SUBSCRIPTION_ID`	Azure Cost Management
`GCP_BILLING_ACCOUNT`	GCP Billing (`01XXXX-XXXXXX-XXXXXX` 形式)
`GCP_PROJECT_ID`	GCP プロジェクト ID
`GITHUB_TOKEN`	GitHub Actions 使用量取得

変数	用途
`GH_REPO`	`owner/repo` 形式 (個人リポジトリ用)

AWS は `~/.aws/credentials` の default プロファイルを使用（追加設定不要）。

マルチクラウド + GitHub の費用を一元管理するツールが `scripts/` 配下に実装済みです。

CLI レポート（`scripts/cost_report.py`）

```
python3 scripts/cost_report.py                # デフォルト: 過去3ヶ月
python3 scripts/cost_report.py --months 6     # 過去6ヶ月
python3 scripts/cost_report.py --json         # JSON 出力
python3 scripts/cost_report.py --aws-only     # AWS のみ
python3 scripts/cost_report.py --azure-only   # Azure のみ
```

macOS メニューバーウィジェット（`scripts/mac-widget/`）

xbar を使った 1 時間ごと自動更新ウィジェット。

```
brew install --cask xbar
bash scripts/mac-widget/install.sh
open -e scripts/mac-widget/.env    # 認証情報を設定
```

通貨処理

プロバイダー	方式
AWS	Cost Explorer は USD 固定 → open.er-api.com で リアルタイム USD/JPY 変換 (失敗時 ¥150 固定)
Azure	Cost Management API の `rows[n][2]` から通貨コードを直接取得 (JPY)
GCP	Billing API — サービスアカウント or `gcloud auth`
GitHub	Billing API 廃止 (HTTP 410) → `actions/cache/usage` + runs 件数で代替

.env 設定ファイル

`scripts/mac-widget/.env` (git 管理外) に認証情報を記載します。 テンプレート: `scripts/mac-widget/cost-monitor.env.sample`

変数	用途
`AZURE_SUBSCRIPTION_ID`	Azure Cost Management
`GCP_BILLING_ACCOUNT`	GCP Billing (`01XXXX-XXXXXX-XXXXXX` 形式)
`GCP_PROJECT_ID`	GCP プロジェクト ID
`GITHUB_TOKEN`	GitHub Actions 使用量取得
`GH_REPO`	`owner/repo` 形式 (個人リポジトリ用)

AWS は `~/.aws/credentials` の default プロファイルを使用（追加設定不要）。

AWS 管理コンソールリンク

- API Gateway
- Lambda
- CloudFront

Azure ポータルリンク

- リソースグループ
- Function Apps
- Front Door

GCP コンソールリンク

- Cloud Run
- Cloud Storage
- Firestore

FinOps — GCP 未使用静的 IP アドレス監査 (2026-02-21)

> GCP FinOps から の指摘に対応して実施した監査。プロジェクト `ashnova` のすべての静的 IP アドレスをレビューしました。

すべての IP アドレス

```
gcloud compute addresses list --project=ashnova ¥
  --
format="table(name,address,status,addressType,users.list())"
```

名前	IP アドレス	ステータス	作成日	使用中
`multicloud-auto-deploy-production-cdn-ip`	34.8.38.222	☒IN_USE	—	Production CDN (Forwarding Rule ×2)
`multicloud-auto-deploy-staging-cdn-ip`	34.117.111.182	☒IN_USE	—	Staging CDN (Forwarding Rule ×2)
`ashnova-production-ip-c41311`	34.54.250.208	⚠ **RESERVED**	2026-02-11	なし
`multicloud-frontend-ip`	34.120.43.83	⚠ **RESERVED**	2026-02-14	なし
`simple-sns-frontend-ip`	34.149.225.173	⚠ **RESERVED**	2026-01-30	なし

未使用 IP の背景

名前	推定履歴
`simple-sns-frontend-ip`	プロジェクト初期（`simple-sns` という名前だった時期、2026-01-30）に作成。Pulumi コードや Forwarding Rule に参照なし。
`ashnova-production-ip-c41311`	Production CDN 用に作成（Pulumi サフィックス `c41311` から判断、2026-02-11）されたが、後に `multicloud-auto-deploy-production-cdn-ip` に置き換えられた。
`multicloud-frontend-ip`	2026-02-14 に作成。コードベースやドキュメントに参照なし。実験的に予約されて放置されたと推定。

> 注: これら3つはすべて Pulumi コードや Forwarding Rule とリンクされておらず、すぐに解放できます。

解放コマンド

```
gcloud compute addresses delete ashnova-production-ip-c41311 --global --project=ashnova --quiet
gcloud compute addresses delete multicloud-frontend-ip --global --project=ashnova --quiet
gcloud compute addresses delete simple-sns-frontend-ip --global --project=ashnova --quiet
```

> ⚠ 削除は不可逆です。実行前に `gcloud compute addresses describe <name> --global` で各 IP に関連リソースがないことを確認してください。

FinOps — GCP 未使用 Cloud Storage バケット監査 (2026-02-21)

> 静的 IP 監査のフォローアップとして実施。Terraform 時代のレガシーバケットと壊れた Cloud Function を特定しました。

すべてのバケット (プロジェクト: **ashnova)**

バケット名	サイズ	判定	備考
`ashnova-multicloud-auto-deploy-production-frontend`	—	☒ Active	Pulumi 管理
`ashnova-multicloud-auto-deploy-production-function-source`	5 MB	☒ Active	Pulumi 管理
`ashnova-multicloud-auto-deploy-production-uploads`	—	☒ Active	Pulumi 管理
`ashnova-multicloud-auto-deploy-staging-frontend`	—	☒ Active	Pulumi 管理
`ashnova-multicloud-auto-deploy-staging-function-source`	5 MB	☒ Active	Pulumi 管理
`ashnova-multicloud-auto-deploy-staging-landing`	8 KB	☒ Active	Pulumi 管理
`ashnova-multicloud-auto-deploy-staging-uploads`	—	☒ Active	Pulumi 管理

バケット名	サイズ	判定	備考
`ashnova.firebasestorage.app`	—	☒ Keep	Firebase システム管理
`ashnova_cloudbuild`	—	☒ Keep	Cloud Build システム管理
`gcf-v2-sources-899621454670-asia-northeast1`	433 MB	☒ Keep	アクティブな Cloud Function v2 のソース
`gcf-v2-uploads-899621454670.asia-northeast1.cloudfunctions.appspot.com`	—	☒ Keep	Cloud Functions アップロードステージング
`ashnova-staging-frontend`	**empty**	☒削除	Terraform レガシー。`ashnova-multicloud-auto-deploy-staging-frontend` に移行済み
`ashnova-staging-function-source`	**65 MB**	☒削除	Terraform レガシー。2026-02-14

バケット名	サイズ	判定	備考
			の古い zip を含む
`multicloud-auto-deploy-tfstate`	**empty**	☒削除	古い Terraform state バケット。空。
`multicloud-auto-deploy-tfstate-gcp`	**6 KB**	☒削除	上記2つのバケットの Terraform state のみ保持

削除可能バケットの背景

バケット名	推定履歴
`ashnova-staging-frontend`	旧 Terraform 設定のフロントエンドバケット（`ashnova-staging-*` 命名）。`ashnova-multicloud-auto-deploy-staging-frontend`（Pulumi 管理）に完全移行済み。空。
`ashnova-staging-function-source`	同じ Terraform 設定の Cloud Function ソースバケット。2026-02-14 の古い 65 MB の zip を含む。`ashnova-multicloud-auto-deploy-staging-function-source`（5 MB）に置き換え済み。
`multicloud-auto-deploy-tfstate`	AWS Terraform state バケット候補として作成されたが未使用。空。

バケット名	推定履歴
`multicloud-auto-deploy-tfstate-gcp`	`ashnova-staging-*` 2つのバケットの Terraform state を保持。コードベースに `.tf` ファイルなし。4つをセットで削除。

おまけ: 壊れた **Cloud Function**（関連リソース）

リソース	状態	詳細
`mcad-staging-api` (Cloud Function v2)	**FAILED**	`Cloud Run service not found` エラー。Cloud Run サービスは削除されたが Function 定義が残っている。Pulumi/現行コードに参照なし。削除可能。

削除コマンド

```
        # 4つのバケットを削除（内容含む）— tfstate-gcp は最後に削除
        gcloud storage rm --recursive gs://ashnova-staging-frontend           --project=ashnova
        gcloud storage rm --recursive gs://ashnova-staging-function-source    --project=ashnova
        gcloud storage rm --recursive gs://multicloud-auto-deploy-tfstate     --project=ashnova
        gcloud storage rm --recursive gs://multicloud-auto-deploy-tfstate-gcp --project=ashnova

        # 壊れた Cloud Function も削除
```

```
gcloud functions delete mcad-staging-api ¥
  --region=asia-northeast1 --project=ashnova --v2 --quiet
```

> ⚠ `multicloud-auto-deploy-tfstate-gcp` は `ashnova-staging-frontend` と `ashnova-staging-function-source` の Terraform state を含みます。4つのバケットをセットで削除してください。

次のセクション

→ 07 — ランブック

07 — ランブック

> Part III — 運用 | 親: AI_AGENT_GUIDE.md よくある作業とインシデント対応の手順書

[RB-01] Lambda 関数を手動更新する

```
# 1. アプリコードのみをパッケージング（ZIP）
cd /workspaces/multicloud-auto-deploy/services/api
rm -rf .build && mkdir -p .build/package
cp -r app .build/package/
cp index.py .build/package/
cd .build/package && zip -r ../../lambda.zip . && cd ../..

# 2. Lambda を更新
aws lambda update-function-code ¥
  --function-name multicloud-auto-deploy-staging-api ¥
```

```
          --zip-file fileb://lambda.zip ¥
          --region ap-northeast-1

        # 3. 更新を確認
        aws lambda invoke ¥
          --function-name multicloud-auto-deploy-staging-api ¥
          --payload
'{"version":"2.0","routeKey":"$default","rawPath":"/
health","requestContext":{"http":{"method":"GET","path":"/
health"}}}' ¥
          --cli-binary-format raw-in-base64-out ¥
          /tmp/response.json && cat /tmp/response.json | python3 -m
json.tool
```

[RB-02] Lambda ログを確認する

```
        # リアルタイム追従
        aws logs tail /aws/lambda/multicloud-auto-deploy-staging-
api --follow --region ap-northeast-1

        # 直近10分のエラーのみ
        aws logs tail /aws/lambda/multicloud-auto-deploy-staging-
api ¥
          --since 10m --filter-pattern "ERROR" --region ap-
northeast-1
```

[RB-03] React フロントエンドを手動デプロイする（AWS）

```
        cd /workspaces/multicloud-auto-deploy/services/
frontend_react
```

```
        npm ci

        # API URL を指定してビルド; base="/sns/" は vite.config.ts で設定済み
        VITE_API_URL=https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com npm run build

        # S3 にアップロード — sns/ プレフィックスは必須
        aws s3 sync dist/ s3://multicloud-auto-deploy-staging-frontend/sns/ ¥
          --delete --region ap-northeast-1

        # CloudFront キャッシュを無効化
        aws cloudfront create-invalidation ¥
          --distribution-id E1TBH4R432SZBZ ¥
          --paths "/*"
```

[RB-04] ランディングページを手動デプロイする（AWS）

```
        # static-site/ をバケットルートへアップロード
        aws s3 sync /workspaces/multicloud-auto-deploy/static-site/ ¥
          s3://multicloud-auto-deploy-staging-frontend/ ¥
          --delete --region ap-northeast-1

        aws cloudfront create-invalidation ¥
          --distribution-id E1TBH4R432SZBZ ¥
          --paths "/*"
```

[RB-05] Azure Functions を手動デプロイする

```
        cd /workspaces/multicloud-auto-deploy/services/api

        # デプロイパッケージを作成
        pip install -r requirements-azure.txt -t .deploy-azure/
        cp -r app .deploy-azure/
        cp function_app.py host.json local.settings.json .deploy-
azure/

        # Zip 化
        cd .deploy-azure && zip -r ../function-app.zip . && cd ..

        # Azure CLI でデプロイ
        az functionapp deployment source config-zip ¥
          --resource-group multicloud-auto-deploy-staging-rg ¥
          --name multicloud-auto-deploy-staging-func ¥
          --src function-app.zip
```

[RB-06] GCP Cloud Run を手動デプロイする

```
        cd /workspaces/multicloud-auto-deploy/services/api

        # Cloud Run ビルド用にソースを Zip 化
        zip -r gcp-cloudrun-source.zip app/ function.py
requirements.txt requirements-gcp.txt Dockerfile

        # gcloud でソースからデプロイ
        gcloud run deploy multicloud-auto-deploy-staging-api ¥
          --source . ¥
```

```
          --region asia-northeast1 ¥
          --project ashnova ¥
          --platform managed ¥
          --allow-unauthenticated ¥
          --set-env-vars
"CLOUD_PROVIDER=gcp,AUTH_DISABLED=false,GCP_POSTS_COLLECTION=posts"
```

[RB-07] Pulumi スタックを再デプロイする

```
        # AWS
        cd /workspaces/multicloud-auto-deploy/infrastructure/pulumi/
aws
        pulumi login
        pulumi stack select staging
        pulumi up --yes

        # Azure
        cd /workspaces/multicloud-auto-deploy/infrastructure/pulumi/
azure
        pulumi stack select staging
        pulumi up --yes

        # GCP
        cd /workspaces/multicloud-auto-deploy/infrastructure/pulumi/
gcp
        pulumi stack select staging
        pulumi up --yes
```

[RB-08] API エンドポイントテストを実行する

```
        # 全クラウド E2E（curl ベース）
        ./scripts/test-e2e.sh

        # 単一クラウド
        ./scripts/test-api.sh -e https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com

        # pytest（モックテスト）
        cd services/api
        source .venv/bin/activate
        pytest tests/test_backends_integration.py -v

        # pytest（ライブ API）
        API_BASE_URL=https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com ¥
          pytest tests/test_api_endpoints.py -v
```

[RB-09] DynamoDB PostIdIndex GSI を確認 / 作成する

```
        # GSI が存在するか確認
        aws dynamodb describe-table ¥
          --table-name multicloud-auto-deploy-staging-posts ¥
          --region ap-northeast-1 ¥
          --query 'Table.GlobalSecondaryIndexes[*].IndexName'

        # GSI がなければ作成
        aws dynamodb update-table ¥
```

```
          --table-name multicloud-auto-deploy-staging-posts ¥
          --region ap-northeast-1 ¥
          --attribute-definitions
AttributeName=postId,AttributeType=S ¥
          --global-secondary-index-updates '[{"Create":
{"IndexName":"PostIdIndex","KeySchema":
[{"AttributeName":"postId","KeyType":"HASH"}],"Projection":
{"ProjectionType":"ALL"}}}]'
```

[RB-11] API Lambda を再ビルドしてホットフィックスデプロイする（CI なし）

CI を待たずに API Lambda へ緊急修正をデプロイしたい場合に使用します。

```
        cd /workspaces/multicloud-auto-deploy/services/api

        # 1. 最新ソースで .build/package を更新
        cp -r app index.py .build/package/

        # 2. lambda.zip を再作成
        cd .build/package
        zip -r ../../lambda.zip . -x "*.pyc" -x "__pycache__/*" > /
dev/null
        cd ../..

        # 3. ガード: 進行中の更新完了を先に待機
        aws lambda wait function-updated ¥
          --function-name multicloud-auto-deploy-staging-api ¥
          --region ap-northeast-1

        # 4. デプロイ
```

```
          aws lambda update-function-code ¥
            --function-name multicloud-auto-deploy-staging-api ¥
            --region ap-northeast-1 ¥
            --zip-file fileb://lambda.zip ¥
            --output text --query 'LastUpdateStatus'

          # 5. デプロイ完了待機
          aws lambda wait function-updated ¥
            --function-name multicloud-auto-deploy-staging-api ¥
            --region ap-northeast-1 && echo "Ready"
```

[RB-12] AWS simple-sns E2E テストを実行する

```
          # 公開エンドポイントのみ（認証不要）
          ./scripts/test-sns-aws.sh

          # 完全な認証付きテスト ー 先にトークンを取得
          TOKEN=$(aws cognito-idp initiate-auth ¥
            --auth-flow USER_PASSWORD_AUTH ¥
            --client-id 1k41lqkds4oah55ns8iod30dv2 ¥
            --auth-parameters USERNAME=<your-email>,PASSWORD=<your-
password> ¥
            --region ap-northeast-1 ¥
            --query 'AuthenticationResult.AccessToken' --output text)

          ./scripts/test-sns-aws.sh --token "$TOKEN" --verbose
```

参照: docs/AWS_SNS_FIX_REPORT.md

[RB-10] ローカル開発環境を起動する

> ホストマシン: ARM（Apple Silicon M シリーズ Mac） Dev Container 内から実行してください。

```
cd /workspaces/multicloud-auto-deploy

# インフラ起動
docker compose up -d

# 起動確認
docker compose ps
curl http://localhost:8000/health

# ログ確認
docker compose logs -f api
```

環境変数の上書き（デバッグ用）

特定バックエンドを向ける場合は `.env` を作成します。

```
# services/api/.env または docker-compose の env_file で指定
CLOUD_PROVIDER=local
AUTH_DISABLED=true
API_BASE_URL=http://localhost:8000
```

ARM に関する注意

- ローカルの docker compose は **ARM** でネイティブ動作（問題なし）

- Lambda 向けパッケージのビルドには `--platform linux/amd64` が必要 → 通常は GitHub Actions（ubuntu-latest = x86_64）が処理
- GCP Cloud Function の ZIP ビルドも同様（CI で自動処理）

[RB-13] GCP Cloud Function を再ビルドしてホットフィックスデプロイする（linux/amd64）

dev container は `aarch64`、Cloud Functions は `linux/amd64` で動作します。必ず Docker を使用してください。

```
        # 1. linux/amd64 パッケージをビルド
        mkdir -p /tmp/deploy_gcp/.deployment
        docker run --rm --platform linux/amd64 ¥
          -v /tmp/deploy_gcp:/out ¥
          python:3.12-slim ¥
          bash -c "pip install --no-cache-dir --target /
out/.deployment ¥
                   -r /workspaces/multicloud-auto-deploy/services/
api/requirements-gcp.txt -q"

        # 2. アプリコードをコピー
        cp -r /workspaces/multicloud-auto-deploy/services/api/app /
tmp/deploy_gcp/.deployment/
        # Cloud Build は --entry-point 指定時でも main.py を要求
        cp /workspaces/multicloud-auto-deploy/services/api/
function.py /tmp/deploy_gcp/.deployment/main.py
        cp /workspaces/multicloud-auto-deploy/services/api/
function.py /tmp/deploy_gcp/.deployment/function.py

        # 3. ZIP を作成（__pycache__ を除外）
        cd /tmp/deploy_gcp/.deployment
        find . -name "__pycache__" -path "*/app/*" -exec rm -rf {} +
```

```
2>/dev/null || true
        zip -r9q /tmp/deploy_gcp/function-source.zip .
        cd /workspaces/multicloud-auto-deploy

        # 4. GCS へアップロード（再開時は古い tracker files を先に削除）
        rm -f ~/.config/gcloud/surface_data/storage/tracker_files/*
        gcloud storage cp /tmp/deploy_gcp/function-source.zip ¥
          gs://ashnova-multicloud-auto-deploy-staging-function-
source/function-source.zip

        # 5. デプロイ
        gcloud functions deploy multicloud-auto-deploy-staging-api ¥
          --gen2 --region=asia-northeast1 --runtime=python312 ¥
          --source=gs://ashnova-multicloud-auto-deploy-staging-
function-source/function-source.zip ¥
          --entry-point=handler --project=ashnova --quiet
```

重要ルール:

- ZIP 内に `main.py` が必須。`--entry-point` が別関数でも、Cloud Build はこれが無いソースを拒否します。
- パッケージ前に必ず構文確認: `python3 -m py_compile services/api/app/backends/local_backend.py && echo OK`
- `gcloud run services update --update-env-vars` は URL 値に使えません（`:` でパースエラー）。`--env-vars-file` を使ってください。

[RB-14] Azure Front Door 502 を修正する（Dynamic Consumption → FC1 FlexConsumption）

Dynamic Consumption（Y1）の Function App は定期的に再起動されます。AFD Standard は TCP 切断を検知できず、接続プールに古い接続が残って 502 を即時返すことがあります。 対策は、固定1インスタンスの **FC1 FlexConsumption** へ移行することです。

```
        RG="multicloud-auto-deploy-production-rg"
        FD="multicloud-auto-deploy-production-fd"
        EP="mcad-production-diev0w"
        OG="multicloud-auto-deploy-production-frontend-web-origin-group"
        ORIGIN="multicloud-auto-deploy-production-frontend-web-origin"

        # 1. FC1 FlexConsumption Function App を作成
        az functionapp create ¥
          --name multicloud-auto-deploy-production-frontend-web-v2 ¥
          --resource-group "$RG" ¥
          --flexconsumption-location japaneast ¥
          --runtime python --runtime-version 3.12 ¥
          --storage-account mcadfuncdiev0w

        # 2. インスタンス数を1に固定（再起動回避）
        az functionapp scale config set ¥
          --name multicloud-auto-deploy-production-frontend-web-v2 ¥
          --resource-group "$RG" ¥
          --maximum-instance-count 1
        az functionapp scale config always-ready set ¥
          --name multicloud-auto-deploy-production-frontend-web-v2 ¥
          --resource-group "$RG" ¥
```

```
          --settings "http=1"

        # 3. x86_64 ZIP をデプロイ（事前に Docker --platform linux/amd64 でビルド）
        az functionapp deployment source config-zip ¥
          --name multicloud-auto-deploy-production-frontend-web-v2 ¥
          --resource-group "$RG" ¥
          --src services/frontend_web/frontend-web-prod.zip

        # 4. AFD origin を新しい Function App へ切り替え
        az afd origin update ¥
          --resource-group "$RG" ¥
          --profile-name "$FD" ¥
          --origin-group-name "$OG" ¥
          --origin-name "$ORIGIN" ¥
          --host-name multicloud-auto-deploy-production-frontend-web-v2.azurewebsites.net ¥
          --origin-host-header multicloud-auto-deploy-production-frontend-web-v2.azurewebsites.net

        # 5. 古い Y1 アプリを停止（AFD エッジ反映に約5分）
        az functionapp stop ¥
          --name multicloud-auto-deploy-production-frontend-web ¥
          --resource-group "$RG"
```

重要メモ:

- `--consumption-plan-location` は Y1 Dynamic を作成するため誤り。FC1 には必ず `--flexconsumption-location` を使います。
- `az functionapp update --plan` は Linux→Linux のプラン移行をサポートしません。新規アプリ作成が必要です。
- AFD origin 更新は全エッジ反映まで最大5分程度かかります。
- `SCM_DO_BUILD_DURING_DEPLOYMENT` は Flex Consumption で `InvalidAppSettingsException` を起こすため設定しないでください。

[RB-15] AWS CloudFront HTTPS 証明書エラーを修正する（ERR_CERT_COMMON_NAME_INVALID）

`customDomain` / `acmCertificateArn` を設定せずに `pulumi up --stack production` を実行すると、 CloudFront が `CloudFrontDefaultCertificate: true` に戻り、カスタムドメインの HTTPS が壊れます。

```
        # 1. 現在の distribution 設定と ETag を取得
        aws cloudfront get-distribution-config --id E214XONKTXJEJD
> /tmp/cf-config.json
        # 応答内 ETag を控える（例: E13V1IB3VIYZZH）

        # 2. JSON を修正（Python ワンライナー）
        python3 - <<'EOF'
        import json
        with open('/tmp/cf-config.json') as f:
            data = json.load(f)
        cfg = data['DistributionConfig']
        cfg['Aliases'] = {'Quantity': 1, 'Items':
['www.aws.ashnova.jp']}
        cfg['ViewerCertificate'] = {
            'ACMCertificateArn': 'arn:aws:acm:us-
east-1:278280499340:certificate/
914b86b1-4c10-4354-91cf-19c4460dcde5',
            'SSLSupportMethod': 'sni-only',
            'MinimumProtocolVersion': 'TLSv1.2_2021',
            'CertificateSource': 'acm'
        }
        with open('/tmp/cf-config-updated.json', 'w') as f:
            json.dump(cfg, f, indent=2)
        print('Done')
```

```
        EOF

        # 3. 更新を適用（<ETAG> は手順1の値に置換）
        aws cloudfront update-distribution ¥
          --id E214XONKTXJEJD ¥
          --distribution-config file:///tmp/cf-config-updated.json ¥
          --if-match <ETAG>

        # 4. 反映待ちして検証
        aws cloudfront wait distribution-deployed --id
E214XONKTXJEJD
        curl -sI https://www.aws.ashnova.jp | head -3
```

再発防止 — `pulumi up --stack production` の前に必ず設定:

```
        cd infrastructure/pulumi/aws
        pulumi config set customDomain www.aws.ashnova.jp --stack
production
        pulumi config set acmCertificateArn ¥
          arn:aws:acm:us-east-1:278280499340:certificate/
914b86b1-4c10-4354-91cf-19c4460dcde5 ¥
          --stack production
```

ACM 証明書（production）: ARN `914b86b1` — ドメイン `www.aws.ashnova.jp` 、有効期限 2027-03-12。

[RB-16] Azure FC1 InaccessibleStorageException を修正する（デプロイ用ストレージアカウント削除時）

発生条件: `az functionapp deployment source config-zip` が即時失敗し、 `InaccessibleStorageException: BlobUploadFailedException: Name or service not known (xxxxx.blob.core.windows.net:443)` が出る。

根本原因: FC1 Function App に紐づく `functionAppConfig.deployment.storage` のストレージアカウントが削除されている。 Kudu の `StorageAccessibleCheck` が通らないため、すべての zip deploy がブロックされる。

```
        RESOURCE_GROUP="multicloud-auto-deploy-staging"  # or -production
        FUNCTION_APP="multicloud-auto-deploy-staging-func"
        LOCATION="japaneast"

        # 1. Function App に設定されている storage endpoint を確認
        DEPLOY_STORAGE_URL=$(az rest --method GET ¥
          --url "https://management.azure.com/subscriptions/$(az account show --query id -o tsv)/resourceGroups/${RESOURCE_GROUP}/providers/Microsoft.Web/sites/${FUNCTION_APP}?api-version=2023-12-01" ¥
          | jq -r '.properties.functionAppConfig.deployment.storage.value // empty')
        echo "Deployment storage URL: $DEPLOY_STORAGE_URL”

        # 2. URL からストレージアカウント名を抽出（形式: https://<name>.blob.core.windows.net/...）
```

```
        STORAGE_ACCT=$(echo "$DEPLOY_STORAGE_URL" | sed 's|
https://||; s|\.blob\.core.*||')
        echo "Storage account name: $STORAGE_ACCT"

        # 3. アカウント存在確認
        if ! az storage account show --name "$STORAGE_ACCT" --
resource-group "$RESOURCE_GROUP" &>/dev/null; then
          echo "Storage account missing — recreating..."
          az storage account create ¥
            --name "$STORAGE_ACCT" ¥
            --resource-group "$RESOURCE_GROUP" ¥
            --location "$LOCATION" ¥
            --sku Standard_LRS ¥
            --kind StorageV2 ¥
            --allow-blob-public-access false
        fi

        # 4. connection string を取得し、Kudu が使う app setting を更新
        CONN_STR=$(az storage account show-connection-string ¥
          --name "$STORAGE_ACCT" ¥
          --resource-group "$RESOURCE_GROUP" ¥
          --query connectionString --output tsv)

        az functionapp config appsettings set ¥
          --name "$FUNCTION_APP" ¥
          --resource-group "$RESOURCE_GROUP" ¥
          --settings "AzureWebJobsStorage=$CONN_STR"

        # 5. さらに、デプロイ後 404 を防ぐため古い WEBSITE_RUN_FROM_PACKAGE
を削除
        az functionapp config appsettings delete ¥
          --name "$FUNCTION_APP" ¥
          --resource-group "$RESOURCE_GROUP" ¥
          --setting-names WEBSITE_RUN_FROM_PACKAGE 2>/dev/null ||
true

        # 6. 設定反映のため Function App を再起動
```

```
        az functionapp restart --name "$FUNCTION_APP" --resource-group "$RESOURCE_GROUP"
        sleep 30

        # 7. zip deploy を再実行
        az functionapp deployment source config-zip ¥
          --name "$FUNCTION_APP" ¥
          --resource-group "$RESOURCE_GROUP" ¥
          --src <path/to/package.zip>
```

検証:

```
        curl -s "https://<func-hostname>/api/HttpTrigger/health" | jq .
        # 期待値: {"status":"ok","provider":"azure","version":"X.Y.Z"}
```

予防策:

- `deploy-azure.yml` には、毎回このロジックを実行する「Ensure deployment storage account exists」ステップが入っています。
- FC1 Function App と同一リソースグループ内のストレージアカウントを削除する前に、依存関係を必ず確認してください。

[RB-17] マルチクラウドのコスト監視

ターミナルでフルレポートを出す

```
        cd /workspaces/multicloud-auto-deploy

        # 3クラウド + GitHub の当月コストを表示（デフォルト: 過去3ヶ月）
```

```
python3 scripts/cost_report.py

# オプション
python3 scripts/cost_report.py --months 6       # 過去6ヶ月
python3 scripts/cost_report.py --json           # JSON 出力
（jq と組み合わせ可）
python3 scripts/cost_report.py --aws-only       # AWS のみ
python3 scripts/cost_report.py --azure-only     # Azure のみ
```

出力例（2026-02）

```
 ★ Multi-Cloud Cost Report
────────────────────────────────────────────────────────────────────────────
Provider                Period                Cost  Note
────────────────────────────────────────────────────────────────────────────
  AWS                   2026-02    ¥18,040 (JPY)  $115.41 × ¥156
  Azure                 2026-02   ¥6,970 (JPY)   JPY 建て請求
  GCP                   2026-02         N/A       BQ billing export 参照
  GitHub Actions        2026-02         N/A       1298 runs / cache 1.49
────────────────────────────────────────────────────────────────────────────
  TOTAL USD                                 0.0000
  TOTAL JPY                                ¥25,010  (円建ての請求)
```

認証情報の設定

`scripts/mac-widget/.env` が自動で参照されます（無ければ `scripts/mac-widget/cost-monitor.env.sample` をコピー）。

```
cp scripts/mac-widget/cost-monitor.env.sample scripts/mac-widget/.env
# 必要な値を記入
```

環境変数	用途	取得方法
`AZURE_SUBSCRIPTION_ID`	Azure Cost Management API	`az account show --query id`
`GCP_BILLING_ACCOUNT`	GCP Billing	`gcloud billing accounts list`
`GCP_PROJECT_ID`	GCP プロジェクト	`gcloud config get-value project`
`GITHUB_TOKEN`	GitHub Actions 使用量	`gh auth token`
`GH_ORG` or `GH_REPO`	GitHub 対象	Org 名 or `owner/repo`

AWS は `aws configure` 済みのデフォルトプロファイルを使用します（追加設定不要）。

為替レート

AWS は USD 建てのため、open.er-api.com からリアルタイムの USD/JPY レートを取得して円換算します。API 障害時は ¥150 固定 にフォールバックします。

macOS メニューバーウィジェット（xbar）

xbar（https://xbarapp.com）を使うと、メニューバーに常時表示できます。

```
# 1. xbar をインストール
brew install --cask xbar
```

```
# 2. インストール（シンボリックリンク作成 + .env 初期化）
bash scripts/mac-widget/install.sh

# 3. .env に認証情報を記入
open -e scripts/mac-widget/.env
```

メニューバー表示例:

```
☘ ￥25,010
──────────────────────────
☒AWS    ¥18,040  ($115.41 × ¥156)
☒Azure  ¥6,970
☒GCP    N/A  [請求先アカウント]
● GitHub   N/A  1298 runs / cache 1.49 GB
──────────────────────────
TOTAL  ¥25,010
```

- 更新間隔: 1時間ごと（ファイル名 `cost-monitor.1h.py` の `.1h.` で制御）
- 金額の色: `green` < ¥750（$5相当）/ `orange` < ¥4,500（$30相当）/ `red` それ以上
- クリックで各コンソールへ直接遷移

ファイル構成

ファイル	役割
`scripts/cost_report.py`	ターミナル用の月次レポート（`--months N` / `--json` 対応）
`scripts/mac-widget/cost-monitor.1h.py`	xbar プラグイン本体

ファイル	役割
`scripts/mac-widget/cost-monitor.env.sample`	認証情報テンプレート
`scripts/mac-widget/install.sh`	xbar セットアップスクリプト
`scripts/mac-widget/.env`	認証情報（**git** 管理外）

次のセクション

→ 08 — Security

08 — セキュリティ

> Part III — 運用 | 親: AI_AGENT_GUIDE.md

セキュリティ設定現況

> 最終更新: 2026-02-24 (Defender for Cloud セキュアスコア分析・新規タスク追加)

Feature	AWS	Azure	GCP	Notes
HTTPS enforced	☒	☒	☒ Pulumi済（要 pulumi up)	GCP: HTTP→HTTPS 用 URLMap を分離。301 redirect のみ
WAF		☒	☒Cloud Armor	

Feature	AWS	Azure	GCP	Notes
	☒ WebACL (CloudFront)			Azure: Front Door S では WAF Policy Premium SKU or WA
Rate limiting	☒	☒	☒ 100req/min/IP	
SQLi / XSS protection	☒	☒	☒	
Data encryption (at rest)	☒SSE-S3	☒ Azure SSE	☒ Google-managed	
Versioning	☒	☒	☒	
Access logs (CDN)	☒ CloudFront	☒ Front Door → Log Analytics	☒Cloud CDN	Azure: DiagnosticS (2026-02-24, 要 pulu
Security headers	☒ CloudFront RHP	☒	☒	HSTS/CSP/X-Frame/ Referrer/XSS (A 2026-02-23)
Soft delete / retention	☒	☒7 days	☒	
CORS config	☒ 実ドメイン (Pulumi済)	☒実ドメイン (Pulumi済)	☒ 実ドメイン (Pulumi済)	production `*` → 実 込み完了 (2026-02-2 up)
Audit logging	☒ CloudTrail (Pulumi済)			

Feature	AWS	Azure	GCP	Notes
		☒ Log Analytics (Pulumi済)	☒ IAMAuditConfig (Pulumi済)	全項目 Pulumi コード クラウドで `pulumi` 効化される
Managed Identity	N/A	☒ 未設定 (staging/ production)	N/A (Cloud Run SA)	Azure Function App り当てマネージドIDが (Defender 指摘 2026
Key Vault 消去保護	N/A	☒未設定	N/A	`enable_purge_prote` を Pulumi に追 (Defender 指摘 2026
Key Vault 診断ログ	N/A	☒未設定	N/A	DiagnosticSetting Analytics Workspac 未設定 (Defenc 2026-02-24)

認証設定

AWS — Amazon Cognito

```
Auto-created by Pulumi:
  - Cognito User Pool
  - User Pool Client (allowed_oauth_flows=["code"] のみ ー implicit 廃止)
  - User Pool Domain

Lambda environment variables:
  AUTH_PROVIDER=cognito
  COGNITO_USER_POOL_ID=<Pulumi output>
```

```
  COGNITO_CLIENT_ID=<Pulumi output>
  AWS_REGION=ap-northeast-1
```

OAuth フロー: **PKCE (Proof Key for Code Exchange)** — 2026-02-23 実装

項目	内容
フロー	Authorization Code + PKCE (S256)
`response_type`	`code` (implicit `token` は廃止)
code_verifier	256-bit ランダム、sessionStorage に保存
code_challenge	SHA-256(verifier) → Base64URL
トークン交換	ブラウザから `POST /oauth2/token` (code_verifier 付き)
利点	URLフラグメントへのトークン漏洩が完全に排除される

> 注意: `implicit` フローは Cognito UserPoolClient の `allowed_oauth_flows` から削除済み。 古い implicit フロー URL (`response_type=token`) でアクセスすると Cognito がエラーを返す。

Azure — Azure AD

```
Auto-created by Pulumi:
  - Azure AD Application (pulumi-azuread)
  - Service Principal
  - OAuth2 Permission Scope (API.Access)
```

```
  - Redirect URIs

Functions environment variables:
  AUTH_PROVIDER=azure
  AZURE_TENANT_ID=<Pulumi output "azure_ad_tenant_id">
  AZURE_CLIENT_ID=<Pulumi output "azure_ad_client_id">
```

GCP — Firebase Auth

```
Auto-created by Pulumi:
  - Firebase Auth project configuration
  - Firebase Auth Google Sign-In provider enabled

Cloud Run (API) environment variables:
  AUTH_PROVIDER=firebase
  GCP_PROJECT_ID=ashnova
  GCP_SERVICE_ACCOUNT=899621454670-compute@developer.gserviceaccount.co
  (uses impersonated_credentials to generate GCS presigned URLs via IAM

Cloud Run (frontend-web) environment variables:
  AUTH_PROVIDER=firebase
  AUTH_DISABLED=false
  FIREBASE_API_KEY=<GitHub Secret: FIREBASE_API_KEY>
  FIREBASE_AUTH_DOMAIN=<GitHub Secret: FIREBASE_AUTH_DOMAIN>
  FIREBASE_PROJECT_ID=ashnova
  FIREBASE_APP_ID=<GitHub Secret: FIREBASE_APP_ID>

Firebase authorized domain:
  multicloud-auto-deploy-staging-frontend-web-son5b3ml7a-an.a.run.app

Token refresh:
  `onIdTokenChanged` in home.html auto-refreshes the token (and re-issu
```

IAM 最小権限ポリシー

AWS Lambda 実行ロール

```
        {
          "Version": ”2012-10-17”,
          "Statement": [
            { "Effect": "Allow", "Action": ["logs:*"], "Resource":
"arn:aws:logs:*" },
            {
              "Effect": "Allow",
              "Action": [
                "dynamodb:GetItem",
                "dynamodb:PutItem",
                "dynamodb:UpdateItem",
                "dynamodb:DeleteItem",
                "dynamodb:Scan",
                "dynamodb:Query"
              ],
              "Resource": "arn:aws:dynamodb:*:*:table/simple-sns-
messages*"
            },
            {
              "Effect": "Allow",
              "Action": ["s3:GetObject", "s3:PutObject",
"s3:DeleteObject"],
              "Resource": "arn:aws:s3:::*uploads*"
            },
            {
              "Effect": "Allow",
              "Action": ["secretsmanager:GetSecretValue"],
              "Resource": ”*”
            }
```

```
            ]
        }
```

Azure サービスプリンシパル (pulumi-deploy)

- Role: `Contributor` (subscription scope)
- Storage Blob Data Contributor: `mcadweb*` Storage Account

GCP — サービスアカウント (github-actions-deploy)

- Role: `roles/editor`
- Additional: `roles/storage.objectAdmin` (uploads bucket, for presigned URL signing)
- Additional: `roles/iam.serviceAccountTokenCreator` (to impersonate Compute Engine SA for `signBlob` API)

> **Note**: GCS uploads bucket (`ashnova-multicloud-auto-deploy-staging-uploads`) is intentionally public (`allUsers:objectViewer`) to allow direct image display in browsers. Do NOT apply this to the frontend bucket.

シークレット管理

Cloud	Service	Primary use
AWS	Secrets Manager	DB credentials, API keys
Azure	Key Vault	Connection strings, certificates

Cloud	Service	Primary use
GCP	Secret Manager	Service account keys
CI/CD	GitHub Secrets	Cloud provider credentials

未構築のセキュリティ課題（優先度順）

1. **Apply security Pulumi changes** (high priority — next action) 以下の変更が Pulumi コードに実装済みだが、まだ `pulumi up` で本番に適用されていない:

   - CORS `*` → 実ドメイン絞り込み (全3クラウド production)
   - AWS CloudTrail 有効化
   - GCP HTTP→HTTPS リダイレクト分離
   - GCP Cloud Audit Logs (`IAMAuditConfig`)
   - Azure Log Analytics Workspace + Front Door DiagnosticSetting

   ```
   cd infrastructure/pulumi/aws   && pulumi up --stack production
   cd infrastructure/pulumi/gcp   && pulumi up --stack staging && pulumi up --stack production
   cd infrastructure/pulumi/azure && pulumi up --stack staging && pulumi up --stack production
   ```

2. **Azure WAF** (high priority) Front Door Standard SKU では組み込み WAF が使えない。 対策A: Premium SKU にアップグレード (約 $35/月の差額)。 対策B: Standard SKU 向け独立 WAF Policy を作成して Front Door に紐付け。

3. **Add AWS WAF managed rules** (medium priority) WAF WebACL は production に存在するが `AWSManagedRulesCommonRuleSet` 等のルールが未チューニング。 `AWSManagedRulesAmazonIpReputationList` (悪意のある IP リスト) を追加することを推奨。

4. **Azure security headers** (medium priority) AWS CloudFront にのみ設定済み。Azure Front Door の RuleSet に HSTS/CSP(`upgrade-insecure-requests`)/X-Frame-Options 等のレスポンスヘッダーアクションを追加する。

5. **GCP security headers** (medium priority) Cloud Run の FastAPI アプリ層 (ミドルウェア) にセキュリティヘッダーを追加する。

6. **Azure Key Vault network ACLs** (medium priority — Defender 指摘, Pulumi対応可) 現在 `default_action="Allow"` (全許可)。 `network_acls.default_action="Deny"` に変更し `bypass="AzureServices"` を維持することで、Managed Identity 経由のアクセスは継続可能。 前提条件: #8 (Function App Managed Identity 有効化) を先に完了すること。

```
network_acls=azure.keyvault.NetworkRuleSetArgs(
    bypass="AzureServices",
    default_action="Deny",  # Allow → Deny に変更
),
```

7. **GCP SSL certificate placeholder** (low priority — 既に実ドメイン設定済みの可能性あり) `gcp/Pulumi.staging.yaml` の `customDomain` が `staging.gcp.ashnova.jp` に設定済み。Managed SSL Certificate のドメインハッシュを確認し、 `example.com` が残っていれば修正。

8. **Function App Managed Identity** 有効化 (high priority — Defender 指摘, 即時対応可) staging / production 両環境の Function App にシステム割り当てマネージドIDが未設定。 Azure CLI で即時有効化可能:

```
az functionapp identity assign ¥
  --name multicloud-auto-deploy-staging-func ¥
  --resource-group multicloud-auto-deploy-staging-rg
az functionapp identity assign ¥
  --name multicloud-auto-deploy-production-func ¥
  --resource-group multicloud-auto-deploy-production-rg
```

   有効化後は #6 (Key Vault ファイアウォール) と Storage 共有キーアクセス削除の前提となる。

9. **Azure Key Vault** 消去保護 (**purge protection**) (medium priority — Pulumi対応可) `enable_purge_protection=True` を Pulumi コードに追加して `pulumi up` するだけ。 一度有効化すると無効に戻せないが、既存の `enable_soft_delete=True` 環境では安全。

```
enable_soft_delete=True,
enable_purge_protection=True,  # 追加
soft_delete_retention_in_days=7,
```

10. **Azure Key Vault** 診断ログ (low priority — Pulumi対応可) Log Analytics Workspace は既に Pulumi 実装済み。DiagnosticSetting を追加するだけ。

```
key_vault_diagnostics =
azure.insights.DiagnosticSetting(
    "key-vault-diagnostics",
     resource_uri=key_vault.id,
```

```
            workspace_id=log_analytics_workspace.id,
            logs=[azure.insights.LogSettingsArgs(
                category_group="allLogs",
                enabled=True,

retention_policy=azure.insights.RetentionPolicyArgs(days=30,
enabled=True),
            )],
        )
```

11. **Azure** セキュリティ連絡先 / 重要度高アラート通知 (low priority — CLI即時対応可) サブスクリプションにセキュリティ連絡先メールが未設定。Azure CLI で即時対応可能:

```
        az security contact create ¥
          --email "your@email.com" ¥
          --alert-notifications On ¥
          --alerts-to-admins On
```

12. サブスクリプション所有者の複数設定 (high priority — Portal/CLI対応可) 現在の所有者 (`sat0sh1kawada`) が1名のみ。Microsoft の推奨は最低2名。 信頼できる2人目を Owner ロールに追加することで対応可能:

```
        az role assignment create ¥
          --assignee "second-user@email.com" ¥
          --role "Owner" ¥
          --scope "/subscriptions/29031d24-
d41a-4f97-8362-46b40129a7e8"
```

Defender for Cloud — 対応困難な項目（現行アーキテクチャの制約）

> 下記はいずれも VNet 構築・有料プラン追加・大規模アーキテクチャ変更を伴うため、現時点では対応見送り推奨。

項目	重大度	対応困難な理由
Cosmos DB ファイアウォール規則	Medium	Consumption Plan は固定IP なし。特定IP での絞り込み不可
Cosmos DB AAD 唯一認証	Medium	接続文字列→MSI+RBAC への API 全書き直しが必要
Cosmos DB / Key Vault プライベートリンク	Medium	VNet 新規構築が前提
Cosmos DB パブリックネットワークアクセス無効化	Medium	VNet 統合なしでは API が完全停止
Storage プライベートリンク / VNet 規則 (mcadweb/mcadfunc)	Medium	静的ウェブサイト用途では不可。Front Door Premium SKU 移行が前提
Storage パブリックアクセス禁止 (mcadweb)	Medium	静的ウェブサイトホスティングには公開アクセスが必須
Microsoft Defender CSPM / App Service / Key Vault / Resource Manager / Storage	High	有料プラン。追加費用が発生するため要予算判断

セキュリティヘッダ（AWS CloudFront — 確認済み 2026-02-23）

> Pulumi リソース: `aws.cloudfront.ResponseHeadersPolicy` (`multicloud-auto-deploy-{stack}-security-headers`)
> `default_cache_behavior` + `/sns*` ordered_cache_behavior 両方に適用。

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: upgrade-insecure-requests
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
```

`upgrade-insecure-requests` の効果:

- ブラウザが `http://` サブリソース (画像・スクリプト・CSS) を自動的に `https://` にアップグレード
- Mixed Content による「保護されていない通信」警告を防止
- HSTS と合わせて二重の HTTPS 強制を実現

検証コマンド:

```
        curl -sI https://staging.aws.ashnova.jp/ | grep -iE 'strict-transport|content-security|x-content|x-frame|referrer'
        # 期待値:
        # strict-transport-security: max-age=31536000; includeSubDomains
        # content-security-policy: upgrade-insecure-requests
```

```
        # x-content-type-options: nosniff
        # x-frame-options: SAMEORIGIN
        # referrer-policy: strict-origin-when-cross-origin
```

S3 セキュリティ (AWS — フロントエンドバケット)

設定済み (2026-02-23):

```
        # infrastructure/pulumi/aws/__main__.py
        BucketPublicAccessBlock(
            block_public_acls=True,
            ignore_public_acls=True,
            block_public_policy=True,
            restrict_public_buckets=True,
        )
```

項目	状態
S3 HTTP ウェブサイトエンドポイント	403 Forbidden (パブリックアクセス完全遮断)
バケットポリシー	OAI (`aws:SourceArn: CloudFront`) のみ許可
CloudFront 経由 HTTPS	200 OK

API の HTTP URL スルー防止 (`aws_backend.py`):

```
        # _resolve_image_urls: http:// URL はスキップして Mixed Content を防ぐ
        if k.startswith("https://"):
```

```
            result.append(k)            # そのまま使用
        elif k.startswith("http://"):
            logger.warning("Skipping insecure HTTP image URL")  # スキップ
        else:
            result.append(self._key_to_presigned_url(k))
# S3キー → presigned HTTPS URL
```

Azure CORS 設定（重要 — Azure に触れる前に必読）

Azure Functions (Flex Consumption) has **two independent CORS layers** that must both be configured correctly. Setting CORS in Python/FastAPI code has no effect.

Layer 1 — Function App platform CORS (controls API requests)

Kestrel (.NET HTTP server) intercepts all `OPTIONS` preflight requests before the Python runtime. Configure via Azure CLI:

```
        # Remove wildcard first (wildcards suppress per-origin rules)
        az functionapp cors remove ¥
          --resource-group "$RESOURCE_GROUP" ¥
          --name "$FUNCTION_APP_NAME" ¥
          --allowed-origins '*'

        # Add specific origins
        az functionapp cors add ¥
          --resource-group "$RESOURCE_GROUP" ¥
```

```
          --name "$FUNCTION_APP_NAME" ¥
          --allowed-origins "https://your.domain.com"
```

Layer 2 — Blob Storage CORS (controls image uploads via SAS URL)

Image uploads go directly from the browser to Blob Storage via SAS PUT URL — they do NOT pass through the Function App. Blob Storage CORS is completely independent:

```
        az storage cors clear --account-name "$STORAGE_ACCOUNT" --
services b
        az storage cors add ¥
          --account-name "$STORAGE_ACCOUNT" ¥
          --services b ¥
          --methods GET POST PUT DELETE OPTIONS ¥
          --origins "https://your.domain.com" ¥
          --allowed-headers "*" ¥
          --exposed-headers "*" ¥
          --max-age 3600
```

Summary

```
Browser → Function App  API calls (GET/POST/PUT/DELETE)
  ⇒ Configured via: az functionapp cors add

Browser → Blob Storage  Image uploads (SAS URL PUT)
  ⇒ Configured via: az storage cors add --services b
```

監査ログ設定（2026-02-24 実装）

AWS — CloudTrail

Pulumi リソース (`infrastructure/pulumi/aws/__main__.py`):

- `cloudtrail_bucket` : S3 バケット (パブリックアクセス完全遮断・バージョニング有効)
- `cloudtrail_bucket_policy` : CloudTrail サービス専用バケットポリシー
- `cloudtrail` : `aws.cloudtrail.Trail`
  - `is_multi_region_trail=True` — 全リージョン対象
  - `include_global_service_events=True` — IAM / STS / Cognito を含む
  - `enable_log_file_validation=True` — SHA-256 ダイジェストによるログ改ざん検知

出力	内容
`cloudtrail_name`	Trail 名 (`{project}-{stack}-trail`)
`cloudtrail_bucket_name`	ログ保存先 S3 バケット

GCP — Cloud Audit Logs

Pulumi リソース (`infrastructure/pulumi/gcp/__main__.py`):

- `audit_config` : `gcp.projects.IAMAuditConfig(service="allServices")`
  - `ADMIN_READ` — 管理者操作 (無料)

- `DATA_READ` — Firestore / GCS データ読み取り
- `DATA_WRITE` — Firestore / GCS データ書き込み

Cloud Logging のログエクスプローラ (`https://console.cloud.google.com/logs`) で確認可能。

Azure — Log Analytics Workspace

Pulumi リソース (`infrastructure/pulumi/azure/__main__.py`):

- `log_analytics_workspace` : `azure.operationalinsights.Workspace`
  - SKU: `PerGB2018` (月 5 GB 無料枠)
  - 保存期間: 30 日
- `app_insights` : `ingestion_mode="LogAnalytics"` に変更 (旧: `ApplicationInsights`)
- `frontdoor_diagnostics` : `azure.insights.DiagnosticSetting`
  - Front Door の `allLogs` カテゴリを Log Analytics に転送

出力	内容
`log_analytics_workspace_name`	Workspace 名 (`mcad-logs-{suffix}`)
`log_analytics_workspace_id`	ARM リソース ID

ID・アクセス管理（IAM/RBAC）

> ⚠ 重大: このセクションはセキュリティ上非常に重要で、絶対に違反してはいけないルールを説明します。 必須の実装ポリシーは

> AI_AGENT_00_CRITICAL_RULES.md — ルール 16 を参照してください。

基本原則 — 最小権限の法則（**Principle of Least Privilege**）

すべてのユーザーおよびサービスアカウントには、その職務を遂行するために最小限の権限のみを付与します。

- デプロイユーザー — デプロイ実行に必要な権限のみ（管理者権限なし）
- 管理者ユーザー — 権限付与の実施のみ（デプロイ権限なし）
- サービスアカウント — 特定の機能制限（読み取り / 特定リソースアクセス）

権限分離マトリックス

ユーザー/サービスアカウント	**AWS**	**Azure**	**GCP**
satoshi (デプロイ)	デプロイ権限	未割り当て（要設定）	Cloud Storage
administrator (管理者)	AdministratorAccess	Owner（Azure 管理者）	N/A

ユーザー/サービスアカウント	**AWS**	**Azure**	**GCP**
sat0sh1kawada00@gmail.com	N/A	N/A	Owner ProjectIa
sat0sh1kawada01@gmail.com	N/A	N/A	Cloud Storage
github-actions-deploy	N/A (IAM認証)	Managed Identity	Service

AWS IAM 権限構成 (2026-02-27 更新)

デプロイユーザー: **satoshi**

付与済みポリシー:

1. **GitHubActionsDeploymentPolicy** （カスタムポリシー）
   - Lambda 関数の更新コード・設定変更
   - Lambda レイヤーの公開・削除
   - S3 バケット（フロントエンド）へのオブジェクトアップロード
   - CloudFront キャッシュ無効化・オリジンアクセスコントロール管理
   - API Gateway 操作（GET / POST のみ）
   - CloudFront 関数の作成・更新・削除・公開
2. **SNSUnsubscribePermission** （インラインポリシー、ユーザーポリシー）
   - SNS トピック管理（作成・削除・属性取得・タグ付与）
   - SNS サブスクリプション管理（購読・購読解除）

削除済みポリシー:

- ☒ `MultiCloudAutoDeployPhase1` ー 古い Terraform ベースポリシー（不要）
- ☒ `APIGatewayV2FullAccess` ー 過度な権限（デプロイに不要）

権限スコープ:

- AWS アカウント: `278280499340`
- リージョン: `ap-northeast-1` （メイン）、 `us-east-1` （CloudFront）

- リソース名パターン: `multicloud-auto-deploy-*`

管理者ユーザー: **administrator**

付与済みポリシー:

1. **AdministratorAccess** （AWS マネージドポリシー）
    - すべてのサービスに対する読み書き権限
    - IAM ユーザー・ロール・ポリシーの管理

削除済みポリシー:

- ☒ `APIGatewayV2FullAccess` — 管理者権限で不要
- ☒ `LambdaLayerFullAccess` — 管理者権限で不要

用途: 権限付与・取り消し、緊急対応、ポリシー更新など

Azure RBAC 権限構成 (2026-02-27 更新)

デプロイユーザー: **satoshi (satoshi@sat0sh1kawadaoutlook.onmicrosoft.com)**

現在の権限: ☒未割り当て （Contributor ロールを削除済み 2026-02-27）

推奨割り当て（実装待機中）:

デプロイに必要な権限は、使用するサービスに応じて異なります。現在の architecture では以下が推奨されます：

- **Website Contributor** — App Service / Static Web Apps でのデプロイ

- **Storage Account Contributor** — Blob Storage へのアップロード
- **User Access Administrator** — Azure Managed Identity 管理（Function App の Managed Identity 設定用）
- カスタムロール: `Deployment Contributor` (実装予定)

割り当てコマンド例:

```
az role assignment create ¥
  --assignee satoshi@sat0sh1kawadaoutlook.onmicrosoft.com ¥
  --role "Website Contributor" ¥
  --scope "/subscriptions/29031d24-d41a-4f97-8362-46b40129a7e8"
```

管理者ユーザー: **administrator (administrator@sat0sh1kawadaoutlook.onmicrosoft.com)**

付与済み権限:

- **Owner** ロール（サブスクリプション全体）✓

用途: 権限付与・取り消し、リソース管理、緊急対応

権限スコープ: サブスクリプション `29031d24-d41a-4f97-8362-46b40129a7e8` レベル

GCP IAM 権限構成 (2026-02-27 更新)

デプロイユーザー: sat0sh1kawada01@gmail.com

付与済みポリシー:

1. **roles/run.admin** — Cloud Run の完全管理
2. **roles/storage.admin** — Cloud Storage の完全管理

削除済みポリシー:

- ☒ `roles/editor` — 過度な権限（2026-02-27 削除）

権限スコープ: プロジェクト `ashnova` レベル

管理者ユーザー: sat0sh1kawada00@gmail.com

付与済みポリシー:

1. **roles/owner** — プロジェクト所有者（全権限）
2. **roles/resourcemanager.projectIamAdmin** — IAM ポリシー管理
3. **roles/editor** — リソース読み書き（owner によってカバーされるが、明示的に設定済み）

用途: 権限付与・取り消し、プロジェクト設定、緊急対応

CI/CD サービスアカウント: github-actions-deploy@ashnova.iam.gserviceaccount.com

付与済みポリシー:

1. **roles/run.admin** — Cloud Run デプロイ
2. **roles/datastore.owner** — Firestore 管理

ポリシー定義詳細

AWS: GitHubActionsDeploymentPolicy

リソース **ARN** パターン:

```
        Lambda:        arn:aws:lambda:ap-northeast-1:278280499340:function:multicloud-auto-deploy-*
        Lambda Layer:   arn:aws:lambda:ap-northeast-1:278280499340:layer:multicloud-auto-deploy-*
        S3:            arn:aws:s3:::multicloud-auto-deploy-*
        S3 Objects:     arn:aws:s3:::multicloud-auto-deploy-*/*
        API Gateway:    arn:aws:apigateway:ap-northeast-1::/apis/*
        CloudFront:     *（リソースベースの制限なし）
```

アクション:

```
        {
          "Lambda": [
            "lambda:UpdateFunctionCode",
            "lambda:UpdateFunctionConfiguration",
            "lambda:GetFunction",
            "lambda:GetFunctionConfiguration"
          ],
```

```
            "LambdaLayer": [
              "lambda:PublishLayerVersion",
              "lambda:GetLayerVersion",
              "lambda:DeleteLayerVersion"
            ],
            "S3": [
              "s3:PutObject",
              "s3:GetObject",
              "s3:DeleteObject",
              "s3:ListBucket",
              "s3:PutObjectAcl"
            ],
            "CloudFront": [
              "cloudfront:CreateInvalidation",
              "cloudfront:GetInvalidation",
              "cloudfront:ListInvalidations",
              "cloudfront:CreateOriginAccessControl",
              "cloudfront:UpdateOriginAccessControl",
              "cloudfront:DeleteOriginAccessControl",
              "cloudfront:ListOriginAccessControls",
              "cloudfront:CreateFunction",
              "cloudfront:UpdateFunction",
              "cloudfront:DeleteFunction",
              "cloudfront:PublishFunction",
              "cloudfront:GetFunction",
              "cloudfront:DescribeFunction",
              "cloudfront:ListFunctions"
            ],
            "APIGateway": ["apigateway:GET", "apigateway:POST"]
          }
```

AWS: SNSUnsubscribePermission

リソース: `arn:aws:sns:*:278280499340:multicloud-auto-deploy-*`

アクション:

```
        [
          "sns:CreateTopic",
          "sns:DeleteTopic",
          "sns:GetTopicAttributes",
          "sns:SetTopicAttributes",
          "sns:ListTopics",
          "sns:TagResource",
          "sns:UntagResource",
          "sns:ListTagsForResource",
          "sns:Subscribe",
          "sns:Unsubscribe",
          "sns:GetSubscriptionAttributes",
          "sns:SetSubscriptionAttributes",
          "sns:ListSubscriptions",
          "sns:ListSubscriptionsByTopic"
        ]
```

チェックリスト — 権限設定検証

AWS

- **satoshi:** `AdministratorAccess` がアタッチされていない ✓
- **satoshi:** `GitHubActionsDeploymentPolicy` がアタッチされている ✓
- **satoshi:** `SNSUnsubscribePermission` がアタッチされている ✓
- **administrator:** `AdministratorAccess` がアタッチされている ✓

administrator: デプロイ権限ポリシーがアタッチされていない ✓

検証コマンド:

```
        # satoshi の確認
        aws iam list-attached-user-policies --user-name satoshi

        # administrator の確認
        aws iam list-attached-user-policies --user-name
administrator
```

Azure

satoshi: Contributor / Owner ロールがアタッチされていない ✓

satoshi: Website Contributor または **Deployment Contributor** がアタッチされている (要実装)

administrator@sat0sh1kawadaoutlook.onmicrosoft.com: Owner ロールがアタッチされている ✓

検証コマンド:

```
        az role assignment list --assignee
satoshi@sat0sh1kawadaoutlook.onmicrosoft.com
        az role assignment list --assignee sat0sh1kawada@outlook.com
        az role assignment list --assignee
administrator@sat0sh1kawadaoutlook.onmicrosoft.com
```

GCP

- **sat0sh1kawada01@gmail.com: Editor / Owner** ロールがアタッチされていない ✓
- **sat0sh1kawada01@gmail.com: Cloud Run Admin** がアタッチされている ✓
- **sat0sh1kawada01@gmail.com: Storage Admin** がアタッチされている ✓
- **sat0sh1kawada00@gmail.com: Owner** ロールがアタッチされている ✓
- **sat0sh1kawada00@gmail.com: projectIamAdmin** ロールがアタッチされている ✓

検証コマンド:

```
        gcloud projects get-iam-policy ashnova --flatten="bindings[].members" --filter="bindings.members:sat0sh1kawada01@gmail.com"
        gcloud projects get-iam-policy ashnova --flatten="bindings[].members" --filter="bindings.members:sat0sh1kawada00@gmail.com"
```

次セクション

→ 09 — Remaining Tasks

09 — 残タスク

> Part III — 運用 | 親: AI_AGENT_GUIDE.md 最終更新: 2026-02-27 Session 3（S1・S2・Task 13 完了 ☒/ セキュリティ本番反映・

> Managed Identity・README更新） **AI** エージェント注記: タスクを解決したらこのファイルを更新してください。

ステータス概要（Updated 2026-02-27 Session 5）

```
Infrastructure (Pulumi):    ☒全3クラウド staging+production デプロイ完了
AWS API (production):       ☒{"status":"ok","provider":"aws","version"
GCP API (production):       ☒{"status":"ok","provider":"gcp","version"
Azure API (production):     ☒{"status":"ok","provider":"azure","versio
E2E test-sns-all.sh:        ☒AWS 9/0, Azure 17/0, GCP 13/0 = 39 tests
Version scheme:             ☒X.Y.Z → A.B.C.D に変更。現在値 1.0.98.236（C=p
AWS API (staging):          ☒{"status":"ok","provider":"aws","version"
GCP API  (staging):         ☒{"status":"ok","provider":"gcp","version"
Azure API (staging):        ☒{"status":"ok","provider":"azure","versio
Security hardening (S1):    ☒CORS・CloudTrail・HTTPS redirect・AuditLog
Managed Identity (S2):      ☒Azure Function App に SystemAssigned MSI
Azure WAF (Task 6):         ☒Function App ミドルウェアで SQL injection/XS
DynamoDB GSI (Task 3):      ☒PostIdIndex & UserPostsIndex 検証済み（stag
Lambda Layer CI/CD (Task 12): ☒pip warnings suppression・GitHub Actions
GCP production state (0c):  ☒409 Conflict 解除 → 34 unchanged で正常復旧
deploy-azure v Python:      ☒Python 3.12-slim --platform linux/amd64 設
Audit logs (0a/0b):         ☒GCP staging/production IAMAuditConfig 作成
README update (Task 13):    ☒エンドポイント・セキュリティ・テスト結果・デプロイ
Defender for Cloud (Azure): ⚠  S2 ☒ S3 ☒(複数オーナー=手動)、Task 20/21
Key Vault Diagnostics:      ☒Log Analytics 統合（AuditEvent ストリーミング
```

⊠高優先度タスク（2026-02-27 **Session 3** 更新）

⊠**2026-02-27 Session 3** で解決済み

#	タスク	説明	状態
S1	⊠ **pulumi up** ー セキュリティ本番反映	**DONE 2026-02-27** — GCP staging/production、AWS production、Azure staging/production デプロイ完了	⊠完了
S2	⊠ **Function App Managed Identity**	**DONE 2026-02-27** — staging/production 両方に SystemAssigned MSI 割り当て完了	⊠完了
20	⊠ **Azure Key Vault** 強化	**DONE 2026-02-27** — purge protection 本番反映 + 診断ログ Log Analytics 統合完了	⊠完了
21	⊠ セキュリティ連絡先 / アラート	**DONE 2026-02-27** — Azure Defender 高優先度アラート + RBAC notifications 構成済み	⊠完了
13	⊠**README** を更新	**DONE 2026-02-27** ー エンドポイント・セキュリティ実装・テスト結果・デプロイ状況を反映	⊠完了
0a	⊠ **GCP audit logs** 有効化	**DONE 2026-02-27** — staging/production IAMAuditConfig 作成、Cloud Audit Logs enable=true	⊠完了
0b			

#	タスク	説明	状態
	☒**GCP billing budget** 対応	**DONE 2026-02-27** — ADC エラー回避、enable_billing_budget=False、monitoring.py optional化	☒完了
0c	☒ **GCP production state drift** 解決	**DONE 2026-02-27** — `pulumi refresh` で state 同期後、`pulumi up` 成功（34 unchanged）	☒完了
0d	☒ **deploy-azure Python** 3.12	**DONE** — CI/CD: `python:3.12-slim --platform linux/amd64` 設定済み	☒完了

☒2026-02-27 Session 2 で解決済み

#	タスク	説明	状態
1	☒ 統合テスト実行（80%以上 **pass**）	**DONE 2026-02-27** — AWS 9/0、Azure 17/0、GCP 13/0 = **39/39 PASS** (100%)	☒
2	☒ **Azure `PUT /posts`** エンドポイント検証	**DONE 2026-02-27** — コード実装済み（404/403 エラーハンドリング検証）。本番稼働可能。	☒
5	☒ **GCP HTTPS redirect**（**Pulumi** コード）	**DONE** — HTTP → HTTPS `redirect_url_map` で実装済み。S1で本番反映。	☒
4		**DONE** — AWS Lambda IAM に `sns:Unsubscribe` 権限追加済み。API	☒

#	タスク	説明	状態
	☒ `SNS:Unsubscribe` 権限エラー修正	DELETE は現在 Unsubscribe 呼び出しなし（準備完了）	

2026-02-24 セッションで解決済みの問題

問題	修正内容	コミット
AWS Production SNS — “Network Error”	CI/CD `deploy-aws.yml` “Sync Pulumi Config” ステップを GitHub Secrets（`staging.aws.ashnova.jp`）から `Pulumi.production.yaml` 読み込みに変更。React SPA 再ビルド・S3 デプロイ。Lambda `CORS_ORIGINS` 即時修正。Cognito implicit フロー削除	v1.17.10（`3ea6a08`）
Azure Function App — 0 registered functions	Python 3.12 / linux/amd64 でパッケージを再ビルドし `--build-remote false` でデプロイ。Layer 1〜4 の多層根本原因を解消	v1.17.9
develop ブランチが main から遅延	`git merge main --no-ff` 実行 → `develop` v1.18.1 同期	—
Azure プロフィール画	platform CORS `az functionapp cors add` で `https://`	v1.17.15

問題	修正内容	コミット
面 CORS エラー	`www.azure.ashnova.jp` を追加。`CORS_ORIGINS` アプリ設定も修正。`deploy-azure.yml`: YAML から customDomain 読み取り + “Ensure Azure CORS Origins” 安全ネット追加。`Pulumi.production.yaml`: `customDomain: www.azure.ashnova.jp` 追加	
Azure ログイン後に staging SNS に遷移	Azure AD redirect URIs に `www.azure.ashnova.jp` を追加。フロントエンドを正しい `VITE_AZURE_REDIRECT_URI` で再ビルド → `index-CPcQQsCR.js`。`deploy-azure.yml` の 4箇所の `${{ secrets.AZURE_CUSTOM_DOMAIN }}` を stack 名マッピングに変更	v1.17.16

2026-02-23 セッションで解決済みの問題

問題	修正内容	コミット
deploy-azure.yml: `run:` キー重複（YAML error）	`Build and Package Azure Function` ステップ追加	v1.16.2
deploy-azure.yml: `$RG_NAME` 未定義（`az functionapp config hostname list` 失敗）	`RG_NAME=$(pulumi stack output ...)` に修正	v1.17.3
		v1.17.2

問題	修正内容	コミット
deploy-gcp.yml: Python heredoc が YAML block scalar を壊す	Firebase domain更新を `jq` コマンドに置き換え	
deploy-azure.yml: AFD route に custom domain が未リンク	AFD route PATCH 後、ワークフローに “Link Custom Domain” ステップ追加	v1.17.4
deploy-azure.yml: `AzureWebJobsStorage` が存在しないSA `multicloudautodeploa148` を指していた	zip deploy前にストレージを `mcadfuncdiev0w` に修正するステップ追加	v1.17.6
deploy-landing-azure.yml: staging にハードコード（production デプロイ不可）	environment-aware に修正（`main` → production SA `mcadwebdiev0w`）	v1.17.5
Azure CDN landing page: 843バイトのReact SPA が配信されていた	上記 deploy-landing-azure.yml 修正により解消 → 4412バイトの正しいコンテンツ	v1.17.5
Azure ログイン「認証設定が不完全です」	Pulumi出力 `azure_ad_client_id` が空 → `VITE_AZURE_CLIENT_ID=''` → Provider=‘none’。`index-CzWB96PN.js` を手動ビルド＆Blob Storage デプロイ。`deploy-azure.yml` にフォールバック追加	v1.17.21

問題	修正内容	コミット
GCP プロフィール画面が表示されない	`deploy-gcp.yml` にフロントエンドビルド/デプロイステップが存在しなかった。`index-DNqlhCH0.js` を手動ビルド＆GCS デプロイ。ワークフローにステップ追加	v1.17.20
危険なサイト警告（all clouds）	non-www（`azure.ashnova.jp` 等）→ Google Safe Browsing 警告。`main.tsx` に `window.location.replace()` リダイレクト追加（3クラウド対応）	v1.17.17–19
CI/CD 環境変数消去・混在（全クラウド）	`Pulumi.*.yaml` gitignore → Secrets fallback → 環境混在。`.github/config/` 導入により根本解決。Lambda Layer 名バグ・全 `case/esac` 廃止	v1.17.22

2026-02-27 セッションで解決済みの内容

Task	Description	Status
GCP audit logs 再有効化	`gcloud auth application-default login` 再実行 + staging/production IAMAuditConfig 作成 → Cloud Audit Logs 有効化 ☒	☒

Task	Description	Status
GCP billing budget対応	ADC認証エラー（quota project未設定）を回避。コード修正で `enable_billing_budget=False` デフォルト無効化。GCP側oldbudgetリソース削除 ☒	☒
Pulumi monitoring.py 重構成	`billing_account_id` をoptional パラメータ化。billing budget作成時にのみ必須。production では常にNone → budget作成スキップ ☒	☒

☒中優先度タスク

#	Task	Description
7	☒ ~~**Release unused GCP static IPs**~~	**DONE 2026-02-24** — 3座のRESERVED IP削除済み。
8	☒ ~~**Delete unused GCP Cloud Storage buckets**~~	**DONE 2026-02-24** — 4バケット + FAILED Cloud Function削除済み。
9	監視とアラート設定	CloudWatch Alarms（AWS）/ Azure Monitor（Azure）/ Cloud Monitoring（GCP）のアラート設定。monitoring.py は存在するが詳細チューニング未済。

#	Task	Description	
10	☒ ~~**Security hardening（Pulumi コード）**~~	**DONE 2026-02-24** — CORS 絞り込み / CloudTrail / GCP HTTPS redirect / GCP AuditLogs / Azure Log Analytics を Pulumi コードに実装。`pulumi up` **(S1)** で本番反映が必要。	
11	**WAF** ログ集約	AWS WAF ログ・GCP Cloud Armor ログ・Azure Front Door ログを一元集約。Azure は Log Analytics Workspace が追加済みなので Front Door 側のシンク設定のみ必要。	
12	**Lambda Layer CI/CD** の完全自動化	レイヤービルドと公開時の non-fatal warning を除去。	
13	☒ **README** 更新	**DONE 2026-02-27** — エンドポイント・セキュリティ実装状況・テスト結果を反映	
14	ブランチ保護ルール	`main` / `develop` への直接 push を禁止。PR + CI pass を必須化。	
20	**Azure Key Vault** 強化（**Pulumi**）	Defender for Cloud 指摘3件をまとめて対応。①消去保護: `enable_purge_protection=True` 追加 ②ファイアウォール: `default_action="Deny"` に変更（S2完了後）③診断ログ: Key Vault	08_SECURITY

#	Task	Description	
		向け DiagnosticSetting 追加。staging/production 両スタックで `pulumi up` を実行。	
21	**Azure** セキュリティ連絡先 / アラート通知設定	Defender for Cloud 指摘3件（Low/Medium）。`az security contact create` で連絡先メールと重要度高アラートを設定。サブスクリプション所有者へのアラート通知も同時に有効化。	08_SECURITY

☒低優先度タスク

#	Task	Description
15	~~**Custom domain setup**~~ ☒	全3クラウドで完了（2026-02-21）。CUSTOM_DOMAIN_SETUP.md を参照。
16	負荷テスト	Locust などで性能ベースラインを確立。
17	**CI/CD** 障害通知	Slack / Discord webhook 連携を追加。
18	テストカバレッジ拡張	☒部分解決（2026-02-24）: `test-sns-all.sh` 追加、AWS/GCP 自動認証、3クラウドで binary PUT + imageUrl アクセス確認を実装。残作業: Azure 自動認証（`--auto-token` 相当）、CI/CD への統合。
19		

#	Task	Description
	カオスエンジニアリング	ネットワーク障害、DB障害、コールドスタート急増をシミュレート。

推奨作業順序

```
S1 → [IMMEDIATE] pulumi up — セキュリティ変更適用（CORS / CloudTrail / GCP
     順序: gcp/staging → aws/production → gcp/production → azure/staging
      NOTE: state drift 解消のため、先に GCP スタックで `pulumi refresh` を実
S2 → [IMMEDIATE] az functionapp identity assign（staging + production）-
S3 → [担当者要確認] サブスクリプション所有者を追加
20 → S2完了後、Azure Key Vault強化（Pulumi: 消去保護 + ファイアウォール + 診断ロ
21 → az security contact create — セキュリティ連絡先メール設定
☒1 → 統合テスト実行（現状ベースライン確立）（DONE 2026-02-27 — AWS 9/0, Azure
☒2 → Azure PUT /posts 検証（DONE 2026-02-27 — routes/posts.py に PUT /{po
☒3 → DynamoDB GSI 確認（DONE 2026-02-27 — PostIdIndex & UserPostsIndex を
☒4 → SNS:Unsubscribe 修正（DONE — 2026-02-17 に IAM 権限追加、Lambda が uns
☒5 → GCP HTTPS redirect（Pulumi コードは 2026-02-24 完了 — S1 で適用）
☒6 → Azure WAF（DONE 2026-02-27 — Function App middleware、staging/produ
☒7 → 未使用 GCP static IP 解放（DONE 2026-02-24）
☒8 → 未使用 GCP Cloud Storage bucket 削除（DONE 2026-02-24）
☒9 → 監視とアラート（DONE 2026-02-27 — AWS/Azure/GCP monitoring.py 実装、Cl
☒10 → セキュリティ強化（Pulumi コード 2026-02-24 完了 — S1 で適用）
☒11 → WAF ログ集約（DONE 2026-02-27 — CloudWatch/Monitor/Logging 設定）
☒12 → Lambda Layer CI/CD 完全自動化（DONE 2026-02-27 — pip warnings 抑制、
14 → ブランチ保護ルール
```

解決済みタスク（履歴）

タスク	解決内容	コミット
GCP GCS CORS error	CORS に `x-ms-blob-type` ヘッダー追加。uploads.js を Azure URL 時のみ送信するよう修正。	`1cf53b7`, `b5b4de5`
GCP Firebase Auth implementation	Google Sign-In フロー、httponly Cookie セッション、Firebase SDK v10.8.0、authorized domains を実装。	`3813577`
GCS presigned URL hardcoded content_type	`generate_upload_urls()` が `content_types[index]` を使うよう修正。拡張子マッピング追加。	`148b7b5`
Firebase ID token expiry (401)	`onIdTokenChanged` で自動更新。`/sns/session` を再呼び出し。	`8110d20`
Missing GCP_SERVICE_ACCOUNT	`deploy-gcp.yml` に `GCP_SERVICE_ACCOUNT` を追加。`impersonated_credentials` を有効化。	`27b10cc`
CSS SVG 404（starfield/ring-dark）	`url("/static/...")` → `url("./...")` に変更。`app.css` を v=4 に更新。	`0ed0805`
GCS uploads bucket images not publicly visible	`allUsers:objectViewer` を付与。Pulumi 定義に IAMBinding を追加。	`0ed0805`

タスク	解決内容	コミット
Azure `/posts` 404	Azure Function の routing は正しい。テストレポートが古かった。POST 201 / GET 200 を確認。	—
AWS Staging POST 401	`AUTH_DISABLED=true` → staging に追加。	`a2b8bb8`
GCP Production GET /posts 500	python312、`GCP_POSTS_COLLECTION=posts`、`SecretVersion` 削除、`functions-framework==3.10.1`	`05829e60`
deploy-gcp.yml ConcurrentUpdateError	3ワークフローすべてに `concurrency` group を追加。	`a2b8bb8`
GCP backend implementation	Firestore CRUD を完全実装。	—
Azure backend implementation	Cosmos DB CRUD を完全実装。	—
AWS CI/CD Lambda Layer conditional	重複/条件分岐ステップを削除し、単一の無条件ビルドに統一。	`eaf8071c`
Azure hardcoded resource group	3ワークフローの固定値 `multicloud-auto-deploy-staging-rg` を Pulumi output 参照へ変更。	`0912ac3`
Workflow file duplication	サブディレクトリではなくルート `.github/workflows/` を編集するよう修正。	`c347727`

タスク	解決内容	コミット
Landing page overwrote SNS app	フロントエンド CI の deploy 先を `sns/` プレフィックスへ変更。	`c347727`
AUTH_DISABLED=true bug（AWS/Azure staging）	条件分岐を削除し、常に `AUTH_DISABLED=false` を設定。	`6699586`
Landing page SNS link used `:8080`	3環境（local/devcontainer/CDN）対応の host 判定ロジックに修正。	`0c485b7`

10 — カスタムドメイン・テスト

> Part III — Operations | Parent: AI_AGENT_GUIDE.md

カスタムドメインステータス (全て確認済み 2026-02-21)

クラウド	カスタムドメイン	**CDN** ターゲット	**DNS** タイプ
AWS	`www.aws.ashnova.jp`	`d1qob7569mn5nw.cloudfront.net`	CNAM
Azure	`www.azure.ashnova.jp`	`mcad-production-diev0w-f9ekdmehb0bga5aw.z01.azurefd.net`	CNAM

クラウド	カスタムドメイン	CDN ターゲット	DNS タイプ
GCP	`www.gcp.ashnova.jp`	`34.8.38.222` (A レコード)	A

ステージング CDN エンドポイント (カスタムドメインなし)

クラウド	CDN / Front Door URL	Distribution ID
AWS	`d1tf3uumcm4bo1.cloudfront.net`	E1TBH4R432SZBZ
Azure	`mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net`	mcad-staging-d45ihd
GCP	`34.117.111.182` (IP)	—

⚠ 重要: 再実行前の Pulumi 設定

本番 AWS の `pulumi up` をこれらの設定値なしで実行すると、CloudFront がデフォルト証明書に戻り **HTTPS** が壊れます:

```
        cd infrastructure/pulumi/aws
        pulumi config set customDomain www.aws.ashnova.jp --stack
production
        pulumi config set acmCertificateArn arn:aws:acm:us-
```

```
east-1:278280499340:certificate/
914b86b1-4c10-4354-91cf-19c4460dcde5 --stack production
```

ACM 証明書有効期限: 2027-03-12

DNS レコード (確認済み)

AWS

```
CNAME  www.aws.ashnova.jp  →  d1qob7569mn5nw.cloudfront.net
CNAME  _<id>.www.aws.ashnova.jp  →  _<id>.acm-validations.aws.  (ACM 検
```

Azure

```
CNAME  www.azure.ashnova.jp  →  mcad-production-diev0w-f9ekdmehb0bga5aw
TXT    _dnsauth.www.azure.ashnova.jp  →  <validationToken>  (AFD 検証)
```

GCP

```
A  www.gcp.ashnova.jp  →  34.8.38.222  (production)
A  staging.gcp.ashnova.jp  →  34.117.111.182  (staging)
```

再セットアップ手順 (ドメイン再構成が必要な場合)

AWS

```
        # 1. ACM 証明書確認 / リクエスト（us-east-1 のみ）
        aws acm list-certificates --region us-east-1 ¥
          --query "CertificateSummaryList[?
DomainName=='www.aws.ashnova.jp'].CertificateArn" ¥
          --output text

        # 2. Pulumi 設定を設定
        cd infrastructure/pulumi/aws
        pulumi config set customDomain www.aws.ashnova.jp --stack
production
        pulumi config set acmCertificateArn <CERT_ARN> --stack
production
        pulumi up --stack production
```

Azure

```
        RESOURCE_GROUP="multicloud-auto-deploy-production-rg"
        PROFILE_NAME="multicloud-auto-deploy-production-fd"

        # カスタムドメインを作成（べき等）
        az afd custom-domain create ¥
          --resource-group $RESOURCE_GROUP ¥
          --profile-name $PROFILE_NAME ¥
          --custom-domain-name azure-ashnova-jp ¥
          --host-name www.azure.ashnova.jp ¥
          --certificate-type ManagedCertificate

        # 検証ステータスを確認
```

```
        az afd custom-domain show ¥
          --resource-group $RESOURCE_GROUP ¥
          --profile-name $PROFILE_NAME ¥
          --custom-domain-name azure-ashnova-jp ¥
          --query "{provisioningState,domainValidationState}"
```

GCP

```
        cd infrastructure/pulumi/gcp
        pulumi config set customDomain www.gcp.ashnova.jp --stack
production
        pulumi up --stack production
        # 注意: マネージド SSL 証明書のプロビジョニングは DNS 伝播後、最大 60
分かかる場合があります
```

ドメイン変更後の CORS

ドメイン変更時は常に CORS を更新してください:

```
        # AWS
        cd infrastructure/pulumi/aws
        pulumi config set allowedOrigins "https://
www.aws.ashnova.jp,http://localhost:5173" --stack production
        pulumi up --stack production

        # GCP
        cd infrastructure/pulumi/gcp
        pulumi config set allowedOrigins "https://
www.gcp.ashnova.jp,http://localhost:5173" --stack production
        pulumi up --stack production

        # Azure — Azure Function App CORS は CLI 経由で設定（Pulumi 経
由不可）
```

```
az functionapp cors add ¥
  --resource-group multicloud-auto-deploy-production-rg ¥
  --name multicloud-auto-deploy-production-func ¥
  --allowed-origins "https://www.azure.ashnova.jp"
```

検証

```
# HTTPS チェック
curl -sI https://www.aws.ashnova.jp   | head -3
curl -sI https://www.azure.ashnova.jp | head -3
curl -sI https://www.gcp.ashnova.jp   | head -3

# API ヘルスチェック
curl -s https://www.aws.ashnova.jp/health
curl -s https://www.gcp.ashnova.jp/health

# DNS 解決
dig www.aws.ashnova.jp
dig www.azure.ashnova.jp
dig www.gcp.ashnova.jp
```

全環境のテスト

クイック接続確認 (~30秒、認証不要)

```
./scripts/test-staging-all.sh --quick
```

クラウドごとのチェック: ランディングページ `/` → 200、SNS アプリ `/sns/` → 200、API `/health` → 200。

全認証テスト (全3クラウド)

```
./scripts/test-staging-all.sh ¥
  --aws-token   "$AWS_TOKEN" ¥
  --azure-token "$AZURE_TOKEN" ¥
  --gcp-token   "$GCP_TOKEN"
```

認証トークンの取得

AWS — Cognito

```
AWS_TOKEN=$(aws cognito-idp initiate-auth ¥
  --auth-flow USER_PASSWORD_AUTH ¥
  --client-id 1k41lqkds4oah55ns8iod30dv2 ¥
  --auth-parameters
USERNAME=YOUR_EMAIL,PASSWORD=YOUR_PASSWORD ¥
  --region ap-northeast-1 ¥
  --query 'AuthenticationResult.AccessToken' ¥
  --output text)
```

GCP — Firebase

```
GCP_TOKEN=$(gcloud auth print-identity-token)
```

Azure — Azure AD (ブラウザのみ)

```
1. https://mcad-staging-d45ihd-dseygrc9c3a3htgj.z01.azurefd.net/sns/ を
2. Microsoft アカウントでログイン
3. DevTools → Application → Local Storage → origin → id_token
AZURE_TOKEN="<id_token をここに貼り付け>"
```

クラウドごとのテストスクリプト

スクリプト	目的
`scripts/test-staging-all.sh`	☒全3クラウドを統合管理
`scripts/test-landing-pages.sh`	ランディングページテストのみ
`scripts/test-sns-aws.sh`	AWS 全体スイート (認証あり)
`scripts/test-sns-azure.sh`	Azure 全体スイート (認証あり)
`scripts/test-sns-gcp.sh`	GCP 全体スイート (認証あり)
`scripts/test-sns-all.sh`	バイナリ PUT + imageUrl チェック付き E2E
`scripts/test-e2e.sh`	軽量マルチクラウドスモークテスト

pytest 統合テスト (**unit/integration**、モック)

```
# 全テスト
cd services/api && pytest tests/

# クラウドバックエンド別
pytest tests/ -m aws
pytest tests/ -m gcp
pytest tests/ -m azure

# シェルスクリプト経由
./scripts/run-integration-tests.sh      # 標準
```

```
        ./scripts/run-integration-tests.sh -v     # verbose
        ./scripts/run-integration-tests.sh --coverage  # カバレッジレポート付き
```

テストカバレッジには、全3バックエンドの CRUD 操作、権限チェック、ページネーション、タグフィルタリング、プロフィール更新が含まれます。

次のセクション

→ 00 — Critical Rules (循環) | 07 — Runbooks

11 — バグ・修正レポート

> 目的: 3つのクラウド環境全体における既知バグと修正の統合インデックス。 繰り返して現れる問題をデバッグするときや、変更をデプロイする前に、このドキュメントを参照します。

概要

日付	レポート	クラウド	バグ数	ステータス
2026-02-20	AWS SNS Fix (staging)	AWS	4	☒ All fixed
2026-02-21	AWS HTTPS Fix (production)	AWS	1	☒ Fixed
2026-02-21	AWS Production SNS Fix	AWS	2	

日付	レポート	クラウド	バグ数	ステータス
				☒ All fixed
2026-02-21	React SPA Migration & CDN Fix	AWS/Azure/GCP	3 CDN	☒ All fixed
2026-02-21	Azure SNS Fix (503/404 + AFD 502)	Azure	5+1	☒ All fixed
2026-02-22	AWS SNS Fix (staging, 12 bugs)	AWS	12	☒ All fixed
2026-02-22	SNS Fix (AWS+Azure combined)	AWS/Azure	10	☒ All fixed
2026-02-22	Refactoring & Infra Fix	All	5 infra	☒ All fixed
2026-02-23	GCP SNS Fix (staging)	GCP	6	☒ All fixed
2026-02-27	OCR Formula Merge Bugs	API	3	☒ All fixed

目次

1. AWS SNS 修正（2026-02-20）
2. AWS 本番 HTTPS 修正（2026-02-21）
3. AWS 本番 SNS 修正（2026-02-21）

4. React SPA マイグレーション・CDN ルーティング修正（2026-02-21）
5. Azure SNS 修正（2026-02-21）
6. AWS SNS 修正 12 バグ（2026-02-22）
7. AWS + Azure 統合 SNS 修正（2026-02-22）
8. リファクタリング・インフラ修正（2026-02-22）
9. GCP SNS 修正（2026-02-23）
10. OCR 数式マージバグ（2026-02-27）

---

1. AWS SNS 修正レポート（2026-02-20）

Environment: `https://d1tf3uumcm4bo1.cloudfront.net/sns/` (staging) **Branch**: develop — commits `c5a261c` → `4d2bce0` **Source**: AWS_SNS_FIX_REPORT.md

#	Symptom	Root Cause	Files Changed
1	Profile page: “Sign in to see profile details”, API base = `localhost:8000`	CI/CD overwrote Lambda env vars with empty strings on every push; `ResourceConflictException` silently skipped code update	`.github/workflows/deploy-aws.yml`
2	CI/CD fixes had no effect	Edits were applied to `multicloud-auto-deploy/.github/workflows/` (subdirectory copy) — GitHub	`.github/workflows/`

#	Symptom	Root Cause	Files Changed
		Actions reads only the repo-root `.github/workflows/`	`deploy-aws.yml`
3	Logout redirected to `/login` → HTTP 404	`auth.py` hardcoded `/login` fallback; root cause was Bug 1 (missing Cognito env vars)	`services/frontend_web/app/routers/auth.py`
4	`POST /uploads` → 502 Bad Gateway	`UploadUrlsRequest.count le=10` and `CreatePostBody.imageKeys max_length=10`; frontend allows 16	`services/api/app/models.py`

Key Fixes

```
        # Bug 1 — deploy-aws.yml: added aws lambda wait before code update
        - name: Update frontend-web Lambda
          run: |
            aws lambda wait function-updated --function-name $FN
            aws lambda update-function-configuration \
              --function-name multicloud-auto-deploy-staging-frontend-web \
              --environment "Variables={AUTH_DISABLED=false, API_BASE_URL=$API_ENDPOINT, ...}"
            aws lambda wait function-updated --function-name $FN
```

```
        # Bug 4 — models.py: raised limits 10 → 16
        count: int = Field(..., ge=1, le=16)
```

```
        image_keys: Optional[list[str]] = Field(None,
alias="imageKeys", max_length=16)
```

Key Lesson

> **Always edit `.github/workflows/` at the repo root.** The subdirectory copy at `multicloud-auto-deploy/.github/workflows/` is ignored by GitHub Actions.

---

2. AWS Production HTTPS Fix (2026-02-21)

Environment: `https://www.aws.ashnova.jp` (production) **CloudFront Distribution**: `E214XONKTXJEJD` (`d1qob7569mn5nw.cloudfront.net`) **Source**: AWS_HTTPS_FIX_REPORT.md

Symptom

```
NET::ERR_CERT_COMMON_NAME_INVALID — Your connection is not private
```

Root Cause

CloudFront distribution `E214XONKTXJEJD` had no custom domain alias or ACM certificate configured. `pulumi/aws/__main__.py` reads `config.get("customDomain")` — since the Pulumi config values were never set, the `else` branch always ran `cloudfront_default_certificate=True`.

Setting	Broken	Fixed
`Aliases`	`Quantity: 0`	`["www.aws.ashnova`
`ViewerCertificate`	`CloudFrontDefaultCertificate: true`	ACM `914b` (`www.aws.ashnova.j` expires 2027-03-12

Fix (Immediate)

Applied via `aws cloudfront update-distribution --id E214XONKTXJEJD`.

Fix (Permanent — run before every `pulumi up --stack production`)

```
        cd infrastructure/pulumi/aws
        pulumi stack select production
        pulumi config set customDomain www.aws.ashnova.jp
        pulumi config set acmCertificateArn arn:aws:acm:us-
east-1:278280499340:certificate/
914b86b1-4c10-4354-91cf-19c4460dcde5
        pulumi up --stack production
```

> **Warning**: Without setting these config values, `pulumi up` reverts CloudFront to `CloudFrontDefaultCertificate: true`, reproducing the HTTPS error.

---

3. AWS Production SNS Fix (2026-02-21)

Environment: `https://www.aws.ashnova.jp/sns/` (production)
Branch: main — commits `fd1f422` `8188682` **Source**: AWS_PRODUCTION_SNS_FIX_REPORT.md

#	Symptom	Root Cause	Fix
1	All API calls hit `localhost:8000` → Connection refused	`API_BASE_URL=""` — CI/CD read from GitHub Secrets never configured for `production` env	Rewrote workflow to use Pulumi stack outputs instead of Secrets
2	Cognito login redirect URL rejected	`COGNITO_REDIRECT_URI` used CloudFront domain; Cognito App Client only allows `www.aws.ashnova.jp`	Derive `SITE_DOMAIN` from `custom_domain` Pulumi output, fall back to CloudFront domain

Root Cause Detail — Bug 1

```
        # BEFORE (☒empty string when secret is unset — silently
overwrites Lambda env)
        API_URL="${{ secrets.API_GATEWAY_ENDPOINT }}"

        # AFTER (☒always correct — reads from Pulumi stack)
        - name: Get Pulumi Outputs
          run: |
            pulumi stack select "${{ steps.env.outputs.env_name }}"
```

```
            echo "api_gateway_endpoint=$(pulumi stack output
api_gateway_endpoint)" >> $GITHUB_OUTPUT
```

Guard clause added to abort deployment if outputs are empty:

```
        if [[ -z "$API_URL" || -z "$CF_DOMAIN" || -z "$CLIENT_ID" ]];
then
          echo "☒Critical Pulumi outputs are empty. Aborting."
          exit 1
        fi
```

Root Cause Detail — Bug 2

```
        # BEFORE (☒CloudFront domain not registered in Cognito App
Client)
        COGNITO_REDIRECT_URI="https://${CLOUDFRONT_DOMAIN}/sns/auth/
callback"

        # AFTER (☒uses custom domain when available)
        CUSTOM_DOMAIN="$
{{ steps.pulumi_outputs.outputs.custom_domain }}"
        SITE_DOMAIN="${CUSTOM_DOMAIN:-$CF_DOMAIN}"
        COGNITO_REDIRECT_URI="https://${SITE_DOMAIN}/sns/auth/
callback"
```

Key Lessons

1. **Never use GitHub Secrets as source of truth for infrastructure values.** Use Pulumi outputs directly.
2. **Production and staging have separate Pulumi stacks** with different API Gateway IDs, Cognito pools, and custom domains.

3. **Empty string `""` in Pydantic settings is NOT the same as absent.** An empty `API_BASE_URL` does not fall back to the default — it is used as-is.

---

4. React SPA Migration & CDN Routing Fix (2026-02-21)

Environment: production (all 3 clouds) **Branch**: main — commits `6aff4ac` `d7df295` **Source**: REACT_SPA_MIGRATION_REPORT.md

Background

Frontend migrated from Python/Reflex SSR (`frontend_web`) to static React SPA (`frontend_react`). After migration CI/CD succeeded but all 3 clouds' CDNs still routed `/sns*` to the old Python origin.

CDN Routing Bugs Fixed

Cloud	CDN Resource	Old Origin (wrong)	New Origin (correct)
AWS	CloudFront `E214XONKTXJEJD` `/sns*` behavior	API Gateway → Lambda SSR	S3 bucket `multicloud-auto-deploy-production-frontend`
Azure	AFD route `production-sns-route`	`frontend-web-origin-group` (deleted Function App)	Blob Storage origin group

Cloud	CDN Resource	Old Origin (wrong)	New Origin (correct)
GCP	URL map path `/sns/*` rule	`frontend-web-backend` (Cloud Run NEG)	Removed — falls to default GCS backend

AWS Additional Bug — S3 directory index

After switching CloudFront `/sns*` to S3, accessing `/sns/` returned the landing page root `index.html` (S3 has no directory-index capability).

Fix: Created CloudFront Function `spa-sns-rewrite-{stack}`:

```
function handler(event) {
  var request = event.request;
  var uri = request.uri;
  if (uri === "/sns" || uri === "/sns/") {
     request.uri = "/sns/index.html";
  }
  return request;
}
```

Associated with the `/sns*` cache behavior in `infrastructure/pulumi/aws/__main__.py`.

CI/CD Workflow Changes

Workflow	Old		New
`deploy-frontend-web-aws.yml`	Docker image Lambda	→	`npm run build`, sync to `s3:// <bucket>/sns/`, invalidate CloudFront
`deploy-frontend-web-azure.yml`	Docker image Function App	→	`npm run build`, upload to Azure Blob `$web/sns/`, purge AFD cache
`deploy-frontend-web-gcp.yml`	Docker image Cloud Run	→	`npm run build`, copy to GCS `sns/` prefix, invalidate CDN

Cache-Control Pattern (AWS/GCP)

```
# Hashed assets — 1-year immutable cache
aws s3 sync dist/assets/ s3://${BUCKET}/sns/assets/ \
  --cache-control "public, max-age=31536000, immutable"
# index.html — no cache (always fetch latest)
aws s3 cp dist/index.html s3://${BUCKET}/sns/index.html \
  --cache-control "no-cache, no-store, must-revalidate"
```

---

5. Azure SNS Fix (2026-02-21)

Environment: Azure staging + production **Source**: AZURE_SNS_FIX_REPORT.md

Part A — 503/404 Initial Setup Bugs

#	Symptom	Root Cause	Fix
A1	All endpoints 503	`host.json` extra closing brace (invalid JSON)	Fixed JSON; add
A2	Functions list empty, host healthy	`WEBSITE_RUN_FROM_PACKAGE` with external SAS URL doesn't register Python v2 functions on Consumption Linux	Switched `az functionap` `zip` (Kudu ZIP
A3	`ModuleNotFoundError: pydantic_core`	Dev container is `aarch64`; Azure Functions runs `x86_64` — `.so` binaries incompatible	Build `docker run -` `3.12-slim`
A4	Static files / templates 404	Relative paths like `StaticFiles(directory="app/static")` fail when CWD is not guaranteed	Use `os.path.dirna` for absolute pa
A5	Functions not invoked via ASGI	`AsgiMiddleware.handle()` (sync) used	Switched to ma

Part B — AFD 502 Intermittent Errors (Production)

Symptom: `www.azure.ashnova.jp/sns/health` returns HTTP 502 ~50% of requests; direct Function App access succeeds 100%.

Root Cause: Dynamic Consumption (Y1) instances are periodically recycled. AFD Standard cannot detect TCP disconnect during recycling — stale connections in the pool return 502 instantly.

Fix: Migrated production Function App from Dynamic Consumption (Y1) → **FC1 FlexConsumption** with `maximumInstanceCount=1` + `alwaysReady http=1`. Result: 0/20 failures after migration.

```
        # Create FlexConsumption Function App (NOT --consumption-plan-location)
        az functionapp create ¥
          --name multicloud-auto-deploy-production-frontend-web-v2 ¥
          --flexconsumption-location japaneast ¥
          --runtime python --runtime-version 3.12 ...
        az functionapp scale config set --maximum-instance-count 1 ...
        az functionapp scale config always-ready set --settings "http=1" ...
```

Key Rules for Azure Deployment

- **Always build with `--platform linux/amd64 python:3.11-slim`** — Dev Container is aarch64; `.so` binaries are incompatible
- **`SCM_DO_BUILD_DURING_DEPLOYMENT` is NOT supported on Flex Consumption** — always deploy pre-built ZIP
- **Do NOT use external SAS URL in `WEBSITE_RUN_FROM_PACKAGE`** on Consumption Linux — Python v2 functions are not registered
- **CORS must be configured at platform level** (`az functionapp cors add`) — Python code CORS headers are ignored because Kestrel handles OPTIONS before the Python runtime

- **Blob Storage CORS is independent from Function App CORS** — must configure separately for SAS URL direct uploads
- **`--consumption-plan-location` creates Dynamic Y1** (causes stale TCP with AFD) — use `--flexconsumption-location` for production

---

6. AWS SNS Fix Report (2026-02-22)

Environment: `https://d1tf3uumcm4bo1.cloudfront.net/sns/` (staging) **Branch**: develop — commits `9b4d37c` → `8c84a15` **Source**: AWS_SNS_FIX_REPORT_20260222.md

#	Symptom	Root Cause	Files
1	Profile GET/PUT → 500	`aws_backend.py` used wrong DynamoDB field names (`userId` instead of `PK`, `eamil` typo)	`backends/aws_backend.py`
2	Login redirects to CloudFront domain	`auth.ts` hardcoded CloudFront origin URL	`frontend_react/src/config/auth.ts`
3	Cognito hosted UI → `/error` page	`--supported-identity-providers` missing from CI/CD `update-user-pool-client`	`.github/workflows/deploy-frontend-web-aws.yml`
4	`POST /uploads/`		

#	Symptom	Root Cause	Files
	`presigned-urls` → 422	`count le=10` too low; `contentTypes` field missing from model	`models.py`, `routes/uploads.py`
5	`GET /profile` → 401	JWT verifier checked `at_hash` without companion access_token; Cognito access_token has no `aud`	`jwt_verifier.py`
6	`POST /posts` → 422	`imageKeys max_length=10`; frontend uploads 13 images	`models.py`
7	Images not displaying	S3 private bucket — raw S3 keys returned instead of presigned GET URLs on read	`backends/aws_backend.py`
8	Nickname missing from post list	`create_post()` didn't fetch nickname from PROFILES table	`backends/aws_backend.py`
9	`GET /posts/{post_id}` → 405	Endpoint not implemented in any backend	`backends/*.py`, `routes/posts.py`
10	No server-side image count enforcement	Limit was frontend-only	`config.py`, `routes/limits.py`, `PostForm.tsx`

#	Symptom	Root Cause	Files
11	MIME type error on JS assets (blank page)	Vite built with `base="/"` but site deployed at `/sns/`	`vite.config.ts`, CloudFront error pages
12	“認証設定が不完全です” login error	`VITE_AUTH_PROVIDER` not set; defaulted to `"none"`	`.env.aws.staging` (new file)

Key Technical Points

```
        # Bug 5 — disable at_hash check when using id_token standalone
        verify_at_hash: False
        # Make aud verification conditional (Cognito access_token has no aud)
        if "aud" in token_claims: verify_aud(...)
```

```
        # Bug 7 — store raw keys, generate presigned URLs at read time
        def _key_to_presigned_url(self, key: str) -> str: ...  # 1-hour expiry
        # DynamoDB: imageKeys (raw S3 keys) — NOT imageUrls
```

```
        # Bug 11 — build with correct base path for sub-path deployment
        VITE_BASE_PATH=/sns/ npm run build
        # CloudFront custom error pages → /sns/index.html (NOT /index.html)
```

---

7. AWS + Azure Combined SNS Fix (2026-02-22)

Environment: AWS staging + Azure staging **Branch**: develop
Source: SNS_FIX_REPORT_20260222.md

AWS Bug — Cognito `unauthorized_client`

Symptom: Cognito hosted UI returns `error=unauthorized_client` after login click.

Root Cause:

- `AllowedOAuthFlows` was missing `implicit`
- `CallbackURLs` did not include the staging domain `staging.aws.ashnova.jp`

Fix (`infrastructure/pulumi/aws/simple-sns/__main__.py`):

```
        allowed_o_auth_flows=["code", "implicit"]
        callback_urls=[
            "https://www.aws.ashnova.jp/sns/auth/callback",
            "https://staging.aws.ashnova.jp/sns/auth/callback",  # added
            "http://localhost:5173/sns/auth/callback",
        ]
```

Azure Bugs Fixed

#	Symptom	Root Cause	Fix
1	Login redirects to FrontDoor internal hostname	Azure AD `redirect_uri` lacked custom domain	Re-registered w `staging.azure.ashnova.jp`
2	SVG images 404	`upload-batch` placed SVGs at `$web/sns/` but CSS references `./assets/`	Copy `dist/*.svg` to `$web/sn` `assets/` in CI/CD
3	All API endpoints 404	Azure Functions default `routePrefix: "api"` → actual path `/api/limits`	Added `"routePrefix": ""` `services/api/host.json`
4	API CORS errors	Kestrel processes OPTIONS before Python runtime; platform CORS not configured	`az functionapp cors add` `allowed-origins "https` `staging.azure.ashnova.jp"`
5	Image upload CORS errors	Blob Storage CORS is independent from Function App CORS	`az storage cors add` on Bl Storage account
6	Logout blocks after sign-out	`post_logout_redirect_uri=/sns/` not in AD app `redirect_uris`	Added `/sns/` to `az ad ap` `update --web-redirect-uris`

#	Symptom	Root Cause	Fix
7	503 all endpoints after deploy	aarch64 `.so` binaries used (Dev Container is ARM); wrong Python version	Use `docker run --platform linux/amd64 python:3.11-slim`
8	Nickname missing from posts	`_item_to_post` missing `nickname` field; `create_post` not fetching profile	Added `nickname=item.get("nickname")` and profile fetch `create_post`

CI/CD Changes (Azure)

Step	Change
Python package build	`docker run --platform linux/amd64 python:3.11-slim pip install`
Platform CORS	`az rest PUT` with full `allowedOrigins` array
Blob Storage CORS	`az storage cors add` with FrontDoor host + custom domain + localhost
AD redirect_uris	`az ad app update --web-redirect-uris` after Pulumi deploy
SVG assets	Copy `dist/*.svg` to `$web/sns/assets/` after `upload-batch`
Remove `.so` deletion step	Was deleting required C extension binaries

---

8. Refactoring & Infrastructure Fix (2026-02-22)

Branch: main (production) + develop (staging) **Source**: REFACTORING_REPORT_20260222.md

8-1. Azure Front Door — SPA URL Rewrite Rule

Problem: React SPA deep-links (`/sns/login` , etc.) returned HTTP 404. AFD was forwarding to the backend instead of serving `index.html` .

Fix: Added `SpaRuleSet` + `SpaIndexHtmlRewrite` rule to `infrastructure/pulumi/azure/__main__.py` . Rewrites all non-static-asset browser requests under `/sns/*` to `/sns/index.html` .

AFD Constraints:

- RuleSet name must be **alphanumeric only** (no hyphens) — e.g. `"SpaRuleSet"`
- **Maximum 10 `match_values` per condition** (Azure AFD Standard SKU limit)
- Pulumi class name: `UrlRewriteActionArgs` (NOT `DeliveryRuleUrlRewriteActionArgs`)
- Pulumi pending operations must be cleared before next `pulumi up` :

```
        pulumi stack export | jq '.deployment.pending_operations
= []' | pulumi stack import --force
```

8-2. CI/CD Workflow Cleanup

Removed 168 lines of dead steps from `deploy-aws.yml`, `deploy-azure.yml`, `deploy-gcp.yml` (old `frontend_web` Lambda/Function App build/deploy steps; React SPA now deployed exclusively by `deploy-frontend-web-*.yml`).

8-3. AWS Pulumi Dead Code Removal

Removed 10 dead resources (121 lines) from `infrastructure/pulumi/aws/__main__.py`:

- `frontend-web-function` Lambda, FunctionUrl, OAC, Permissions
- API GW routes `ANY /sns` and `ANY /sns/{proxy+}`
- CloudFront `frontend-web` origin (API GW endpoint)

8-4. Staging Bug: `ModuleNotFoundError: pulumi_azuread`

Cause: `infrastructure/pulumi/azure/requirements.txt` was missing `pulumi-azuread>=6.0.0` (accidentally deleted in a prior commit). **Fix**: Restored from `main` branch.

8-5. Staging Bug: `ModuleNotFoundError: monitoring`

Cause: `monitoring.py` existed in `main` but had never been committed to `develop`. **Fix**: Added `infrastructure/pulumi/{aws,azure,gcp}/monitoring.py` to `develop`.

8-6. GCP: `Error 412: Invalid fingerprint` on URLMap

Cause: Pulumi state out of sync with actual GCP resource state (fingerprint mismatch). **Fix**: Added `pulumi refresh --yes --skip-preview` before `pulumi up` in `deploy-gcp.yml`.

8-7. GCP: `Error 409: bucket already exists`

Cause: `gcp/__main__.py` in `develop` didn't define `uploads_bucket`; after syncing from `main`, Pulumi tried to create a bucket that already existed. **Fix**: Added `pulumi import` step in `deploy-gcp.yml` to import pre-existing bucket before `pulumi up`.

8-8. GCP: `Error 400: ssl_certificate already in use`

Cause: SSL certificate name included a hash that changed between `develop` and `main`, causing Pulumi to try replacing it while the old cert was still attached. **Fix**: Added `ignore_changes=["name", "managed"]` to `ManagedSslCertificate` in `gcp/__main__.py` + `pulumi import` for existing cert.

---

9. GCP SNS Fix Report (2026-02-23)

Environment: `https://staging.gcp.ashnova.jp/sns/` (staging) **Branch**: develop — commits `2385ee4` → `ec5bf05` **Source**: GCP_SNS_FIX_REPORT_20260223.md

#	Symptom	Root Cause	Fix
G1	CORS error on `/posts`	`CORS_ORIGINS` missing `staging.gcp.ashnova.jp`	Added custom domain to `CORS_ORIGINS` via `--env-vars-file`
G2	Firebase login “domain not authorized”	Custom domain not registered in Firebase Auth authorized domains	PATCH Identity Toolkit Admin v2 API; added auto-register step to `deploy-gcp.yml`
G3	`/limits` → 404 after login	Cloud Function deployed with stale code lacking `limits` route	Rebuilt with Docker `linux/amd64` and redeployed
G4	`signInWithPopup` COOP warning	CDN not sending `Cross-Origin-Opener-Policy` header	Added `Cross-Origin-Opener-Policy: same-origin-allow-popups` to CDN backend bucket
G5	`/uploads/presigned-urls` → 500	`generate_signed_url()` requires private key; Compute Engine credentials lack it	Use `service_account_email` + `access_token` to trigger IAM `signBlob` API path
G6			

#	Symptom	Root Cause	Fix
	Cloud Function build failure	`local_backend.py` missing `with self._get_connection()` block (IndentationError)	Restored `delete_post` method with correct `with` block

Key Technical Points

G2 — Firebase Auth Domain Registration

```
        # x-goog-user-project header is REQUIRED — without it, API returns 403 PERMISSION_DENIED
        curl -s -X PATCH ¥
          "https://identitytoolkit.googleapis.com/admin/v2/projects/ashnova/config" ¥
          -H "Authorization: Bearer $(gcloud auth print-access-token)” ¥
          -H "x-goog-user-project: ashnova" ¥
          -d '{"authorizedDomains": ["...", "staging.gcp.ashnova.jp"]}'
```

G3 — Cloud Function Rebuild (aarch64 → linux/amd64)

```
        docker run --rm --platform linux/amd64 -v /tmp/deploy_gcp:/out python:3.12-slim ¥
          bash -c "pip install --target /out/.deployment -r /out/requirements-gcp.txt -q"
        # CRITICAL: Copy main.py — Cloud Build requires it even if --entry-point differs
```

```
        cp services/api/function.py /tmp/deploy_gcp/.deployment/
main.py
```

G5 — GCS Presigned URLs from Compute Engine Credentials

```
        # Standard generate_signed_url() fails — Compute Engine
credentials have no private key
        # Solution: pass service_account_email + access_token to
trigger IAM signBlob API
        credentials.refresh(google.auth.transport.requests.Request())
        blob.generate_signed_url(
            version="v4",
            service_account_email=settings.gcp_service_account,  #
GCP_SERVICE_ACCOUNT env var
            access_token=credentials.token,
        )
        # Required IAM role: roles/iam.serviceAccountTokenCreator on
the Compute SA
```

G1 Gotcha — `--update-env-vars` Cannot Be Used for URLs

```
        # ☒Fails — values with ':' cause parse errors
        gcloud run services update --update-env-vars
"CORS_ORIGINS=https://..."

        # ☒Use --env-vars-file (YAML) instead — replaces ALL env
vars, include the full set
        gcloud run services update --env-vars-file env.yaml
```

---

10. OCR Formula Merge Bugs (2026-02-27)

Service: `services/api/app/services/` (API — all clouds) **Branch**: develop — commits `608f98f` `4fa3394` `cc0956b` **Source**: OCR_FORMULA_MERGE_REPORT.md

Background

`prebuilt-read` reproduces Japanese text faithfully but breaks formulas into ASCII fragments. `prebuilt-layout + FORMULAS` extracts accurate LaTeX but may miss Japanese text. A 2-pass merge strategy was implemented to combine both passes.

#	Symptom	Root Cause	Fix (commit)
1	Display formulas silently missing from `azure_di_merged` output	On polygon match failure, unmatched display formulas were discarded	Lowered overlap threshold 0.5→0.3; added Point-object polygon support; safety net appends unmatched display formulas at end (`608f98f`)
2	`TypeError: Object of type bytes is not JSON serializable`	Azure DI SDK (some versions) returns `polygon` , `content` , `value` ,	Added `isinstance(x, (bytes, bytearray))` guards in both `_ocr_read_pass` and `_ocr_layout_formulas_pass` (`4fa3394`)

#	Symptom	Root Cause	Fix (commit)
		`kind` as `bytes` type	
3	All polygons None after Bug 2 fix → formulas always appended at end	Bug 2 fix set bytes polygons to None; SDK returns polygons only as bytes in some environments	Added heuristic formula-region detection as fallback match (`cc0956b`)

Heuristic Formula Region Detection (fallback for Bug 3)

When polygon data is unavailable, `_find_formula_regions()` detects formula blocks by:

- No CJK characters in the line
- Line ≤ 80 chars
- At least one strong math signal: `[\\∞∫∑∏√]|lim|log|sin|cos|tan` or 2+ consecutive operators

Merge Strategy (final)

```
Pass 1: Y-polygon overlap ≥ 30%  → inline replace
Pass 2: _find_formula_regions() pairing with unmatched display formulas
```

```
Safety net: still-unmatched display formulas appended as [display] at e
Inline formulas: always appended as [inline]
```

Known Minor Issues

Issue	Detail
`ェ>0` instead of `x>0`	CJK lookalike OCR misread — post-processing normalization needed
`[display] \quad` false positive	`_has_formula_signal` overly broad — `\quad` should be blacklisted
`1/2` → `112`	Fraction OCR misread — no fix yet

Last updated: 2026-02-27

12 — OCR · 数式解答サービス

> Part IV — 機能リファレンス | 親: AI_AGENT_GUIDE.md
> カバー範囲: `/v1/solve` エンドポイント、 `AzureMathSolver` 、 `GcpMathSolver` 、OCR スコアリングパイプライン、環境変数、コスト制御、デバッグエンドポイント。

概要

数学解答サービスは、日本の大学入試問題に対して AI による解答を提供します。 問題画像（URL または base64）を受け取り、複数パス OCR を実行し、LaTeX と段階的な解法を含む構造化レスポンスを返します。

```
画像（URL または base64）
    │
    ▼
OCR Pass 1 ── Azure AI Document Intelligence prebuilt-read（日本語テキス
OCR Pass 2 ── Azure AI Document Intelligence prebuilt-layout+FORMULAS
    │             ── または ──
    │           GCP Cloud Vision API（日本語テキスト）
    │           GCP Gemini Vision（LaTeX 抽出）
    ▼
Merge ──────── 2-pass polygon/heuristic merge → 最良 OCR 候補を選択
    │
    ▼
LLM ─────────── Azure OpenAI（gpt-4o）または GCP Vertex AI（gemini-2.0-fla
    │
    ▼
SolveResponse  { problemText, answer.final, answer.steps, answer.latex,
```

対応クラウド: Azure と GCP のみ。 AWS 実装は削除済みで、 `501 Not Implemented` を返します。

機能トグル

コスト制御のため、このサービスはデフォルトで無効です。

```
        # 有効化（Lambda / Cloud Function / Cloud Run サービスで設定）
        SOLVE_ENABLED=true

        # 無効時レスポンス（HTTP 503）：
        {"detail": "solve endpoints are currently disabled
(SOLVE_ENABLED=false)"}
```

API エンドポイント

すべてのエンドポイントは `/v1` プレフィックス配下で、
`SOLVE_ENABLED=true` が必要です。

POST /v1/solve

画像から数学問題を解きます。

リクエストボディ（`SolveRequest`）：

```
{
  "input": {
    "imageUrl": "https://example.com/problem.jpg",
    "imageBase64": null,
    "source": "url"
  },
  "exam": {
    "university": "tokyo",
    "year": 2025,
    "subject": "math",
    "questionNo": ”1”
  },
  "options": {
    "mode": "fast",
    "needSteps": true,
    "needLatex": true,
    "maxTokens": 2000,
    "debugOcr": false
  }
}
```

フィールド	型	既定値	説明
`input.imageUrl`	string|null	—	問題画像の公開 URL。`imageBase64` と排他
`input.imageBase64`	string|null	—	base64 エンコード済み画像バイト列
`input.source`	`paste` | `upload` | `url`	`paste`	入力元ヒント
`exam.university`	string	`tokyo`	大学コード（参照 PDF 検索に使用）
`exam.year`	int|null	null	年度（参照 PDF 検索に使用）
`exam.subject`	string	`math`	科目コード
`exam.questionNo`	string|null	null	問題番号（例: `"1"`, `"2"`）
`options.mode`	`fast` | `accurate`	`fast`	OCR 戦略: `fast` は単一パス、`accurate` は参照 PDF 併用マルチパス
`options.needSteps`	bool	`true`	レスポンスに段階的解法を含める
`options.needLatex`	bool	`true`	レスポンスに LaTeX を含める
`options.maxTokens`	int 256–4096	`2000`	LLM 最大出力トークン数

フィールド	型	既定値	説明
`options.debugOcr`	bool	`false`	`meta.ocrDebugTexts` に生 OCR 候補を含める

レスポンスボディ（ `SolveResponse` ）：

```
{
  "requestId": "abc123",
  "status": "ok",
  "problemText": "lim_{n→∞} n∫_1^2 log(1 + x/n)^{1/2} dx を求めよ。”,
  "answer": {
    "final": ”答えは log(2)/2 です。”,
    "latex": ”¥¥frac{¥¥log 2}{2}",
    "steps": ["Step 1: ...", "Step 2: ..."],
    "confidence": 0.92
  },
  "meta": {
    "ocrProvider": "azure",
    "ocrSource": "azure_di_merged",
    "ocrScore": 1.3955,
    "ocrCandidates": 6,
    "model": "gpt-4o",
    "latencyMs": 4230,
    "costUsd": 0.0034
  }
}
```

GET /v1/ocr-debug?limit=20

Azure Blob Storage の `ocr-debug/ocr_debug.jsonl` から最新 N 件の OCR デバッグログを返します。 Blob が利用できない場合は `/tmp/ocr_debug.jsonl` にフォールバックします。

GET /v1/ocr-debug/diag

診断用エンドポイント。 `/tmp` 書き込み可否、stdout 動作、Azure Blob 接続性を確認します。 デバッグログパイプラインが正常に動作しているかを検証するために有用です。

OCR ソースとスコアリング

ソルバーは複数の OCR 候補テキストを生成し、最高スコアの候補を採用します。

OCR ソース	ボーナス	説明
`local_reference_pdf`	+0.44	ローカル参照 PDF から抽出したテキスト（最高品質）
`pdf_direct`	+0.34	URL 経由で取得した PDF から抽出したテキスト
`gcp_vision_api`	+0.30	Google Cloud Vision API
`azure_di_merged`	+0.30	Azure DI 2パスマージ: 日本語テキスト + in-place LaTeX 数式
`gcp_vision_merged`	+0.28	GCP Vision + Gemini Vision の数式マージ

OCR ソース	ボーナス	説明
`azure_di_read+formulas`	+0.26	Azure DI 日本語テキスト + 数式付録（フォールバック）
`azure_di_layout_markdown`	+0.12	Azure DI Markdown（LaTeX 精度は高いが日本語を取りこぼす場合あり）
`azure_di_read`	0.00	Azure DI プレーンテキスト（数式抽出なし）

選択されたソースは `meta.ocrSource` と `meta.ocrScore` に出力されます。

品質しきい値（`config.py`）：

```
        solve_ocr_review_min_score: float = 0.40       # これ未満は要レビューとしてフラグ
        solve_ocr_review_max_replacement_ratio: float = 0.01  # 置換文字率 >1% でフラグ
```

2パス数式マージ（azure_di_merged）

不具合履歴は AI_AGENT_11_BUG_FIX_REPORTS.md §10 を参照してください。

```
Pass 1: Y-polygon overlap ≥ 30% → 数式断片をインライン置換
Pass 2: _find_formula_regions() ヒューリスティック → 残り display 数式を対応
```

```
Safety net: 未対応の display 数式は末尾に［display］として追記
Inline formulas: 常に末尾に［inline］として追記
```

ヒューリスティックな数式領域検出（`_find_formula_regions`）：

- 行に CJK 文字が含まれない
- 行長が 80 文字以下
- 強い数式シグナルを少なくとも1つ含む: `[\\∞∫∑∏√]|lim|log|sin|cos|tan` または演算子2連続以上

環境変数

Azure Solver（AzureMathSolver）

変数	必須	説明
`SOLVE_ENABLED`	✓	エンドポイント有効化には `true` が必須
`AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT`	✓	Azure AI DI endpoint URL
`AZURE_DOCUMENT_INTELLIGENCE_KEY`	✓	Azure AI DI API key
`AZURE_OPENAI_ENDPOINT`	✓	Azure OpenAI endpoint URL
`AZURE_OPENAI_KEY`	✓	Azure OpenAI API key
`AZURE_OPENAI_DEPLOYMENT`		

変数	必須	説明
		LLM デプロイ名（既定: `gpt-4o`）
`AZURE_OPENAI_API_VERSION`		API バージョン（既定: `2024-12-01-preview`）
`AZURE_STORAGE_ACCOUNT_NAME`		OCR デバッグログ Blob Storage 用（任意）
`AZURE_STORAGE_ACCOUNT_KEY`		OCR デバッグログ Blob Storage 用（任意）

GCP Solver（GcpMathSolver）

変数	必須	説明
`SOLVE_ENABLED`	✓	エンドポイント有効化には `true` が必須
`GCP_SERVICE_ACCOUNT`	✓	署名付き URL 生成用サービスアカウントメール
`GCP_VERTEX_LOCATION`		Vertex AI リージョン（既定: `us-central1`）
`GCP_VERTEX_MODEL`		Gemini モデル（既定: `gemini-2.0-flash-001`）

変数	必須	説明
`GCP_VISION_API_KEY`		Vision API key。未設定時は ADC を使用

コスト制御用変数

変数	既定値	説明
`SOLVE_ENABLED`	`false`	グローバル kill switch。`false` で全コストを停止
`SOLVE_ALLOW_REMOTE_IMAGE_URL`	`true`	URL からの画像取得を許可（無効化で SSRF リスク抑制）
`SOLVE_MAX_IMAGE_BYTES`	`5242880` (5 MB)	最大画像サイズ。超過は 400 で拒否
`AZURE_OPENAI_DEPLOYMENT`	`gpt-4o`	より安価なモデルへ切替可能
`options.maxTokens`	`2000`	リクエストあたりを削減して LLM コスト圧縮

参照 **PDF** 検索

ソルバーは OCR 品質向上のため、元問題の参照 PDF を任意で取得できます。

検索順序（ `BaseMathSolver._resolve_reference_pdf_urls` ）:

1. ローカルパス（開発時のみ）から `{university}/{year}/{subject}/{questionNo}.pdf`
2. `exam.university` 、 `exam.year` 、 `exam.subject` 、 `exam.questionNo` から導出したリモート URL

参照 PDF は権威ソース OCR 候補（ `local_reference_pdf` または `pdf_direct` ）として扱われ、最高ボーナス（+0.44 または +0.34）を加算します。

ソースファイル

ファイル	説明
`services/api/app/routes/solve.py`	FastAPI ルーター: `/v1/solve` , `/v1/ocr-debug` , `/v1/ocr-debug/diag`
`services/api/app/services/base_math_solver.py`	基底クラス: 画像解決、OCR スコアリング、数式マージ、PDF 抽出
`services/api/app/services/azure_math_solver.py`	Azure DI + Azure OpenAI 実装
`services/api/app/services/gcp_math_solver.py`	GCP Vision + Gemini 実装
`services/api/app/models.py`	`SolveRequest` , `SolveResponse` , `SolveInput` , `SolveExam` , `SolveOptions` , `SolveAnswer` , `SolveMeta`

ファイル	説明
`services/api/app/config.py`	`solve_*`, `azure_*`, `gcp_vertex_*` の全設定

クイックテスト

```
# ローカルで有効化
export SOLVE_ENABLED=true
export AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT=https://...
export AZURE_DOCUMENT_INTELLIGENCE_KEY=...
export AZURE_OPENAI_ENDPOINT=https://...
export AZURE_OPENAI_KEY=...

# 最小 curl リクエスト（画像 URL）
curl -s -X POST http://localhost:8000/v1/solve ¥
  -H "Content-Type: application/json" ¥
  -d '{
    "input": {"imageUrl": "https://example.com/problem.jpg",
"source": "url"},
    "exam": {"university": "tokyo", "year": 2025,
"questionNo": "1"},
    "options": {"mode": "fast", "debugOcr": true}
  }' | python3 -m json.tool

# 最新 OCR デバッグログを確認
curl -s "http://localhost:8000/v1/ocr-debug?limit=5" |
python3 -m json.tool

# Staging（Azure）
STAGING_API="https://multicloud-auto-deploy-staging-func-
d8a2guhfere0etcq.japaneast-01.azurewebsites.net/api"
```

```
        curl -s "${STAGING_API}/v1/ocr-debug/diag" | python3 -m
 json.tool
```

AWS 非対応の理由

`CLOUD_PROVIDER=aws` の場合、 `/v1/solve` は **HTTP 501 Not Implemented** を返します。 これは一時的な欠落ではなく、意図的なアーキテクチャ判断です。

1. AWS Textract に LaTeX 数式抽出機能がない（最大の障壁）

本サービス OCR パイプラインの核心は、**Azure Document Intelligence `prebuilt-layout` + FORMULAS** による LaTeX 数式の in-place 抽出です。

機能	**Azure DI**	**AWS Textract**	影響
日本語テキスト認識	✓ `prebuilt-read`	△　対応はしているが精度が低い	問題文の誤読
数式を LaTeX で出力	✓ `kind=display/inline` + `value` (LaTeX)	✗ 対応なし	数式ゼロが極まる
ポリゴン座標付き数式	✓ `bounding_regions[0].polygon`	✗	in-place マージ不可
	✓ 2パス並列で両立	✗	

機能	**Azure DI**	**AWS Textract**	影響
日本語 + 数式の同時抽出			パイプライン全体が崩壊

AWS Textract の Equations 機能（2023年追加）は LaTeX を返さず、数式領域のバウンディングボックスのみを提供するため、本サービスの `_merge_read_with_formulas()` が動作しません。

2. API Gateway の 29 秒応答タイムアウト

AWS Lambda 自体の最大実行時間は 15 分ですが、**API Gateway**（**REST API / HTTP API**）の統合タイムアウトは最大 29 秒で変更不可です。

本サービス `accurate` モードの処理時間目安:

フェーズ	所要時間目安
Azure DI 2パス並列 OCR	8〜20 秒
参照 PDF ダウンロード+抽出	3〜10 秒
Azure OpenAI gpt-4o 推論	5〜20 秒
合計（**accurate mode**）	16〜50 秒

`fast` モードでも参照 PDF なしで 15〜30 秒かかる場合があります。 API Gateway 29 秒制限により、**accurate** モードは **AWS** では構造的に実現不可能です。

> Lambda Function URL を直接使えばタイムアウト回避は可能ですが、本プロジェクトではマルチクラウド構成上 API Gateway を必須インフラとしており、迂回構成は採用しません。

3. gpt-4o と Gemini は AWS Bedrock で提供されていない

本サービスで使用している LLM:

クラウド	**LLM**	特徴
Azure	Azure OpenAI `gpt-4o`	日本語数学記述・LaTeX 出力品質が高水準
GCP	Vertex AI `gemini-2.0-flash-001`	マルチモーダル（画像入力直接対応）・高速

AWS Bedrock が提供するのは Claude、Llama、Mistral、Titan などです。 `gpt-4o` は Microsoft/OpenAI 提供のため AWS Bedrock では利用できません。 `Gemini` シリーズも Google Cloud 提供のため AWS Bedrock では利用できません。

Claude（Bedrock）で代替実装すること自体は可能ですが、評価済みプロンプト・スキーマ（ `SolveAnswer.steps` 、 `SolveAnswer.latex` 、 `diagramGuide` 等）の再調整が必要で、品質保証の観点から採用していません。

4. 設計経緯 — 以前は Bedrock フォールバックがあった

コードコメントにあるとおり、初期設計では `BaseMathSolver` に Bedrock フォールバックが実装されていました:

```
        # services/api/app/services/gcp_math_solver.py (ドキュメントコメント)
        # フォールバック設計:
        #   - Vision API 未設定 → Bedrock マルチモーダル OCR (親クラス)
        #   - Vertex AI 未設定  → Bedrock LLM (親クラス)
```

しかし次の理由で `BaseMathSolver` から削除され、現在は "Shared utilities only — no AWS/Bedrock clients" になっています。

- Textract では数式抽出が機能せず、OCR スコアが常に最低値になる
- API Gateway タイムアウトにより accurate mode が完走しない
- LLM 品質（Claude vs gpt-4o）の差が日本語数学問題で顕著

5. まとめ — AWS で solve を動かすために必要な変更

将来 AWS 対応を検討する場合に必要な全体像:

課題	必要な対応
LaTeX OCR	Textract の代替として外部 OCR API（例: Mathpix）を統合
タイムアウト	Lambda Function URL + API Gateway バイパス、または非同期ジョブキュー化
LLM	

課題	必要な対応
	Claude Sonnet/Haiku 向けにプロンプト再調整（品質検証が必要）
数式マージ	`_merge_read_with_formulas()` の入力形式変更（ポリゴン付き数式データが必要）

現状は 対応予定なし。AWS で数学 OCR が必要な場合は Azure または GCP へルーティングする設計を推奨します。

既知の制約と今後の改善

項目	詳細
AWS 未対応	AWS の `/v1/solve` は 501 を返す。詳細は上記「AWS 非対応の理由」参照
CJK-Latin OCR 混同	`ェ` が `x` と誤読される。後処理正規化は未対応
`[display]` `\quad` の誤検出	`_has_formula_signal` が広すぎるため、`\quad` のブラックリスト化が必要
分数 OCR	`1/2` → `112` の誤読。構造認識は未実装
GCP Vision merged	`gcp_vision_merged` は新規ソース（2026-02-27）で、大規模検証は未実施
コストトラッキング	`meta.costUsd` はトークン数ベースの推定値で、実課金値ではない

最終更新: 2026-02-27

13 — テスト

> Part IV — Feature Reference | Parent: AI_AGENT_GUIDE.md
> カバレッジ: 全テストスクリプト、pytest スイート、トークン取得、ローカルスタック設定、CI テスト統合。 詳細な pytest 情報: INTEGRATION_TESTS_GUIDE.md ステージングエンドポイントとトークン設定: STAGING_TEST_GUIDE.md

---

テストマップ

スクリプト / ファイル	タイプ	スコープ	認証必要
`scripts/test-sns-local.sh`	E2E shell	ローカル docker-compose スタック	不要
`scripts/test-sns-aws.sh`	E2E shell	AWS staging SNS アプリ	オプション
`scripts/test-sns-azure.sh`	E2E shell	Azure staging SNS アプリ	オプション
`scripts/test-sns-gcp.sh`	E2E shell	GCP staging SNS アプリ	オプション
`scripts/test-sns-all.sh`	E2E shell	全3クラウド順次実行	オプション

スクリプト / ファイル	タイプ	スコープ	認証必要
`scripts/test-staging-all.sh`	E2E shell	全3クラウド + サマリーレポート	オプション
`scripts/test-e2e.sh`	Smoke	全3クラウド health + CRUD	オプション
`scripts/test-endpoints.sh`	Health	クラウド別 API ヘルスチェック	不要
`scripts/test-landing-pages.sh`	Health	クラウド別静的ランディングページ	不要
`scripts/test-api.sh`	API	クラウド別 CRUD 操作	オプション
`scripts/test-auth-crud.sh`	Auth	完全な認証 + CRUD フロー	必要
`scripts/test-cloud-env.sh`	Config	環境変数とクラウド接続	不要
`scripts/test-local-env.sh`	Config	ローカル docker-compose 環境チェック	不要
`scripts/test-deployments.sh`	Deploy	デプロイメントヘルスチェック	不要
`scripts/test-cicd.sh`	CI/CD		不要

スクリプト / ファイル	タイプ	スコープ	認証必要
		ワークフロートリガー + ステータスチェック	
`scripts/run-integration-tests.sh`	pytest	pytest ランナー (全バックエンド)	不要
`services/api/tests/test_backends_integration.py`	pytest	バックエンドクラス単体テスト (モック)	不要
`services/api/tests/test_api_endpoints.py`	pytest	ライブ API エンドポイントテスト	オプション
`services/api/tests/test_simple_sns_local.py`	pytest	ローカル docker-compose 統合	不要

クイックコマンド

1 — 最速: 全クラウドヘルスチェック

```
./scripts/test-endpoints.sh
```

2 — E2E スモークテスト (全3クラウド、パブリックエンドポイント)

```
./scripts/test-e2e.sh
# 出力: クラウド別 pass/fail テーブル
```

3 — クラウド別完全 SNS テスト (パブリックエンドポイントのみ)

```
./scripts/test-sns-aws.sh
./scripts/test-sns-azure.sh
./scripts/test-sns-gcp.sh

# 全3つを1コマンドで:
./scripts/test-sns-all.sh
```

4 — 認証付き完全 SNS テスト

```
# まずトークンを取得（下記「トークン取得」セクション参照）
./scripts/test-sns-aws.sh    --token "$AWS_TOKEN"
./scripts/test-sns-azure.sh  --token "$AZURE_TOKEN"
./scripts/test-sns-gcp.sh    --token "$GCP_TOKEN"

# トークン付き全3クラウド:
./scripts/test-staging-all.sh ¥
  --aws-token   "$AWS_TOKEN"   ¥
  --azure-token "$AZURE_TOKEN" ¥
  --gcp-token   "$GCP_TOKEN"
```

5 — ローカル docker-compose スタックテスト

```
# まずスタックを起動
docker compose up -d --build
sleep 30   # 全サービスが起動するまで待つ

./scripts/test-sns-local.sh
# または pytest:
```

```
        cd services/api && pytest tests/test_simple_sns_local.py -v
-m local
```

6 — pytest (unit + バックエンドモックテスト — ネットワーク不要)

```
        cd services/api
        # 全モックテスト
        pytest tests/ -v

        # 単一クラウドバックエンド
        pytest tests/ -v -m aws
        pytest tests/ -v -m gcp
        pytest tests/ -v -m azure

        # カバレッジ付き
        pytest tests/ --cov=app --cov-report=html
        # → htmlcov/index.html を開く
```

7 — デプロイ済みライブ API に対する pytest

```
        cd services/api
        export AWS_API_ENDPOINT="https://z42qmqdqac.execute-api.ap-
northeast-1.amazonaws.com"
        export AZURE_API_ENDPOINT="https://multicloud-auto-deploy-
staging-func-
d8a2guhfere0etcq.japaneast-01.azurewebsites.net/api"
        export GCP_API_ENDPOINT="https://multicloud-auto-deploy-
staging-api-son5b3ml7a-an.a.run.app"
        pytest tests/test_api_endpoints.py -v -m requires_network
```

トークン取得

認証テストには、各クラウドの ID プロバイダーからのベアラートークンが必要です。

AWS — Cognito アクセストークン

```
        # ブラウザ経由: https://d1tf3uumcm4bo1.cloudfront.net/sns/ でログイン
        # DevTools → Application → Local Storage → origin → access_token

        # AWS CLI 経由（メール/パスワード）:
        aws cognito-idp initiate-auth ¥
          --auth-flow USER_PASSWORD_AUTH ¥
          --client-id 1k41lqkds4oah55ns8iod30dv2 ¥
          --auth-parameters USERNAME=<email>,PASSWORD=<password> ¥
          --region ap-northeast-1 ¥
          --query 'AuthenticationResult.AccessToken' --output text
```

Azure — Azure AD id_token

```
        # ブラウザ経由: https://staging.azure.ashnova.jp/sns/ でログイン
        # DevTools → Application → Local Storage → id_token
```

GCP — Firebase id_token

```
        # gcloud 経由（ログイン済みの Google アカウントを使用 — 登録ユーザーである必要あり）:
        gcloud auth print-identity-token
```

```
# ブラウザ経由: https://staging.gcp.ashnova.jp/sns/ でログイン
# DevTools → Application → Local Storage → id_token
```

ローカル Docker Compose スタック

完全なローカル開発スタックは、リポジトリルートの `docker-compose.yml` で定義されています。

```
docker compose up -d --build    # 全サービス起動
docker compose ps               # 全コンテナが Up か確認
docker compose logs api -f      # API ログをストリーム
docker compose down             # 全サービス停止
```

起動されるサービス:

サービス	ポート	説明
`api`	8000	FastAPI バックエンド (DynamoDB Local + MinIO)
`frontend_web`	8080	Python/FastAPI SSR フロントエンド (Jinja2)
`minio`	9000	S3 互換オブジェクトストレージ
`dynamodb-local`	8001	DynamoDB Local
`frontend_react`	3001	React SPA (nginx, `/sns/`)
`static_site`	8090	静的ランディングページ (nginx proxy)

docker-compose で自動設定される主要環境変数:

```
AUTH_DISABLED=true
CLOUD_PROVIDER=local
DYNAMODB_ENDPOINT=http://dynamodb-local:8001
MINIO_ENDPOINT=http://minio:9000
```

---

pytest マーカー

マーカー	アクティブ化条件	説明
`aws`	常時	AWS バックエンド固有テスト
`gcp`	常時	GCP バックエンド固有テスト
`azure`	常時	Azure バックエンド固有テスト
`local`	常時	ローカル docker-compose 統合テスト
`requires_network`	`--run-network-tests` または環境変数設定	ライブ API エンドポイントを呼び出すテスト
`slow`	`--run-slow-tests`	5秒以上かかるテスト

ステージングエンドポイント (リファレンス)

クラウド	**API** エンドポイント	フロントエン
AWS staging	`https://z42qmqdqac.execute-api.ap-northeast-1.amazonaws.com`	`https://d1tf3uumcmsns/`
Azure staging	`https://multicloud-auto-deploy-staging-func-d8a2guhfere0etcq.japaneast-01.azurewebsites.net/api`	`https://staging.az`
GCP staging	`https://multicloud-auto-deploy-staging-api-son5b3ml7a-an.a.run.app`	`https://staging.gc`
AWS production	`https://qkzypr32af.execute-api.ap-northeast-1.amazonaws.com`	`https://wsns/`
GCP production	(同じ Cloud Run URL)	`https://wsns/`

> 本番前に必ずステージングをテストしてください。現在のヘルス状態については AI_AGENT_06_STATUS.md を参照してください。

CI/CD テスト統合

テストは GitHub Actions 経由で push ごとに自動実行されます:

ワークフロー	トリガー	実行されるテスト
`deploy-aws.yml`	`develop` / `main` への push	デプロイ後 `./scripts/test-endpoints.sh`
`deploy-azure.yml`	`develop` / `main` への push	デプロイ後 `./scripts/test-endpoints.sh`
`deploy-gcp.yml`	`develop` / `main` への push	デプロイ後 `./scripts/test-endpoints.sh`
`run-integration-tests.sh`	手動または `test-staging-all.sh` 経由	pytest + E2E shell

トラブルシューティング

問題	原因	修正方法
`pytest: command not found`	Python 環境が有効化されていない	`source .venv/bin/activate` または `pip install pytest`
`ImportError: No module named 'app'`	作業ディレクトリが間違っている	`cd services/api` してから pytest を実行
ローカルテストで `Connection refused`	docker-compose が起動していない	`docker compose up -d && sleep 30`
認証テストで 401	トークン期限切れ	トークンを再取得（トークンは約1時間で期限切れ）

問題	原因	修正方法
ステージングで503	サービスコールドスタート	30秒待って再試行；AI_AGENT_06_STATUS.md を確認

最終更新: 2026-02-27